



Smart contract vulnerabilities, tools, and benchmarks: An updated systematic literature review

Gerardo Iuliano *, Dario Di Nucci

University of Salerno, Italy

ARTICLE INFO

Editor: Dr. Lingxiao Jiang

Keywords:

Smart contracts
Vulnerabilities
Benchmarks
Systematic literature review

ABSTRACT

Smart contracts are self-executing programs on blockchain platforms like Ethereum, which have revolutionized decentralized finance by enabling trustless transactions and the operation of decentralized applications. Despite their potential, the security of smart contracts remains a critical concern due to their immutability and transparency, which expose them to malicious actors. Numerous solutions for vulnerability detection have been proposed, but it is still unclear which one is the most effective. This paper presents a systematic literature review that explores vulnerabilities in Ethereum smart contracts, focusing on automated detection tools and benchmark evaluation. We reviewed 3380 studies from five digital libraries and five major software engineering conferences, applying a structured selection process that resulted in 222 high-quality studies. The key results include a hierarchical taxonomy of 192 vulnerabilities grouped into 13 categories, a comprehensive list of 219 detection tools with corresponding functionalities, methods, and code transformation techniques, a mapping between our taxonomy and the list of tools, and a collection of 133 benchmarks used for tool evaluation. We conclude with a discussion about the insights into the current state of Ethereum smart contract security and directions for future research.

1. Introduction

Since Bitcoin was introduced in 2009 (Nakamoto, 2008), blockchain and decentralized solutions have gained considerable attention and adoption (Alharby et al., 2018). Although blockchain was initially associated with cryptocurrencies, its potential has expanded, with smart contracts representing a notable and promising application. Smart contracts (SCs) are self-executing programs that run on blockchain platforms such as Ethereum, where transactions are automatically enforced without intermediaries. Originally, the term *smart contract* referred to the concept of automating traditional legal contracts using software (Szabo, 1997). However, they have been adopted in many other domains such as finance, healthcare care and supply chain management by ensuring trustless computation. Smart contracts are pivotal in decentralized systems due to their inherent transparency properties, tamper resistance, and immutability.

In decentralized finance (DeFi), smart contracts form the backbone of financial protocols, allowing decentralized applications (dApps) to facilitate lending, borrowing, trading, and other financial services without traditional intermediaries (Chaliasos et al., 2024). Ethereum, the most widely used blockchain for DeFi and dApps, supports a vast ecosystem of smart contracts managing valuable assets, including fungible and non-

fungible tokens (NFTs) (Wang et al., 2021c). This surge in decentralized financial activity has attracted significant attention from developers and attackers, with billions of dollars now at stake.

Despite their potential, smart contracts face many challenges, particularly regarding security (Zou et al., 2019). Developing secure smart contracts is inherently difficult due to platform-specific limitations, complex business logic, and the immutability of deployed contracts. Once a contract is deployed on-chain, any error, weakness, or vulnerability becomes permanent, making smart contracts appealing targets for malicious actors. The transparency of the blockchain allows attackers to analyze and exploit contracts, often leading to high-stakes breaches and financial losses in DeFi protocols. Furthermore, the interconnected nature of smart contracts, where contracts can interact and trigger each other, adds another layer of complexity, making vulnerabilities harder to detect and mitigate.

Existing taxonomies like DASP top 10 or SWC Registry are open and collaborative projects that join efforts to discover smart contract vulnerabilities within the security community. The main issue is their relevance due to outdatedness; in fact, the latest update of the DASP Top 10 is from 2018, while the SWC Registry is from 2020. The OWASP Smart Contract Top 10 (2025) is the only updated taxonomy to list the top 10 vulnerabilities found in smart contracts. Unfortunately, it

* Corresponding author.

E-mail addresses: geiuliano@unisa.it (G. Iuliano), ddinucci@unisa.it (D. Di Nucci).

<https://doi.org/10.1016/j.jss.2026.112788>

Received 19 May 2025; Received in revised form 18 November 2025; Accepted 14 January 2026

Available online 4 February 2026

0164-1212/© 2026 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

comprises only the top 10 vulnerabilities. In recent years, several studies have revealed new vulnerabilities that affect smart contracts. Fang et al. (2023) conducted a security analysis of the modifiers used in real-world Ethereum smart contracts and identified insecure modifiers that can be bypassed from one or more unprotected smart contract functions. Huang et al. (2024a) identified eight general patterns of library misuse: three arise during library development and five during library utilization, covering the entire library lifecycle. Library misuse refers to incorrect implementation or improper use of libraries, which can result in unexpected results, such as the introduction of vulnerabilities or the inclusion of unsafe libraries. The latest taxonomies do not collect recent updates. Furthermore, we observed several discrepancies between them in terms of categories adopted to classify vulnerabilities and synonyms used to refer to the same vulnerabilities. In particular, focusing on the most recent and comprehensive taxonomies such as those proposed by Rameder et al. (2022), Li et al. (2024c), and Vidal et al. (2024a) (OpenSCV), we observed that the categories used to organize vulnerabilities are different between taxonomies. Different taxonomies share a few categories, and each taxonomy uses a different approach to classify and organize vulnerabilities. The intersection of categories in different taxonomies does not provide a comprehensive set of criteria to categorize all vulnerabilities reported in the literature. Each taxonomy has a characteristic that distinguishes it from the other. In addition, synonyms or alternative names of vulnerabilities are used to identify different vulnerabilities. For example, OpenSCV (Vidal et al., 2024a) assigns the same synonym to multiple distinct vulnerabilities, creating a core issue when consulting a taxonomy. Especially when performing reverse engineering from alternative names, it becomes unclear which vulnerability is actually being referenced. Moreover, the growing adoption of dApps has led to the emergence of new types of vulnerability, particularly in multiblockchain environments. As systems increasingly integrate and facilitate communication between two or more blockchains, they introduce a new class of risks known in the literature as cross-system vulnerabilities. In particular, several studies (Duan et al., 2023a; Lee et al., 2023; Zhang et al., 2022a) have identified a range of problems associated with cross-chain technology. A comprehensive taxonomy is essential for vulnerability awareness, but its construction is complicated by the lack of clarity and agreement in existing studies.

Based on the challenges affecting current taxonomies and the emerging vulnerabilities discussed in recent studies, we defined the first research question: *Which vulnerabilities are mentioned? How are vulnerabilities classified?* The result of RQ_1 is a structured and hierarchical taxonomy that collates vulnerabilities into 14 different categories. Vulnerabilities are posed on different levels based on their level of abstraction and the details provided about the cause and condition in which they arise. The first level contains 120 vulnerabilities, the second contains 66, while the third contains six.

The research on vulnerabilities goes hand in hand with the research on vulnerability detection. Several studies have proposed novel solutions to detect vulnerabilities automatically, analyze gas and resource consumption, exploit vulnerabilities, provide reactive defenses, and repair smart contracts. The number of different methods adopted when implementing a tool is significantly high (symbolic execution, taint analysis, fuzzing, formal methods, etc.), along with the different approaches used to represent and analyze the source code (Abstract Syntax Tree, Control Flow Graph, Data Flow Graph, Transaction trace, etc). Each time a new tool is proposed, it improves the current state-of-the-art performance or achieves a new goal. For example, Li et al. (2024d) proposed a tool capable of finding two previously undisclosed vulnerabilities, i.e., CVE-2023-36979 and CVE-2023-36980. Given the high number of vulnerabilities and the different implementable solutions, we explore the landscape of tools proposed to address problems affecting smart contracts. We formulated the second research question: *Which methods do automated tools use to detect vulnerabilities?* As a result, we collected 219 automated analysis tools.

For each tool, we have collected its main functionality, the method, and the code transformation technique adopted. Given that each vulnerability has its unique characteristics and poses distinct challenges for effective detection. To identify which vulnerabilities are most actively studied and targeted, we mapped the vulnerabilities included in our taxonomy to the detection tools designed to address them. The third research question *Which vulnerabilities are addressed by the tools?* shows that the focus is on a narrow set of 79 vulnerabilities, and many of them remain unexplored. We found that some vulnerabilities, such as Price oracle manipulation, are not detectable by existing static analysis tools and require a deeper understanding of the contract by analyzing natural language from a semantic perspective (Zhang et al., 2023d). In addition, we found that tools often omit their compatibility with the Solidity version, complicating their effectiveness comparison.

To close the loop, we decided to collect information about the evaluation of tools, and then we formulated the fourth research question: *How are the tools evaluated?* We collected a set of 133 benchmarks used for tool evaluation or to conduct other experiments. About half of them are open source, while the most widely used is SmartBugs-wild, which consists of mainly smart contracts written using Solidity 0.4.x.

Our study updates the systematic literature review conducted by Rameder et al. (2022). They analyzed more than 300 studies published up to 2021, examining the classification of vulnerabilities, detection methods, security analysis tools, and benchmarks for evaluating tools. Their work laid the foundation for systematically understanding the complexities of smart contract vulnerabilities, the tools available to address them, and the open challenges and gaps in automated detection systems. We have overcome previous limitations such as the absence of a structured snowballing process, a comprehensive mapping between tools and vulnerabilities, and the unchecked benchmarks for duplicates. We answered four research questions to provide an updated and more complete perspective on Ethereum vulnerabilities, detection tools, and benchmarks. We analyzed five digital libraries and five main conferences on software engineering and conducted two snowballing processes to collect a total of 3380 papers. We then applied a carefully documented process to select and evaluate 332 studies of immediate relevance. Quality appraisal reduced the analysis to 222 studies with sufficient intrinsic and contextual data quality that allowed us to answer our RQs. As a result, we realized a structured taxonomy that organizes hierarchically 192 vulnerabilities in 13 categories, a list of 219 automated detection tools with relative functionalities, methods, and code transformation techniques, a mapping between our taxonomy and the list of tools, and a list of 133 benchmarks used for tool evaluation. We provide the following contributions:

- A taxonomy of vulnerabilities comprising 192 vulnerabilities hierarchically organized on three levels and into 13 categories, serving as a comprehensive aggregation of the most classifications.
- A list of 219 tools used in the literature to address various problems concerning SCs, providing their functionalities along with the method and code transformation technique they apply.
- A mapping between our taxonomy and the tools that clarify the research focus and gaps.
- A list of 133 benchmarks used for tool evaluation, tool comparison, or performing experiments.

The paper is structured as follows. Section 2 describes the background of Ethereum SCs and their vulnerabilities. Section 3 exposes the research method and design adopted in our study and describes all the steps followed to answer our research questions. The results of each research question are reported in Section 4. Section 5 is dedicated to discussing our findings compared to the Rameder et al. (2022) and lists the research opportunities. Section 6 defines the threats to validity, Section 7 discusses the related work, highlighting the gaps and our contribution, and finally Section 8 concludes the paper.

2. Background

This section presents preliminary information on Ethereum, smart contracts, and the vulnerabilities associated with them.

2.1. Ethereum

Ethereum is a decentralized open source blockchain platform that facilitates the creation and execution of smart contracts and dApps (Buterin et al., 2014). Proposed by Vitalik Buterin in late 2013 (Buterin et al., 2013), Ethereum was designed to extend blockchain technology beyond simple data storage to include code execution, thus creating a versatile platform for developers. Unlike Bitcoin (Nakamoto, 2008), which works primarily as a digital currency, Ethereum allows the development and deployment of dApps. At its core, Ethereum features the Ethereum Virtual Machine (EVM), a decentralized virtual machine that executes code on the Ethereum network. This platform supports a near Turing-complete programming language, enabling complex computations and dApps. A key feature of Ethereum is its use of three Merkle Patricia (Liu et al., 2021a) trees to maintain blockchain state information, which facilitates efficient and verifiable data queries. Ether (ETH), Ethereum's native cryptocurrency, powers transactions and rewards miners, ensuring the network's security and functionality.

2.2. Smart contracts

The idea of smart contracts, which Nick Szabo initially introduced in 1994 (Szabo, 1994), has found a new life on the Ethereum blockchain. Smart contracts in Ethereum are self-executing contracts with terms directly written in the code (Antonopoulos and Wood, 2018). These contracts run on the Ethereum network, a decentralized platform that eliminates the need for intermediaries by automating contract execution. Each smart contract has a unique address and is initiated by a transaction between a sender and the contract. Execution incurs a computational cost measured in gas, which regulates resource usage and rewards miners. Developers write smart contracts in Solidity (Dannen, 2017) or Vyper (Kaleem et al., 2020), two languages explicitly designed for this purpose. Once deployed on the blockchain, these contracts become immutable and tamper-proof (Kaushal et al., 2021). They automatically execute predefined conditions, ensuring trust and reliability. Smart contracts enable seamless collaboration, enforce contract clauses, and are pivotal for developing dApps on the Ethereum platform.

2.3. Smart contract vulnerabilities

Smart contracts on the Ethereum blockchain offer significant benefits but are susceptible to vulnerabilities, further compounded by blockchain technology's immutable nature (Politou et al., 2019), stemming from coding errors and bugs in software development. Immutability ensures transparency and trust, but makes code defects permanent. Fixing such defects requires creating new contracts and user migration, which is a complex and risky task, underscoring the importance of exhaustive security measures. For upgradeable contracts that are not transparent to the user, the user will know their upgrades directly. For upgradeable contracts that are transparent to the user, they may be upgraded secretly, changing the functionality without the user's knowledge. Huang et al. (2024b) studied upgradeable smart contracts in Ethereum, providing information on two common techniques used to upgrade SC: the *proxy pattern* and the *metamorphic contract factory*. The *proxy pattern* is a design approach in smart contracts in which a proxy contract forwards all calls to a separate implementation (logic) contract using the *delegate call* function. Unlike a normal call, the *delegate call* executes the logic contract's code in the context of the proxy, allowing access to the proxy's storage. This enables upgradable smart contracts, as the logic

can be changed without altering the proxy's address or storage. The *metamorphic contract factory* is a mechanism that creates metamorphic contracts, contracts that can be redeployed with new code at the same address. This is achieved using the *CREATE2* opcode, which allows the address of a contract to be precomputed before deployment. After the original contract self-destructs, the factory can deploy a new version at the same address. However, this process resets the contract's storage, replacing it entirely with the new implementation's code. Smart contracts can contain mistakes once deployed, potentially leading to unintended behaviors or exploitations.

The *reentrancy* attack (Rameder et al., 2022; Samreen and Alalfi, 2020; Vidal et al., 2024a; Li et al., 2024c) stands out among these vulnerabilities, where a contract calls another before updating its state, opening the door to recursive calls that could drain funds or disrupt the intended execution. Furthermore, smart contracts often interact with external contracts and data sources, introducing additional vulnerabilities if these interactions are not checked appropriately.

Timestamp Dependency (Rameder et al., 2022; Chu et al., 2023; Wu et al., 2024a; Vidal et al., 2024a; Li et al., 2024c) refers to a vulnerability in smart contracts where the contract's logic relies on the *block.timestamp* (or *now*) value to make critical decisions, such as triggering payouts, closing auctions, or generating randomness. Since miners have limited control over the block timestamp (within a reasonable range), this can be exploited to manipulate the contract's behavior, especially in scenarios involving small time windows or incentives.

Authorization through tx.origin (Rameder et al., 2022; Vidal et al., 2024a; Li et al., 2024c) is a security vulnerability that occurs when a smart contract uses *tx.origin* to check the identity of the caller for access control. The *tx.origin* returns the original external account that initiated the transaction, even if the call was forwarded through multiple contracts. This can be exploited in phishing-style attacks, where a malicious contract tricks a user into calling it. Then that contract calls the vulnerable contract using the user's address as *tx.origin*, bypassing the intended access control. Instead, developers should use *msg.sender* for secure authorization, as it correctly reflects the immediate caller of the function.

Integer Overflow and Underflow (Rameder et al., 2022; Sun et al., 2022b; Kushwaha et al., 2022b; Agarwal et al., 2021; Munir and Taha, 2023; Vidal et al., 2024a; Li et al., 2024c) occur when arithmetic operations on unsigned integers exceed their maximum or minimum limits. An overflow occurs when a value exceeds the maximum limit (e.g., *uint256* going above $2^{256} - 1$), causing it to wrap around to zero. An underflow happens when a value goes below zero (e.g., subtracting 1 from *uint256(0)*), wrapping around to the maximum value. These bugs can lead to unexpected behavior, such as bypassing balance checks or locking/unlocking tokens incorrectly. Starting with Solidity 0.8.0, these operations automatically revert on overflow or underflow, helping prevent such vulnerabilities by default.

Vulnerable delegatecall (Rameder et al., 2022; Zaazaa and Bakkali, 2023; Zaazaa and El Bakkali, 2023; Dia et al., 2021; Vidal et al., 2024a; Li et al., 2024c) arises when a smart contract uses *delegatecall* to execute external code without properly validating the target address or ensuring the safety of the called contract. Since *delegatecall* runs the external contract's code in the context of the caller, it has full access to the caller's storage and permissions. If misused, an attacker can trick the contract into executing malicious logic, potentially leading to loss of control over the contract state or funds. To avoid this vulnerability, *delegatecall* should only be used with trusted, well-audited code, and dynamic or user-controlled addresses should never be passed as targets.

Unprotected selfdestruct (Rameder et al., 2022; Staderini, 2022; Zhou et al., 2022; Vidal et al., 2024a; Li et al., 2024c) is a vulnerability that occurs when a smart contract allows anyone, or unauthorized users, to trigger the *selfdestruct* function. This opcode removes the contract from the blockchain and forwards its remaining ether balance to a specified address. If not properly restricted

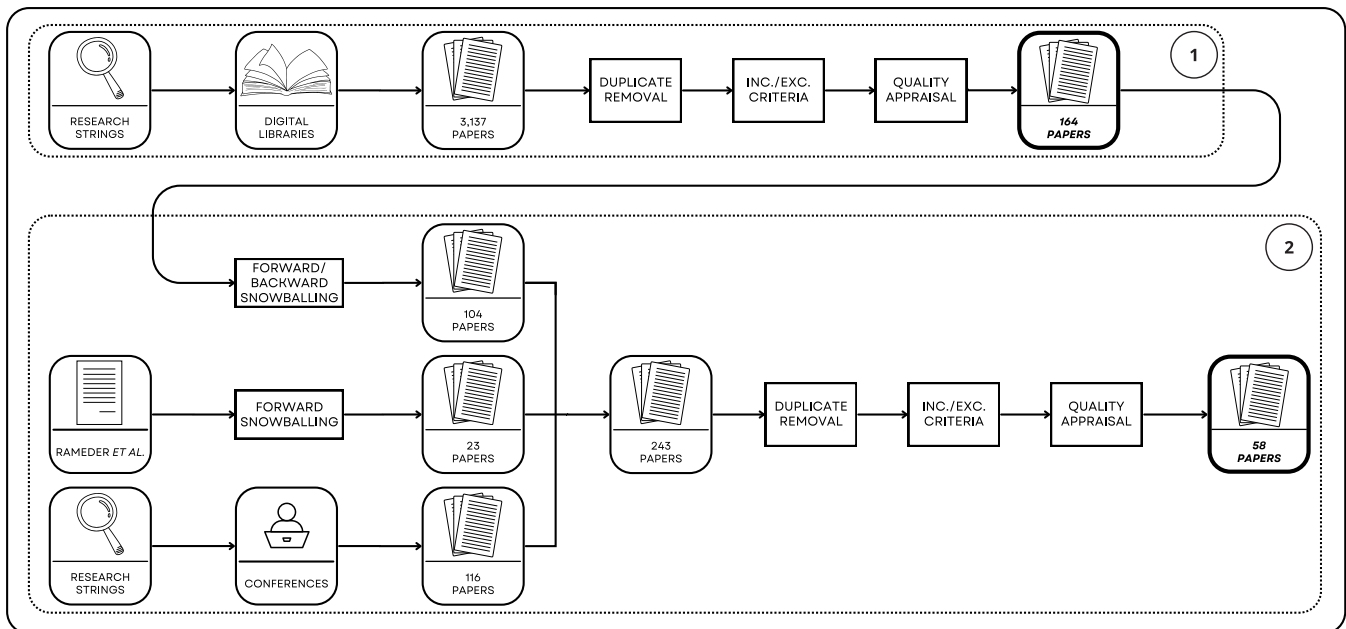


Fig. 1. Research method.

(e.g., with an *onlyOwner* modifier), an attacker could permanently destroy the contract, disrupting its functionality, and potentially stealing funds. To prevent attacks, *selfdestruct* should only be callable by trusted accounts, typically using access control mechanisms like *Ownable*.

The potential of smart contracts to enable dApps is vast, although realizing this potential requires prioritizing security considerations. While blockchain's public nature enhances transparency, it also exposes smart contract codes to potential attackers, who can exploit weaknesses upon scrutiny. Mitigating smart contract vulnerabilities requires rigorous code audit and testing, adherence to best practices, and the implementation of upgradeable mechanisms. By prioritizing security measures, we can harness the full potential of smart contracts while minimizing associated risks.

3. Research method

Nepomuceno and Soares (Nepomuceno and Soares, 2019) highlighted the desire among researchers to keep SLRs current, even if others carry out the update process. However, Mendes et al. (2020) pointed out the importance of employing decision support mechanisms to assist the software engineering community in determining the need for SLR updates. In fact, to decide whether the literature by Rameder et al. (2022) needed to be updated, we opted to apply the 3PDF framework introduced by Garner et al. (2016) and approved by Mendes et al. (2020).

The decision framework includes three steps to be applied sequentially. The first step consists of answering three questions, and if the answer to at least one of Step1's questions is 'NO,' there is no arguable need to update an SLR. Of course, the SLR addresses current problems related to Ethereum SCs, detection tools, and benchmarks; it is accessible through the open-access publication on Frontiers in Blockchain and uses a rigorous methodology, ensuring valid and well-conducted results. The second step invites one to check whether more recent research offers significant contributions that justify updating or supplementing the systematic review. It requires at least one positive answer. In recent years, important events related to Blockchain technology have occurred. In 2021, there was the NFT boom; in 2022, the transition of Ethereum to PoS was finalized; and in 2023, the increase in blockchain-based projects was 19% compared to the previous year, and several companies around

the world are still quietly investing in blockchain.¹ These changes have triggered strong interest in the research world, which has contributed to numerous studies. The third step requires an assessment of the impact of any research updates on the SLR conclusions. In addition, the final step requires at least one positive answer. The addition of recent studies would bring new perspectives, increase the comprehensiveness of the review, improve credibility, and update the conclusions based on the latest available data.

Finally, the decision framework revealed that the SLR is suitable for upgrading, as outlined in Table 1. We deliberately chose not to modify the search strings and data extraction form of the original study, given the findings of Nepomuceno and Soares (Nepomuceno and Soares, 2019), who observed that such alterations typically result in the creation of a new SLR rather than an update. By incorporating insights from previous research and using the 3PDF framework, we aimed to ensure a systematic and informed approach to updating the SLR on automated vulnerability analysis of Ethereum smart contracts. Our methodology underscores the importance of maintaining up-to-date literature while respecting established frameworks and guidelines.

3.1. Goal and research questions

The goal of this work is to investigate current research studies and the most significant contributions in the field of smart contract vulnerabilities. Concurrently, offering a comprehensive and systematic perspective can contribute to the research community by facilitating assessments and discussions on future directions related to these issues. We also aimed to comprehend how the research area has evolved. Consequently, as mentioned in the previous section, we examined the same research questions (RQs) posed by Rameder et al. (2022) except for one. In fact, we decided to exclude RQ₅ (i.e., "What are the open challenges of automated detection?") because its systematic and rigorous replication was challenging due to the lack of a data extraction form or supplementary material that clarified how information from the primary studies was collected (e.g., thematic analysis or content analysis). Consequently, we

¹ <https://www.forbes.com/sites/ninabambysheva/2023/02/07/forbes-blockchain-50-2023/>

Table 1
Application of the 3PDF framework to [Rameder et al. \(2022\)](#).

SLR	1.a	1.b	1.c	2.a	2.b	3.a	3.b	Result
Rameder et al. (2022)	YES	YES	YES	–	YES	–	YES	Upgradeable
1.a Does the published SLR still address a current question? 1.b Has the SLR had good access or use? 1.c Has the SLR used valid methods and was well-conducted? 2.a Are there any new relevant methods? 2.b Are there any new studies or new information? 3.a Will adopting new methods change the findings, conclusions, or credibility? 3.b Will the inclusion of new studies/information/data change findings, conclusions, or credibility?								

could not determine whether Rameder et al. used a systematic process to address that research question. Therefore, to avoid overclaiming systematicity, we analyzed the open challenges related to automated detection of software vulnerabilities, and discussed them in [Section 5](#). In detail, we posed the following RQs.

Q RQ₁ Which vulnerabilities are mentioned? How are vulnerabilities classified?

RQ₁ supports the understanding of smart contract vulnerabilities and their classification, which is essential for effective security management. A comprehensive taxonomy serves as a point of aggregation, helping researchers, practitioners, and auditors systematically consult, compare, and build on existing results. A taxonomy can improve risk assessment and mitigation, prioritizing efforts, and allocating resources more effectively and efficiently.

Q RQ₂ Which methods do automated tools use to detect vulnerabilities?

RQ₂ analyzes automated tools and solutions proposed to enhance, support, and facilitate the work of practitioners, researchers, and security auditors. Understanding which methods were used and how they were applied can reveal the capabilities and limitations of existing solutions, thus guiding the development of new techniques and informing the selection of tools.

Q RQ₃ Which vulnerabilities are addressed by the tools?

RQ₃ helps assess coverage and gaps in automated vulnerability detection. It enables the selection of complementary tools for comprehensive protection, highlighting areas that require manual review or additional measures.

Q RQ₄ How are the tools evaluated?

RQ₄ examines how the performance of vulnerability detection tools is evaluated. Proper evaluation is crucial to ensure that tools perform as expected and to allow effective comparisons between different solutions. A deeper understanding of the benchmarks and evaluation practices used in the literature could also provide insight into current trends and future directions in the field.

3.2. Identification of relevant literature

[Fig. 1](#) depicts the process we followed to update the systematic literature review proposed by [Rameder et al. \(2022\)](#). [Brereton et al. \(2007\)](#) emphasize the importance of searching multiple databases because no single repository contains all pertinent papers. We conducted our search in five digital libraries in Step 1, with the aim of identifying relevant studies in a comprehensive way. We used the same search string (see [Table 2](#)) as the original SLR; consequently, the choices and motivations behind each search string are detailed in the original SLR ([Rameder et al., 2022](#)), which also highlighted the need for a structured snowballing process to avoid overlooking essential studies. Therefore, in addition to analyzing the libraries, we conducted a second search in Step 2. We applied backward and forward snowballing to the papers extracted from Google Scholar that passed the quality assessment stage to ensure a thorough exploration of the literature and capture relevant studies that may have been missed in the initial search. Furthermore, we ap-

Table 2
Query strings on different search engines.

Database	Query
ACM Digital Library	Title: (“smart contract” OR “smart contracts”) AND AllField: (vulnerability OR vulnerabilities OR bug OR bugs OR tool OR security OR analysis OR detection OR verification)
Google Scholar	allintitle: (“smart contract” OR “smart contracts” OR Ethereum) AND (vulnerability OR vulnerabilities OR bug OR bugs OR tool OR security OR analysis OR detection OR tool OR verification)
IEEE Xplore	“All Metadata”: (“smart contract” OR “smart contracts”) AND (vulnerability OR vulnerabilities OR bug OR bugs OR tool) AND (security OR analysis OR detection OR verification)
Science Direct	Title, abstract, keywords: “smart contract” AND (vulnerability OR vulnerabilities OR bug OR tool OR security OR analysis OR detection OR verification)
TU Wien CatalogPlus	title contains (“smart contract” OR “smart contracts”) AND Subject contains (security OR vulnerability OR vulnerabilities OR bug OR bugs OR analysis OR detection OR tool OR verification)

plied forward snowballing on [Rameder et al. \(2022\)](#) to identify additional relevant studies through references to the initial paper. Finally, we analyzed publications from major conferences such as ICSE, FSE, ASE, ISSTA, and ICST. These highly relevant conferences often feature high-quality research on the analyzed topics; therefore, this further step provides valuable information on current trends and developments in smart contract vulnerabilities.

3.3. Inclusion and exclusion criteria

We adopted the inclusion and exclusion criteria of the original SLR with two modifications. The original SLR, conducted at the end of January 2021, included publications from 2014. Given the nature of our study, we modified this exclusion criterion from “published before 2014” to “published before 2021”, in accordance with the timeframe of our study. The second change is about the fourth exclusion criteria; it passed from “Not accessible online via the library network of TU Wien” to “Not accessible online via the library network of the University of Salerno”. All other criteria remained unchanged and are shown in [Table 3](#).

Exclusion Criterion 1 specifies the removal of papers unrelated to Ethereum. While this criterion excludes studies focusing solely on other blockchains, it does not exclude papers that, although based on other blockchains, relate their findings to Ethereum. This approach enabled us to identify cross-chain vulnerabilities, as cross-system architectures often integrate Ethereum with other blockchain platforms. Notably, [Buterin \(2016\)](#) provides a comprehensive analysis of the interoperability problem in blockchain ecosystems, highlighting the relevance of cross-chain considerations.

Table 3
Inclusion and exclusion criteria.

Inclusion Criteria
1 - SC security bugs or vulnerabilities
2 - SC vulnerability detection, analysis, or security verification tools
3 - Automated detection or verification methods
Exclusion Criteria
1 - Unrelated to Ethereum
2 - Not written in English
3 - A patent, blog entry, or other grey literature
4 - Not accessible online via the library network of the University of Salerno
5 - Published before 2021

3.4. Selection and classification

We carefully reviewed and assigned each study to one of these three distinct groups.

Systematic Literature Reviews (SLRs) include comprehensive reviews that systematically collect, critically analyze, and synthesize existing research on smart contract security.

Surveys and review studies on smart contract security analysis tools, methods, and vulnerabilities provide overviews and comparative analyses of the state of the art.

Primary Studies (PSs) consist of original research on developing and evaluating smart contract security analysis and vulnerability detection tools and methods. Primary Studies provide novel methodologies and empirical findings.

3.5. Quality appraisal

The quality of the papers was evaluated in detail by applying the metrics for intrinsic and contextual data quality described by [Strong et al. \(2002\)](#).

Intrinsic Data Quality (IDQ) assesses the accuracy, objectivity, credibility, and reputation of the publication venues. We converted other metrics, including the SCImago Journal Ranking, the CORE Journal or Conference Ranking, and the Scopus CiteScore percentile ranking, into scores ranging from 0 to 1. [Table 4](#) shows the IDQ score for the ranking of the venues. In cases where a journal or conference has yet to be ranked for the current year, we consider the most recent ranking available. We select the highest score as the IDQ (Indicator of Journal/Conference Quality) if multiple rankings are available.

Contextual Data Quality (CDQ) assesses the relevance, added value, timeliness, completeness, and amount of data, and thus depends on the context of the evaluation, e.g., the purpose of the systematic review, the research questions, and the data to extract and analyze. The CDQ score is the arithmetic mean of the answers to the questions, each a number between 0 and 1. The questionnaire is based on the DARE criteria suggested by Kitchenham and Charters ([Kitchenham and Charters, 2007](#)) and consists of seven questions for SLRs and surveys (see [Table 5](#)) and two questions for Primary Studies (see [Table 6](#)).

Final Data Quality (FDQ) combines IDQ and FDQ. We compute the FDQ score as the arithmetic mean of the other two scores; thus, it is a number between 0 and 1. We map CDQ and FDQ to a three-point Likert scale for readability and ease of classification. We consider a score low if it is below 0.5, high if it is at least 0.8, and medium otherwise. Studies with a CDQ or FDQ score below 0.5 are excluded.

3.6. Data extraction

Once we identified the final set of sources, we extracted information relevant to our research questions, following the Nepomuceno and Soares guidelines ([Nepomuceno and Soares, 2019](#)). We used the same data extraction forms as [Rameder et al. \(2022\)](#), ensuring consistency in collecting and organizing data on vulnerabilities, analysis

Table 4
IDQ assessment by venue ranking.

SCImago	CORE	Scopus	IDQ
Q1	A*, A	$\geq 75^{\text{th}}$ percentile	1.0
Q2	B	$\geq 50^{\text{th}} \wedge < 75^{\text{th}}$ percentile	0.8
Q3, Q4	C	$\geq 25^{\text{th}} \wedge < 50^{\text{th}}$ percentile	0.5
not indexed, but pub. by IEEE, ACM, Springer			0.4
		$< 25^{\text{th}}$ percentile	0.2
other sources, e.g., arXiv, preprints, theses, reports			0.2

tools, datasets, and test sets. We collected detailed information about vulnerabilities, including their identifier or code, name and alternate names, description, category, and reference paper. With respect to tools and frameworks, we collected essential information such as their name, description, and the article that introduced them. We also reported whether the tool is open source, provided links to the tool's page or GitHub repository, and provided the publication date and the last update date. Additionally, we documented the source language. Technical specifications were also recorded, including the input needed by the tools (i.e., EVM bytecode or source code), its purpose, the type of analysis conducted, and any code transformation techniques and analysis methods employed. This comprehensive collection allowed us to assess the tools thoroughly. For datasets and test sets used to compute tool benchmarks, we noted the reference article introducing the dataset, the tools or projects using it, and the repository link. We also recorded the number of contracts within the dataset and the form in which the contracts are represented (i.e., Solidity source code, bytecode, address). Furthermore, we checked the analysis results and any identified exploits. This detailed approach ensured that we clearly understood the roles of the datasets and test sets in tool evaluation. By meticulously collecting and organizing this information, we ensured an accurate and systematic approach to understanding vulnerabilities, analysis tools, and performance benchmarks. This methodical process enhanced the reliability and comprehensiveness of our study.

3.7. Search process execution

The search was carried out between February and March 2025. We started the research with Step 1, which yielded 3137 results from the five digital libraries. After removing 392 duplicates, we retained 2745 unique papers. Then, we applied inclusion and exclusion criteria. [Table 7](#) shows the occurrences of each criterion. An article was included if it satisfied at least one inclusion criterion and no exclusion criteria. After applying the inclusion and exclusion criteria, 260 papers were retained for the quality appraisal. In total, quality appraisal in Step 1 produced 164 papers, of which 12 were SLRs ([Abubakar et al., 2024; Faruk et al., 2024; Hejazi and Lashkari, 2025; Yulianto et al., 2023; Kushwaha et al., 2022b,a; Piantadosi et al., 2022; Rameder, 2021; Zaazaa and El Bakkali, 2023; Rosaire and Jules, 2022; Vacca et al., 2021; Kiani and Sheng, 2024](#)), 22 were surveys ([Wu et al., 2024b; Li et al., 2024c; Vidal et al., 2024a; Zheng et al., 2023; Huang et al., 2024a; HajiHosseiniKhani et al., 2024a; Mishra et al., 2024; Huang et al., 2024b; Obradović and Vuletić, 2024; Wu et al., 2024a; Zhu et al., 2024; Chu et al., 2023; Chen et al., 2022b; Zhang et al., 2023d; He et al., 2023b; Qian et al., 2023a; Jiang et al., 2023; Chaliasos et al., 2023; Zhou et al., 2022; Staderini, 2022; Zaazaa and Bakkali, 2023; Agarwal et al., 2021](#)), and 130 were primary studies ([Iuliano et al., 2025a; Chen et al., 2024d; Zhang, 2024; Sun et al., 2024; Luo et al., 2024a; Deng et al., 2024; Chen et al., 2024a; Wang et al., 2024c; Chen et al., 2024f; Li et al., 2024d; Wu et al., 2024c; Wang et al., 2024a; Eshghie et al., 2024; Eshghie and Artho, 2024; Chen et al., 2024e; Liu and Song, 2024a,b; Chen et al., 2024c; Ye et al., 2024a; Chen et al., 2023a; Guo et al., 2024a; Ye et al., 2024b; Song et al., 2022; Liu et al., 2022; Li, 2021; Zheng et al., 2024a; Zhang et al., 2024c; Mothukuri et al., 2024; Özdemir Sönmez and Knotenbelt, 2024; Jain and Tripathi, 2024; Chen et al., 2024b; He et al.,](#)

Table 5

CDQ assessment criteria of SLRs and surveys.

Q1: Is the methodology and literature search of the review systematic and documented? Are all relevant studies at the time likely to be covered?	
1.0	Yes, in comprehensive detail, systematic and good quality. To be (re)classified as SLR.
Not Graded	No, to be (re)classified as survey.
Q2: Does the study focus on work about smart contract vulnerabilities, detection tools, or approaches to automated detection or verification?	
1.0	Yes, the research topic closely related and covers major parts of this work.
0.6	It covers related topics, but important areas differ or are missing.
0.3	Some similar areas are covered.
0.0	The review focuses on different research topics, only a very small part is related.
Q3: Does the study cover the evaluation of smart contract vulnerabilities?	
1.0	Yes, the work includes a survey, evaluation and detailed description of vulnerabilities including classifications.
0.6	Descriptions or classifications of certain vulnerabilities are given, but it is not a major focus of the work.
0.3	Some vulnerabilities are superficially listed or mentioned.
0.0	No, the focus is not on the survey of vulnerabilities.
Q4: How many tools or analysis/verification methods does the review identify, classify, compare or evaluate?	
1.0	more than 12.
0.6	8 to 12.
0.3	4 to 7.
0.0	less than 4.
Q5: Did the author(s) assess and compare the quality/validity of tools or automated detection/verification approaches in detail?	
1.0	Yes, in detail and high quality, including practical experiments or comprehensive/in-depth coverage and classification.
0.6	Yes, compared/evaluated in some detail, but only theoretically.
0.3	Only superficial description.
0.0	No.
Q6: Is a set of smart contract benchmarks provided?	
1.0	Yes.
Not Graded	No.
Q7: How current is the review?	
1.0	2023/2024.
0.5	2022.
0.0	2021.

Table 6

CDQ assessment criteria of primary studies (PSs).

Q1: Is the PS referenced in a selected SLR or survey? How often is it cited according to Google Scholar?	
1.0	referenced in selected SLR or Survey, or cited more than 12 times according to Google Scholar.
0.6	8 to 12 citations.
0.3	4 to 7 citations.
0.0	less than 4 citations.
Q2: Is the PS related to an executable tool or to a framework for verifying security properties or identifying vulnerabilities of Ethereum smart contracts?	
1.0	Yes.
0.0	No.

2024b; Liu and Jiao, 2024; Qian et al., 2024; Ibba et al., 2024; Ding et al., 2024; Wang et al., 2024d; Khan and Namin, 2024; Wang et al., 2024b; Guo et al., 2024c; Li et al., 2024b; Dai et al., 2024; Luo et al., 2024b; Li et al., 2024a; Wu et al., 2023; Khanzadeh and Alalfi, 2024; Zeng et al., 2023; He et al., 2023a; Chu et al., 2024; He et al., 2024c; Wang et al., 2024f; Cao et al., 2023a; Dong et al., 2024; Fang et al., 2023; Duan et al., 2023b; Jie et al., 2023; Zhang et al., 2022e; Vivar et al., 2021; Otoni et al., 2023; Ghaleb et al., 2023; Jin et al., 2022; Sun et al., 2023; Sujeetha and Akila, 2023; Narayana and Sathiyamurthy, 2023; Soud et al., 2023; Xie et al., 2023; Zhang et al., 2022b; Hwang et al., 2022; Cai et al., 2023; Liu et al., 2023a; Yang et al., 2024a; Qian et al., 2023c; Cao et al., 2023c; Yu et al., 2021b; Chen et al., 2022a; Qian et al., 2023b; Jeon et al., 2023; Hu et al., 2024; Eshghie et al., 2021; De Salve et al., 2024; Su et al., 2023; Ji et al., 2023; Ferreira Torres et al., 2022; Pasqua et al., 2023; Lutz et al., 2021; Ghaleb et al., 2022; Ashizawa et al., 2021; Wang et al., 2022; Linoy et al., 2021; Mazurek, 2021; Zhu et al., 2023; He et al., 2023c; Peng et al., 2023; Ma et al., 2023; Huang et al., 2021a; Yang et al., 2023; Nguyen et al., 2023; Wang et al., 2021d; Fei et al., 2023; Sun et al., 2022b; Zheng et al., 2022; Yu

et al., 2023a; Barboni et al., 2023; Liu et al., 2023b; Bose et al., 2022; Cao et al., 2023b; Liang and Zhai, 2023; Ali et al., 2021; Wang et al., 2023; Nguyen et al., 2021; Zhang et al., 2023b; Guo et al., 2024b; Zhang et al., 2022d; Di Angelo et al., 2023; Zhaoxuan et al., 2022; Liao et al., 2023; Praitheeshan et al., 2021; Hu et al., 2024; Zhang et al., 2022c; Barboni et al., 2022; Dai et al., 2022; Zhang et al., 2023a; Olsthoorn et al., 2022; Chen et al., 2023b; Zhang and Liu, 2022; Yu et al., 2023b; He et al., 2024a; HajiHosseiniKhani et al., 2024b; Li et al., 2023; Ye et al., 2022).

Afterward, we started Step 2, conducting forward and backward snowballing on the 164 papers obtained in Step 1. During snowballing, pre-2021 papers were not considered due to one of the exclusion criteria. Snowballing contributed 104 papers. We also conducted forward snowballing on Rameder et al. (2022), which helped to find another 23 papers. Finally, we analyzed the publications at major conferences. We manually applied a research string to papers to collect only relevant papers. Conferences contributed another 116 papers. In total, in Step 2, we analyzed 243 papers, 133 of which were removed because they were duplicates. We applied the inclusion and exclusion criteria to the

Table 7
Statistics about the search process execution.

	Init Set	Dupl	Unique	Inc 1	Inc 2	Inc 3	Exc 1	Exc 2	Exc 3	Exc 4	Exc 5	Post Crit	QA
Step 1	3137	392	2745	92	158	72	385	25	35	29	4	260	164
Step 2	243	133	110	11	44	25	5	2	0	1	1	72	58
Total	3380	525	2855	103	202	97	390	27	35	30	5	332	222

Table 8
Quality assessment results for evaluated papers.

Paper set	IDQ average	1st QR	2nd QR (Median)	3rd QR
332 papers (reached QA step)	0.74	0.50	1.00	1.00
222 papers (passed QA step)	0.86	0.80	1.00	1.00

remaining 110 papers and obtained 72 papers. Quality appraisal introduced 58 papers, of which seven were surveys (Ren et al., 2021b; Zheng et al., 2024b; Dia et al., 2021; Di Angelo and Salzer, 2024; Di Angelo et al., 2023; Munir and Taha, 2023; Wei et al., 2023) and 51 were primary studies (Yang et al., 2024b; Godbole and Krishna, 2024; Liang et al., 2024; Wang and Zhao, 2025; JJ and Singh, 2024; Rodler et al., 2023; Liu et al., 2024b; Liu and Li, 2022; Zhong et al., 2024; Liao et al., 2024; Wang et al., 2024g; Groce and Grieco, 2021; Ji et al., 2021a; Chen et al., 2021; Gao et al., 2020; Wang et al., 2024e; Zhang et al., 2024; Yang et al., 2025; Gao et al., 2024; Wang et al., 2024i; Sun et al., 2022a; Xu et al., 2021; Torres et al., 2021; Xu et al., 2022; Lakadawala et al., 2024; Liu et al., 2024a; Han et al., 2024; Zeng et al., 2022; Contro et al., 2021; Nassirzadeh et al., 2023; Shou et al., 2023; Feng et al., 2023; Nguyen et al., 2022; Song et al., 2024; Wu et al., 2021; Pan et al., 2021; Yu et al., 2021a; Zhang et al., 2023c; Ren et al., 2021a; Huang et al., 2022; Liu et al., 2021b; Liao et al., 2022; So et al., 2021; Zhou et al., 2021b; Zhukov and Korkhov, 2023; Choi et al., 2021; Samreen and Alalfi, 2021; Ferreira Torres et al., 2021; Mi et al., 2021; Jiang et al., 2021; Xue et al., 2024).

We analyzed 3380 publications, 3137 in step one and 243 in step two. The first step contributed 164 papers, and the second contributed 58 papers, for a total of 222 papers. As shown in Table 7, the most frequent inclusion criterion was the second (i.e., SC vulnerability detection, analysis, or security verification tools), while the most frequent exclusion criterion was the first (i.e., Unrelated to Ethereum).

Table 8 summarizes the quality assessment scores of the papers analyzed in the review. The average IDQ score increased from 0.74 to 0.86 among the papers that passed the QA step, indicating that higher-quality studies were retained. Moreover, the quartile values show that the majority of the included papers exhibit high and consistent quality, with 75% of the studies scoring above 0.8, confirming the overall robustness and homogeneity in terms of quality of the selected literature.

3.7.1. Search-string sensitivity to evolving terminology

The SLR we are updating was initially conducted in 2021. In recent years, new terminology has emerged, partly due to updates to the Ethereum Virtual Machine (EVM). Our study adopts the same search strings used in 2021. To ensure these strings remain sufficiently comprehensive, we conducted a sensitivity analysis to assess how the inclusion of newly introduced terms in the search string would affect results. We conducted a sensitivity analysis on Step 2 of our research method. We included 29 new emerging keywords in the research string used to explore the main conferences. Newer terms led to the discovery of two new papers not covered by the original research string. The two additional papers suggest that the difference is marginal ($\approx 5\%$), which indicates that our original search query was already reasonably comprehensive. After all, terms like *vulnerability*, *detection*, *security*, *bugs*, are well-established and widely used in the literature, effectively capturing the vast majority of relevant studies despite the emergence of newer terminology. We included the sensitive analysis in the replication package.

3.8. Data synthesis and analysis

We analyzed the studies selected above to answer our research questions. To answer RQ_1 , we focused on the studies that mention vulnerabilities. In particular, to classify vulnerabilities and organize them hierarchically, we followed a series of steps:

1. **Pilot Study.** We randomly selected and analyzed 20 sources to establish an initial taxonomy draft, as suggested by Ralph (Ralph, 2019).
2. **Pilot Study: Inter-rater Assessment.** We performed an inter-rater assessment of the vulnerabilities to reach unanimity on their classification, clustering, and hierarchical position, leading to several changes.
3. **Full Study.** The rest of the collected papers were analyzed following the consensus reached in the previous step. The sources were split into two halves, each independently analyzed by one researcher.
4. **Full Study: Inter-rater Assessment.** After the independent extraction of vulnerabilities, the researchers looked at each other's taxonomy to inter-rater assess them. All discrepancies were solved through discussions and reasoning.

Each researcher followed the following steps:

1. **Vulnerability Collection.** We extracted all information related to vulnerabilities, e.g., their name, ID, description, cause, classification, and eventually, the alternative names used to refer to them.
2. **Duplicate Removal.** We focused exclusively on vulnerabilities with the same name or similar description to reduce the number of vulnerabilities but also to ensure that in the literature, two vulnerabilities with the same name describe the same issue.
3. **Vulnerability Clustering.** We noted numerous vulnerabilities with similar names, which allowed us to divide them into unique clusters and analyze them deeply. Clusters of synonyms were created by joining terms that describe the same scenario. We primarily focused on the vulnerability description to understand the type of problem it represents. Then we added the vulnerability name to the cluster of names representing the same scenario. For each cluster, we documented the decision rule used to select a representative name for all its synonyms. Since our work extends and updates a previous taxonomy (Rameder et al., 2022), we first prioritized names already used in that taxonomy. For newly introduced vulnerabilities, we prioritized names already used in other taxonomies.
4. **Vulnerability Classification.** To categorize vulnerabilities, we focused on their root causes rather than their effects or attack vectors. Root causes are less ambiguous and help minimize classification ambiguities, providing a more precise, more reproducible taxonomy. We used categories proposed by Rameder et al. (2022), as starting set. When the categories were insufficient to cover a particular vulnerability, we introduced a new category.
5. **Hierarchical Organization.** The previous cluster analysis revealed that vulnerabilities are present in the literature with numerous alternative names and are described in various levels of detail. Therefore, we organized the vulnerabilities hierarchically, placing them on different levels: the first level, which is more general and representative, and the subsequent levels, which are more detailed.
6. **Existing Terminology Usage.** Due to the vast terminology used in the literature to refer to vulnerabilities, our work of classifying and organizing them was based exclusively on existing terminology without introducing new identifiers and names or adapting exist-

ing nomenclatures. Since our work extends and updates a previous taxonomy (Rameder et al., 2022), we prioritized the names already employed. We selected the most frequent names in the state of the art for the new vulnerabilities. When the number of occurrences in the literature was equal, we selected the most explanatory name.

The approach we used, particularly Step 4, enables us to adapt the taxonomy to emerging vulnerabilities that were not previously considered in existing taxonomies and are not represented by any categories in the state-of-the-art taxonomies. The entire process makes the taxonomy easily upgradable by researchers in future works, as updates to the Ethereum Virtual Machine or Solidity may introduce new vulnerabilities. When new vulnerabilities are discovered, the proposed taxonomy should be consulted to determine whether an existing category is suitable for classification. If no existing category is sufficiently representative, the taxonomy should be extended to include a new representative category.

Semi-structured interviews for taxonomy validation. The proposed taxonomy extends the validated taxonomy of Rameder et al. (2022) by incorporating additional vulnerabilities from other validated taxonomies, such as OpenSCV (Vidal et al., 2024a). Vulnerabilities whose classifications remained unchanged from these already validated sources were excluded from our validation process. Accordingly, the validation focused on vulnerabilities originating from non-validated taxonomies, those from validated taxonomies whose classifications had been revised, and newly introduced vulnerabilities positioned in hierarchical relationships to existing ones. We excluded already-validated vulnerabilities from three taxonomies (Rameder et al., 2022; Vidal et al., 2024a; Li et al., 2024c). The remaining vulnerabilities were validated through semi-structured interviews with four security and Solidity experts. We divided the vulnerabilities into two clusters, each of which was validated by two experts.

The semi-structured interviews were conducted in two stages. In the first stage, we validated the vulnerability categories and their completeness. Given a vulnerability and the list of categories, the participant was asked to select the most appropriate category based on the vulnerability's root cause. If, according to the participant, no category adequately represented the vulnerability, we discussed it with them to reach a shared agreement. In the second stage, we validated the hierarchical structure. Given the vulnerability under validation and the list of vulnerabilities in the selected category from the first stage, the participant was asked to identify any hierarchical relationships with other vulnerabilities in the same category. All responses were accompanied by justifications to assess the consistency and validity of the agreements. The semi-structured interviews allowed us to combine systematic validation with exploratory discussion. This format enabled participants not only to provide their judgments on specific validation tasks but also to share broader reflections and observations related to the taxonomy. By offering the flexibility to explore topics beyond the predefined questions, the interviews allowed participants to highlight cross-cutting aspects, identify potential ambiguities, and propose improvements that might not have been expected by the interviewer but were nevertheless relevant to the nature of the validation process.

To answer RQ_2 , we applied Closed Coding (Saldaña, 2021), adopting the same categories used by Rameder et al. (2022) to collect all the information from the tool. The three main areas analyzed are functionality, code transformation technique, and methods, each comprising several categories:

Functionalities: Vulnerability Detection, Program Correctness, Gas or Resource Analysis, Bulk Analysis or Full Automation, Exploit Generation, Reactive Defenses, Manual Analysis or Developer Support.

Code Transformation Techniques: Control Flow Graph, Data Flow, Transaction Analysis, Traces, Abstract Syntax Tree, Decompila-

tion, Compiler, Intermediate Representation, Specification Language, Disassembly, Finite State Machine.

Methods: Symbolic Execution, Formal Verification, Constraint Solving, Model Checking, Abstract Interpretation, Fuzzing, Runtime Verification, Concolic Testing, Mutation Testing, Pattern Matching, Code Instrumentation, Machine Learning, Taint Analysis.

We also collected additional information, such as tool name, brief description, link to the repository, date of publication, last update, development languages, input type, and analysis type. The vulnerabilities identified in RQ_1 and the tools identified in RQ_2 provided the basis to answer RQ_3 . We identified the vulnerabilities detected by each tool and created a detailed mapping between them.

To ensure that the mapping accurately reflected tools targeting explicit vulnerabilities, we applied an inclusion-exclusion process based on the tool's objectives and scope. Each tool mentioned in the reviewed studies was analyzed according to its primary purpose, detection granularity, and methodological focus. Tools were included only if they were specifically designed to identify or analyze well-defined vulnerability types in smart contracts. Conversely, we excluded tools that serve general compilation or decompilation purposes, those that perform contract-level vulnerability prediction through machine learning without identifying specific weakness types, and mutation-based approaches that do not implement vulnerability-oriented operators. This filtering ensured that only tools with explicit vulnerability-detection capabilities were retained in the mapping.

Finally, to answer RQ_4 , we adopted the Closed Coding approach (Saldaña, 2021). We collected various benchmarks to evaluate tools or conduct other experiments. We gathered the same information extracted by Rameder et al., like the smart contract types (i.e., source code, bytecode, or addresses), their size in terms of the number of SCs, the associated links, if present, and the analysis results when available. Furthermore, we mapped the benchmarks to the tools to evaluate and compare their performance.

3.9. Replication package

To facilitate the validation and replication of our study, we have made the replication package online.² It contains the full list of studies that emerged from queries on digital libraries, the results of snowballing, and all conference papers. The inclusion/exclusion criteria and the results of the qualitative analysis are presented for each paper. Finally, a complete and comprehensive taxonomy, a full list of tools, a mapping between tools and vulnerabilities, and benchmarks used for tool evaluation or other experiments are included. We also provide a webpage³ and a GitHub repository⁴ for easy consultation of the results.

4. Analysis of the results

Before delving into the results to address the RQs, we provide some meta-information about the studies in our systematic literature review. Fig. 2 shows the number of papers published yearly. The plot shows a slight growth in the number of papers published recently. However, the graph represents only the papers that have passed the quality assessment. The trend may suggest that several other papers will be published soon, which motivates our work. Furthermore, 2025 was excluded from the plot because we analyzed only two months of data.

² <https://figshare.com/s/6e49677bc49ada0f3753>

³ <https://gerardoiluliano.github.io/slr.html>

⁴ <https://github.com/Gerardoiluliano/SLR-Ethereum>

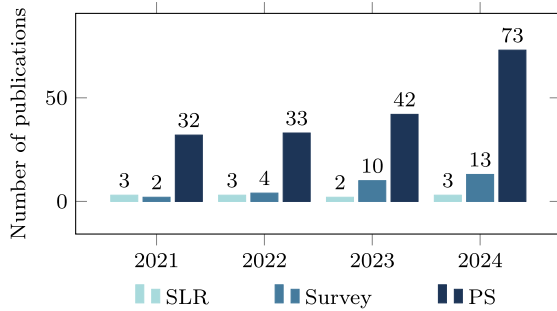


Fig. 2. Publication trend per year for SLRs, surveys, and primary studies.

4.1. RQ1. Which vulnerabilities are mentioned? How are vulnerabilities classified?

Following the steps described in Section 3.8, we answered to RQ₁ realizing a structured taxonomy. Initially, we identified 561 vulnerabilities. The first challenge was to remove all duplicates, which allowed us to reduce this number to 420. The second step was to analyze the remaining vulnerabilities to cluster them. We divided the 420 vulnerabilities into 192 unique clusters. For each cluster, a vulnerability was placed as a cluster representative. In contrast, the remaining 228 vulnerabilities were posed as alternative names. In other words, we have identified 192 vulnerability clusters with alternative names used in the literature to refer to the same phenomenon. For example, one cluster comprises *Block State Dependency*, *ChoiEtAl2021* and *Block Info Dependency* (Chen et al., 2022b), two vulnerabilities with similar names and expressing the same concept. Specifically, this vulnerability arises from smart contracts that rely on blockchain state attributes, which can be exploited due to the predictable nature of specific blockchain attributes. In the context of Ethereum, miners can manipulate these attributes of the block to delay submission by up to 900 ms (Zaazaa and El Bakkali, 2023). Another example is the cluster that includes *Timestamp dependency* (Chu et al., 2023; Zhou et al., 2022; Kushwaha et al., 2022b,a; Staderini, 2022; Rameder et al., 2022), *Timestamp restrictions* (Vivar et al., 2021), *Time Constraints* (Rameder et al., 2022), *Time manipulation* (Agarwal et al., 2021; Rameder et al., 2022), *Block values as a proxy for time* (Zaazaa and Bakkali, 2023; Zaazaa and El Bakkali, 2023), *Incorrect use of event blockchain variables for time* (Vidal et al., 2024a). These vulnerabilities express the same concept: the block's timestamp should not be used for precise calculations or critical time dependencies in the contract logic.

As previously explained, many vulnerabilities could overlap across multiple categories; therefore we focused on their root cause rather than their effect or attack vector. For instance, *Unchecked low-level call/send return value* (3A) refers to cases where a return value is not correctly handled or checked. While an unhandled exception may cause a Denial of Service, the vulnerability itself stems from the missing or unhandled check of the return value. Therefore, we placed 3A under *Exception & Error Handling Disorders*. Similarly, *Reentrancy* (1A1) is an input- or transaction-driven vulnerability. Once exploited, it can drain all funds (making the contract Greedy, 1J) or exhaust gas through recursive calls (leading to DoS due to block gas limit, 4I). Its effects can vary, but its primary cause is a crafted malicious transaction that triggers the vulnerable external call, along with a malicious contract prepared to exploit it. For these reasons, we selected the *Malicious Environment, Transactions, or Input* category. Finally, *Timestamp Dependency* (2A1) arises from dependency on a block property. If a timestamp is used to lock resource access, it may lead to an access control weakness; if it is used to generate randomness, it becomes a weak source of randomness. Again, the primary cause - block property dependency - guided our categorization of 2A1 as *Environment/Blockchain Dependency*.

The next step was to hierarchically organize the clusters based on the level of detail in their descriptions. As shown in Table 10, we organized

113 vulnerabilities at the first level, 73 at the second level, and six at the third level.

The hierarchical organization reveals that the well-known vulnerability, *1A1-Reentrancy*, is actually a specific instance of a broader vulnerability, *1A-Call to the unknown*, Rameder2022. In addition, the *Reentrancy* vulnerability exhibits four distinct behaviors (1A1.1, 1A1.2, 1A1.3, and 1A1.4). Similarly, the *3B1-Mishandled out-of-gas exception* is a subcategory of the broader *3B-Unexpected throw or revert* vulnerability, as also exposed in previous research by Rameder et al. (2022). The sheer variety of names that refer to a single vulnerability category is remarkable. The literature references vulnerabilities such as 1B, 2D, 3C, 5B1, 5B5, 7G, and 7I6 using at least seven names. Rameder's taxonomy (Rameder et al., 2022) addresses this issue by grouping several vulnerabilities under alternative names or synonyms for a single vulnerability. For instance, *5C1-Missing protection against signature replay attacks*, *5C2-Lack of proper signature verification*, *5C3-Hash collisions with multiple variable-length arguments*, *5C4-Function clashing*, and *5C5-Signature malleability* are all alternative names for *5C-Signature-based vulnerabilities*. Conversely, the taxonomy proposed by Zaazaa and Bakkali (2023) classifies each vulnerability independently and further delineates their differences. This divergence in classification was resolved by organizing vulnerabilities into hierarchical levels: *5C-Signature-based vulnerabilities* occupy the first level due to its general nature, whereas the specific vulnerabilities are placed at the second level to distinguish their unique characteristics. Similar hierarchical structuring has been applied to other vulnerabilities, such as *1A1:1A1.1-1A1.2-1A1.3-1A1.4*, *1B:1B1-1B2*, *2A:2A1-2A2-2A3*, *2B:2B1-2B2-2B3-2B4*, *2E:2E1-2E2-2E3*, *2F:2F1-2F2-2F3*, *3A:3A1-3A2-3A3*, *4C:4C1-4C2*, *5B:5B1-5B2-5B3-5B4-5B5-5B5.1-5B5.2-5B6*, and *6A:6A1-6A2*.

4.1.1. Taxonomy validation results

To validate the proposed taxonomy, we analyzed all new vulnerabilities added to the baseline taxonomy (Rameder et al., 2022). Vulnerabilities derived from three already validated taxonomies (Rameder et al., 2022; Vidal et al., 2024a; Li et al., 2024c) were excluded from the validation process when their classifications remained unchanged. In total, 36 vulnerabilities required validation. We conducted semi-structured interviews with four experts in security and the Solidity language. Further information about the experts is reported in Table 9. The interviews were structured in two stages: the first aimed to validate the vulnerability categories, and the second to validate the hierarchical relations among vulnerabilities. The 36 vulnerabilities were divided into two clusters of 18 vulnerabilities each, with two experts independently validating each cluster. To assess inter-rater agreement among experts, we calculated Cohen's Kappa coefficient for both validation stages and then averaged the results. For the first cluster, the experts achieved Cohen's Kappa values of 0.87 for category validation and 0.73 for hierarchy validation, yielding an average of 0.80. For the second cluster, Cohen's Kappa values were 0.77 for category validation and 1.00 for hierarchy validation, resulting in an average of 0.88.

All the experts identified an inconsistency in the taxonomy due to the vulnerabilities classified as Denial-of-Service (DoS). The taxonomy is well-structured and follows a classification strategy focused on the root causes of vulnerabilities. From their perspective, this approach is highly effective for auditors, as it enables them to identify the underlying cause of a vulnerability immediately and, consequently, to clearly determine where to act to remove or mitigate the issue. However, the DoS category does not align with this root-cause-oriented strategy, since a DoS represents the effect of a malicious operation rather than its cause. A vulnerability that causes a DoS typically originates from problems in other categories within the taxonomy. For example, a DoS can be caused by block gas limit exhaustion, unhandled exceptions, or dependency on an external resource that becomes unavailable. During the interviews, the experts identified a specific cause for each vulnerability whose effect was a DoS. Based on their experience and to maintain consistency throughout the taxonomy, they suggested removing the DoS category

Table 9
Experts experiences.

Id	Role	Experience	Affiliation
Expert 1	Team Leader and Developer	Five years of experience in Solidity	Company
Expert 2	Developer	Four years of experience in Solidity	Company
Expert 3	Developer	Two years and half of experience in Solidity	Company
Expert 4	Post Doc	Six years of experience in Security and two in Solidity	University of Luxembourg

and reallocating its vulnerabilities to the categories that best represent the root cause of Denial of Service. As a result of the interviews, we validated vulnerabilities and reclassified those initially categorized as DoS. Validation with experts enables us to assess the completeness and correctness of the categories, thereby creating a consistent taxonomy that organizes vulnerabilities by root cause. The interview results are available in the replication package.

RQ₁ Summary. We developed a structured taxonomy for Ethereum smart contract vulnerabilities by clustering 420 unique vulnerabilities into 192 groups. The groups are classified into 13 categories based on impact or effect. Each group consists of a representative vulnerability and alternative names, reflecting the various terminology used in the literature for similar issues. The structure organizes vulnerabilities into three levels based on their specificity. The first level contains 113, the second contains 73, while the third contains six. Taxonomy has been validated by four experts.

RQ₁ Key Point. Our taxonomy covers a wide range of vulnerabilities, unifying all categories identified across existing taxonomies, none of which individually provide complete coverage. The alternative names used in the literature are consistently assigned to a single vulnerability, ensuring clarity and avoiding ambiguity. Vulnerabilities are categorized based on their root causes.

4.1.2. How to consult the taxonomy

This guide is designed to help auditors navigate the taxonomy and accurately classify vulnerabilities encountered during smart contract auditing. The process begins with identifying the vulnerability of interest (e.g., Block values as a proxy for time). Next, the synonym clusters in the taxonomy should be checked to verify whether the chosen term appears there. If it does, the corresponding representative name should be used as the canonical reference (e.g., Timestamp Dependency). Once the representative name is identified, its position in the hierarchy can be explored to determine whether it represents a general vulnerability with specialized children or a specialization of a broader parent vulnerability. This hierarchical exploration helps clarify whether the vulnerability should be interpreted as a general pattern or a specific instance. In this example, Timestamp Dependency is a child of Block State Dependency, meaning it is a specific type of dependency related to the block's state. Furthermore, examining its position in the hierarchy may reveal additional specializations that differ from the aspect initially considered, providing a more comprehensive understanding of its possible variants (e.g., Block Number Dependency).

4.2. RQ2. Which methods do automated tools use to detect vulnerabilities?

To address the second research question, we extracted and analyzed the tools from the selected studies in detail. The criteria used to assess the contextual data quality of the primary studies proved to be highly effective. Through this analysis, we identified a total of 219 distinct tools listed in Table 12.

Input and analysis types. The input type most analyzed is the source code. Of 219 tools, 142 analyze only source code, 59 analyze only bytecode, and 12 analyze both types. Six tools (Ye et al., 2024a; Li, 2021; Wu et al., 2023; Chen et al., 2024f; Liu and Li, 2022; Zhang et al., 2022a) analyze addresses or transaction traces. It is important to note that all

tools are implemented and tested on Solidity code, and Vyper has never been considered.

Static analysis is the most common type of analysis, with 145 tools exclusively performing this type of analysis. In contrast, 39 tools perform only dynamic analysis, and 35 tools perform both types of analysis.

Tool's functionalities. Fig. 3 details the functionalities implemented by the tools. The most frequent functionality observed is Vulnerability Detection, accounting for 82.5% of cases. This high percentage is expected because many functionalities indirectly contribute to vulnerability detection. These include, for example, tools that analyze the use of gas and resources or offer developer support. An analysis of the functionalities revealed the need for fully automated tools, i.e., performing vulnerability detection, generating patches, and ultimately fixing the vulnerability by updating the contract using appropriate techniques or patterns. Another limitation lies in the tools used for exploit generation. In particular, the few tools that generate attack vectors or vulnerable transactions do so to prove the actual presence of the vulnerability they identify. An example is SMARTTEST (So et al., 2021) as discussed by Zhang et al. (2023d), which aims to detect bugs in smart contracts and generate vulnerable transaction sequences that automatically prove flaws. A relevant aspect that has emerged is the focus on resource and gas use. Minimizing and optimizing resource use leads to a reduction in fees and, thus, gas. All this can be translated into savings on the user's side. The 16% of the tools focus on this aspect, showing interest in optimizing resources and gas. Although most tools focus on vulnerability detection, only a few are integrated into development environments and some still require full implementation. Therefore, further studies are required to investigate their performance.

Code transformation techniques. As shown in Fig. 4, the considered tools use different techniques and structures to represent and analyze the code. The most used code structures leveraged by static analysis tools are Control Flow Graphs (CFG), Intermediate Representations, Specification Languages, and Abstract Syntax Trees (AST). Techniques such as Decompilation, Finite State Machines (FSM), and Disassembly remain underutilized. Dynamic analyzers mostly use Data Flow Graphs, Transaction Analysis, and Execution Traces. They use intermediate representations very few times, followed by CFGs and ASTs. When performing dynamic analysis, contract execution traces, transaction analysis, and data flows are crucial considerations. These techniques can capture the behavior of smart contracts when they are executed. Finally, the most used structures leveraged by tools performing both types of analysis are CFGs, with nine occurrences; Data Flow, Transaction Analysis, and Execution Traces, with seven occurrences; and ASTs, with five occurrences.

Tool's methods. As detailed in Fig. 5, the tools use several methods to analyze code, with some methods being more suitable than others. Static analysis tools mostly use machine learning. Numerous tools extract code metrics to feed ML models, as discussed by Zaazaa and El Bakkali (Zaazaa and El Bakkali, 2023), (Qian et al., 2023a), and (Zhang et al., 2023d). Peculiar (Wu et al., 2021) analyze Crucial Data Flow Graphs extracted from the source code. Wang et al. (2021a) use AST Fuse program Slicing (AFS) to fuse code characteristics and feed them to deep learning models. SC-VDM (Zhou et al., 2021a) transforms smart contract bytecode into grayscale matrix pictures and then uses Convolutional Neural Networks (CNNs) for vulnerability detection. Finally, Huang et al.

Table 10

A Comprehensive List of Vulnerabilities Hierarchically Organized.

Id	Name	Alternative Names
1 - Malicious Environment, Transactions, or Input		
1A	Call to the unknown Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Staderini (2022) , Kushwaha et al. (2022b)	Unexpected fallback function Rameder et al. (2022)
1A1	Reentrancy Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Choi et al. (2021) , Chu et al. (2023) , Zhou et al. (2022) , Kushwaha et al. (2022a) , Staderini (2022) , Chaliasos et al. (2023)	SWC-107, OWASP-SC05, DASP-1, Cross function race condition, Recursive call vulnerability, Race to empty Rameder et al. (2022)
1A1.1	Reentrancy with ETH transfer Li et al. (2024c)	Reentrancy-eth Agarwal et al. (2021) , Feist et al. (2019a) , Unsafe Credit Transfer Vidal et al. (2024a)
1A1.2	Reentrancy without ETH transfer Li et al. (2024c)	Reentrancy-no-eth Agarwal et al. (2021) , Feist et al. (2019a) , Unsafe System State Changes Vidal et al. (2024a)
1A1.3	Reentrancy with same effect Li et al. (2024c)	Reentrancy benign Agarwal et al. (2021) , Feist et al. (2019a)
1A1.4	Reentrancy with negative events Li et al. (2024c)	Reentrancy events Agarwal et al. (2021) , Feist et al. (2019a)
1B	Exact balance dependency Rameder et al. (2022)	SWC-132, Unexpected Ether, Manipulated balance, Unexpected Balance Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Unexpected Ether balance Munir and Taha (2023) , Unprotected Ether Balance Staderini (2022) , Balance equality Agarwal et al. (2021) , Strict Balance Equality Chen et al. (2022b) , Improper Locking Vidal et al. (2024a)
1B1	Forcing Ether to contracts Rameder et al. (2022)	Check on force to receive balance Dia et al. (2021) , Force Sending Ether with this.balance check in require() or assert() Li et al. (2024c)
1B2	Pre-send Ether Rameder et al. (2022)	
1C	Improper data validation Rameder et al. (2022)	Untrustworthy data feeds Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Kushwaha et al. (2022b)
1D	Vulnerable Delegatecall Rameder et al. (2022)	SWC-112, Delegatecall to Untrusted Callee Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Munir and Taha (2023) , Vidal et al. (2024a) , Delegate call to Insecure Contracts Zhou et al. (2022) , Unsafe delegatecall Wei et al. (2023) , Li et al. (2024c) , check on target address in delegatecall Dia et al. (2021) , Delegate call Agarwal et al. (2021) , Kushwaha et al. (2022b,a)
1E	External Contract Referencing Zhou et al. (2022)	Misuse of address parameters in the constructor Munir and Taha (2023) , Value assigned to contract address Dia et al. (2021)
1F	Sending Ether to arbitrary destinations Li et al. (2024c)	
1G	Unsafe send() in the require() condition Li et al. (2024c)	
1H	Send to zero address Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
1I	Insufficient gas griefing Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Munir and Taha (2023) , Wei et al. (2023) , Kushwaha et al. (2022b)	
	SWC-126, Incorrect Revert Implementation in a Loop Vidal et al. (2024a)	
1J	Greedy contract Rameder et al. (2022)	Locked Ether Rameder et al. (2022) , Freezing Ether Staderini (2022) , Zhou et al. (2022) , Choi et al. (2021) , Kushwaha et al. (2022a)
1J1	Frozen Ether Rameder et al. (2022)	
2 - Blockchain/Environment Dependency		
2A	Block State Dependency Chen et al. (2022b) , Choi et al. (2021)	Block Info Dependency Chen et al. (2022b)
2A1	Timestamp dependency Rameder et al. (2022) , Chu et al. (2023) , Zhou et al. (2022) , Kushwaha et al. (2022b,a) , Staderini (2022)	SWC-116, DASP-8, Time constraints, Time manipulation Rameder et al. (2022) , Time restriction Vivar et al. (2021) , Block values as a proxy for time Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Incorrect Use of Event Blockchain variables for Time Vidal et al. (2024a)
2A2	Block number dependency Rameder et al. (2022)	
2A3	Blockhash dependency Rameder et al. (2022)	Blockhash usage Staderini (2022)
2B	Transaction-ordering dependency Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Staderini (2022) , Agarwal et al. (2021) , Zhou et al. (2022) , Chu et al. (2023) , Kushwaha et al. (2022b,a)	SWC-114, Event-ordering bugs Rameder et al. (2022) , Incorrect Function Call Order Vidal et al. (2024a) , time-of-check vs time-of-use
2B1	Front-running attack Rameder et al. (2022)	DASP-7, Race condition Rameder et al. (2022)
2B2	Transfer Pre-Condition Dependent on Transaction Order Vidal et al. (2024a)	
2B3	Transfer Amount Depending on Transaction Order Vidal et al. (2024a)	
2B4	Transfer Recipient Depending on Transaction Order Vidal et al. (2024a)	
2C	Bad random number generation Rameder et al. (2022)	SWC-120, OWASP-SC09, DASP-6, Generating Randomness Rameder et al. (2022) , Zhou et al. (2022) , Staderini (2022) , Bad randomness Rameder et al. (2022) , Weak Sources of Randomness from Chain Attributes Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Random Number Vivar et al. (2021) , Weak PRNG Li et al. (2024c)
2C1	Randomness Using Block Hash Kushwaha et al. (2022b)	

Table 10
Continued.

2D	Leakage of confidential information Rameder et al. (2022)	SWC-136, Lack of transaction privacy Rameder et al. (2022) , Kushwaha et al. (2022b) , Private information leakage, Nothing is secret, Confidentiality failure, Keeping secrets, Data exposure Rameder et al. (2022) , On-chain oracle manipulation Chaliasos et al. (2023) , Secrecy failure Staderini (2022) , Exposed secret Munir and Taha (2023) , Unencrypted private data on-chain Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Secrets Vivar et al. (2021)
2E	Unpredictable state Rameder et al. (2022) , Vivar et al. (2021)	Unsafe External Library Call, External Library Calling Vidal et al. (2024a)
2E1	Dynamic Library Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
2E2	References to destroyed contracts Munir and Taha (2023)	
2E3	Use of Malicious Libraries Vidal et al. (2024a)	Using components with known vulnerabilities Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , External Library Calling, Malicious Libraries Vidal et al. (2024a)
2F	Unsafe Non-Blockchain External Call Vidal et al. (2024a)	
2F1	Unsafe External Web Service Call Vidal et al. (2024a)	Web Service Vidal et al. (2024a)
2F2	Unsafe External Command Execution Vidal et al. (2024a)	System Command Execution Vidal et al. (2024a)
2F3	Unsafe External File Access Vidal et al. (2024a)	External File Accessing Vidal et al. (2024a)
2G	Cross Channel Invocation Vidal et al. (2024a)	Cross Channel Chaincode Invocation Vidal et al. (2024a)
3 - Exception & Error Handling Disorders		
3A	Unchecked low-level call/send return values Rameder et al. (2022)	OWASP-SC06, DASP-4, Unhandled exception, Mishandled exceptions Rameder et al. (2022) , Chu et al. (2023) , Improper handling of exceptions Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Improper Check of Low-Level Call Return Value Vidal et al. (2024a) , Unchecked external calls Chen et al. (2022b) , Check on external call Munir and Taha (2023) , Improper Check of External Call Return Value, Improper Exception Handling of External Calls Vidal et al. (2024a)
3A1	Unchecked call Li et al. (2024c)	SWC-104, Unchecked call return values Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Agarwal et al. (2021) , Staderini (2022)
3A2	Unchecked send Li et al. (2024c)	Unchecked/unsafe send Rameder et al. (2022)
3A3	Unchecked transfer Li et al. (2024c) , Feist et al. (2019a)	Improper Check on Transfer Credit Vidal et al. (2024a)
3B	Unexpected throw or revert Rameder et al. (2022)	Improper Use of Exception Handling Functions Vidal et al. (2024a)
3B1	Mishandled out-of-gas exception Rameder et al. (2022)	Gasless send Rameder et al. (2022) , Staderini (2022) , Kushwaha et al. (2022b)
3C	Assert, require or revert violation Rameder et al. (2022)	SWC-123, SWC-110, Assert violation Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Assertion failure Choi et al. (2021) , Requirement violation Choi et al. (2021) , Wei et al. (2023) , Staderini (2022) , Requirement violation by the called smart contract Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Require on input variable Dia et al. (2021) , Revert require, Exception state Agarwal et al. (2021)
3C1	Wrong Selection of Guard Function Vidal et al. (2024a)	
3D	Send instead of transfer	Wrong use of transfer credit function Vidal et al. (2024a)
3E	DoS with failed call Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Munir and Taha (2023) , Kushwaha et al. (2022b)	SWC-113, Unexpected-throw-DoS, DoS with unexpected revert/throw Rameder et al. (2022) , Staderini (2022) , Dos by External contracts Rameder et al. (2022) , Staderini (2022)
3F	DoS by Exception inside loop Rameder et al. (2022)	DoS under external influence Rameder et al. (2022) , Chen et al. (2022b) , Wallet griefing causing out of gas exception Rameder et al. (2022) , Improper Exception Handling in a Loop Vidal et al. (2024a)
4 - Resource Consumption & Gas Related Issues		
4A	Gas costly loops Rameder et al. (2022)	Invariants in loops Rameder et al. (2022)
4B	Gas costly pattern Rameder et al. (2022)	Inefficient Operation Sequence Vidal et al. (2024a)
4C	High gas consumption of variable data type or declaration	
4C1	Byte Array Rameder et al. (2022)	Variable type Dia et al. (2021) , Bytes Variables Risk Li et al. (2024c) , Wrong Type in Variable Declaration Vidal et al. (2024a)
4C2	Invariant State variables not declared constant Rameder et al. (2022)	Declaration of invariant state variable Dia et al. (2021) , Missing Constant Modifier in Variable Declaration Vidal et al. (2024a)
4D	High gas consumption function type Rameder et al. (2022) , Chen et al. (2022b)	Function declaration public instead of external Rameder et al. (2022) , Vidal et al. (2024a)
4E	Under-priced opcodes Rameder et al. (2022)	
4F	Transfer of all the gas Rameder et al. (2022)	
4G	Unused statement Chen et al. (2022b)	
4H	Dangerous usage of msg.value inside a Loop	
4I	DoS with block gas limit reached Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Kushwaha et al. (2022b)	SWC-128, Check on gas Limit Dia et al. (2021) , Gas limit in loops, Extra Gas in loop Agarwal et al. (2021) , DoS with block gas limit Li et al. (2024c)
4I1	DoS Costly Pattern and Loops Staderini (2022)	Multiple calls in a single transaction Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Nested Call Chen et al. (2022b) , Multiple calls, Transfer in loop Agarwal et al. (2021)

Table 10
Continued.

5 - Authentication & Access Control Vulnerabilities		
5A	Prodigal contract	
5A1	Authorization via transaction origin Rameder et al. (2022)	SWC-115, Authentication through tx.origin Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Staderini (2022) , Agarwal et al. (2021) , Zhou et al. (2022) , Kushwaha et al. (2022b) , Transaction state dependency Munir and Taha (2023) , Luo et al. (2024a) , Transaction origin use Choi et al. (2021) , Dangerous Usage of tx.origin Li et al. (2024c) , Wrong Caller Identification Vidal et al. (2024a)
5B	Access Control management Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	OWASP-SC01, DASP-2, Improper access control Vidal et al. (2024a)
5B1	Unprotected self-destruction Rameder et al. (2022) , Chu et al. (2023) , Agarwal et al. (2021) , Kushwaha et al. (2022b,a) , Staderini (2022) , Zhou et al. (2022)	SWC-106, Unexpected kill, Destroyable contract Rameder et al. (2022) , Suicidal contract Choi et al. (2021) , Unsafe suicide Wei et al. (2023) , Check on suicide functionality Dia et al. (2021)
5B2	Unauthorized Ether withdrawal Rameder et al. (2022)	SWC-105, Unprotected Ether withdrawal Staderini (2022) , Wei et al. (2023) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Kushwaha et al. (2022b) Check on balance withdrawal Dia et al. (2021) , Ether leak Choi et al. (2021)
5B3	Initializing method without permission check Li et al. (2024c)	
5B4	Missing msg.sender check for transferFrom() Li et al. (2024c)	Arbitrary from in transferFrom() without msg.sender check Li et al. (2024c)
5B5	Unauthorized accessibility due to wrong function or state variable visibility Rameder et al. (2022)	SWC-108, Visibility of exposed functions Rameder et al. (2022) , Staderini (2022) Function or state variable default visibility Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Function/State visibility error Chaliasos et al. (2023) , Unspecified visibility level Munir and Taha (2023) , Bad visibility Agarwal et al. (2021) , Default visibility Zhou et al. (2022) , No restricted write, Implicit visibility level, Non-public variables accessed by public/external functions, Default/Unintended/Erroneous visibility Rameder et al. (2022)
5B5.1	Function default visibility Wei et al. (2023) , Kushwaha et al. (2022b)	SWC-100, Default function visibility Li et al. (2024c) , Wrong function modifier Vidal et al. (2024a)
5B5.2	State variable default visibility Wei et al. (2023)	Missing visibility modifier in variable declaration Vidal et al. (2024a)
5B6	Bypassable Modifier Fang et al. (2023)	
5C	Signature based vulnerabilities Rameder et al. (2022)	Insufficient signature information Rameder et al. (2022)
5C1	Missing protection against signature replay attack Rameder et al. (2022) , Staderini (2022) , Wei et al. (2023) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	SWC-121, Improper Check against Signature Replay Attacks Vidal et al. (2024a)
5C2	Lack of proper signature verification Rameder et al. (2022) , Wei et al. (2023) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	SWC-122, Method permit() Used for Arbitrary from in transferFrom() Li et al. (2024c) , Improper authenticity check Vidal et al. (2024a)
5C3	Hash collision with multiple variable length arguments Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Kushwaha et al. (2022b)	SWC-133, Incorrect Argument Encoding Vidal et al. (2024a)
5C4	Function Clashing Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
5C5	Signature Malleability Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Staderini (2022) , Wei et al. (2023)	SWC-117, Incorrect Verification of Cryptographic Signature Vidal et al. (2024a)
5D	Identity Verification Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
5E	Usage of public mint or burn Li et al. (2024c)	
5F	Overpowered Role Agarwal et al. (2021)	Check on overpowered role Dia et al. (2021)
6 - Arithmetic bugs		
6A	Integer over- or underflow Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Staderini (2022) , Kushwaha et al. (2022b) , Agarwal et al. (2021) , Chu et al. (2023) , Munir and Taha (2023)	SWC-101, OWASP-SC08, DASP-3, Arithmetic UnderflowOverflow Kushwaha et al. (2022a) , Integer bug Choi et al. (2021) , Value assignment with too many digits Dia et al. (2021)
6A1	Integer overflow Vidal et al. (2024a)	
6A2	Integer underflow Vidal et al. (2024a)	
6B	Unchecked division Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
6B1	Integer division Rameder et al. (2022)	Money leak Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Unexpected result from division Munir and Taha (2023) , Floating point & Precision Rameder et al. (2022)
6B2	Integer Division before Multiplication Li et al. (2024c)	Arithmetic Precision Staderini (2022)
6B3	Divide by Zero Vidal et al. (2024a)	
6C	Integer bugs or Arithmetic Issues Rameder et al. (2022)	Check on arithmetic operation Dia et al. (2021) , Unchecked Math, Wrong operator Rameder et al. (2022)
6C1	Incorrect Calculation Li et al. (2024d)	
6C2	Truncation Bugs Agarwal et al. (2021) , Vidal et al. (2024a)	Integer Truncation Rameder et al. (2022) , Dia et al. (2021)
6C3	Signedness Bugs Agarwal et al. (2021) , Vidal et al. (2024a)	Integer sign Rameder et al. (2022) , Dia et al. (2021)
6D	Array length manipulation Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Agarwal et al. (2021)	Unsafe array length Assignment Li et al. (2024c)

Table 10
Continued.

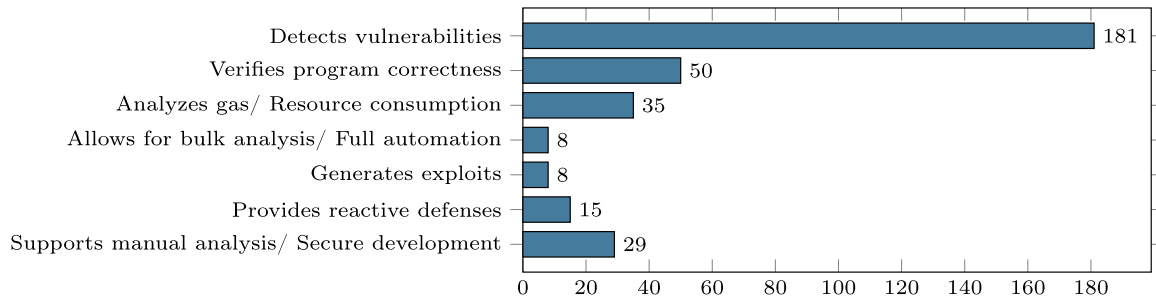
7 - Bad Coding Quality and Programming Language Specifics		
7A	Type cast Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Staderini (2022)	Unsafe type declaration, Type conversion Vivar et al. (2021) , Wrong Function Call Vidal et al. (2024a)
7B	Coding error Rameder et al. (2022)	SWC-129, Typographical error Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Vidal et al. (2024a) , Immutable bugs Rameder et al. (2022) , Vivar et al. (2021) , Coding Bug Rameder et al. (2022)
7C	Bad coding pattern Rameder et al. (2022)	
7D	Deprecated source language features Rameder et al. (2022)	SWC-111, Use of deprecated Solidity function Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)
7E	Use of assembly Rameder et al. (2022) , Agarwal et al. (2021)	Assembly-based vulnerabilities Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Usage of assembly Agarwal et al. (2021)
7E1	Arbitrary jump with function type variable Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023)	SWC-127, Arbitrary jump Staderini (2022) , Control-Flow hijack Choi et al. (2021) , Function type variables assignment using arbitrary values Dia et al. (2021) , Specify function variable as any type Rameder et al. (2022)
7E2	Constant function using assembly code Agarwal et al. (2021)	Constant Function Agarwal et al. (2021) , Interoperability issues with other contracts Vidal et al. (2024a)
7E3	Returning results using assembly code in constructor Rameder et al. (2022)	
7F	Incorrect inheritance order Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023)	SWC-125, Wrong class inheritance order Vidal et al. (2024a)
7G	Variable shadowing Rameder et al. (2022)	SWC-119, Hidden state variable Rameder et al. (2022) , Shadowing state variables Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Variable name Dia et al. (2021) , Tainted memory Munir and Taha (2023) , Shadowing state, Shadowing abstract, Shadowing builtin, Shadowing local Agarwal et al. (2021)
7G1	Use of Same Variable or Function Name In Inherited Contract Vidal et al. (2024a)	
7G2	Variables or Functions Named After Reserved Words Vidal et al. (2024a)	
7G3	Use of the Same Variable or Function Name In a Single Contract Vidal et al. (2024a)	
7H	Misleading source code Rameder et al. (2022)	API inconsistency Rameder et al. (2022)
7H1	Right to Left override Staderini (2022) , Wei et al. (2023) , Rameder et al. (2022)	SWC-130, Function Call with Wrong Arguments Vidal et al. (2024a)
7H2	Hidden built-in symbols Rameder et al. (2022)	Built-in symbol shadowing Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Use of deprecated built-in symbols Dia et al. (2021)
7I	Missing logic Rameder et al. (2022)	OWASP-SC03, Logical errors or dead code Rameder et al. (2022) , Business logic vulnerabilities, Wrong logic Vidal et al. (2024a)
7I1	Code with no effects Rameder et al. (2022) , Wei et al. (2023) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	SWC-135, No Effect Code Execution Vidal et al. (2024a)
7I2	Unused variable Vidal et al. (2024a)	SWC-131, Presence of unused variables Rameder et al. (2022) , Wei et al. (2023)
7I3	Unused return Agarwal et al. (2021) , Li et al. (2024c)	
7I4	Missing return type on function Vidal et al. (2024a)	
7I5	Missing type in variable declaration Vidal et al. (2024a)	Insecure inference Munir and Taha (2023) , Unsafe type inference Rameder et al. (2022)
7I6	Missing variable initialization Vidal et al. (2024a)	Uninitialized local/state variable Rameder et al. (2022) , Uninitialized state variables Chu et al. (2023) , Uninitialized state, Uninitialized local Agarwal et al. (2021) , Initialization of local variables, Initialization of state variables, Missing initialization Dia et al. (2021)
7I7	Missing input validation Li et al. (2024c)	OWASP-SC04, Improper input validation Vidal et al. (2024a)
7I8	Function return type mismatch Vidal et al. (2024a)	
7I9	Parameter type mismatch Vidal et al. (2024a)	
7I10	Wrong type of Function Vidal et al. (2024a)	
7I11	Redundant Functionality Vidal et al. (2024a)	
7J	Insecure contract upgrading Rameder et al. (2022)	Upgradable contract, Patching Rameder et al. (2022)
7K	Inadequate or incorrect logging or documentation Rameder et al. (2022)	
7L	Hardcoded addresses Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Agarwal et al. (2021) , Chen et al. (2022b) , Vidal et al. (2024a)	
7M	Ether lost in transfer Rameder et al. (2022)	Transfer to orphan address Rameder et al. (2022) , Lost of Ether Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Staderini (2022) , Kushwaha et al. (2022b) , Loss of Ether Vivar et al. (2021)
7N	Locked Money Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Agarwal et al. (2021) , Chu et al. (2023)	Unreachable Payable Function Vidal et al. (2024a)
7O	Missing constructor Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Vidal et al. (2024a)	

Table 10
Continued.

7P	Extraneous Field Declaration Vidal et al. (2024a)	
7Q	Inadequate Data Representation Vidal et al. (2024a)	
7R	Map overlap Munir and Taha (2023)	
7S	Missing Interrupter Chen et al. (2022b)	
7T	Missed to implement a fallback function Munir and Taha (2023)	
7U	Improper locking during external calls Vidal et al. (2024a)	
7V	Infinite loop Wei et al. (2023)	Unbounded mass operation Munir and Taha (2023)
8 - Environment Configuration Issues		
8A	Short address Rameder et al. (2022) , Zhou et al. (2022) , Chu et al. (2023) , Staderini (2022)	DASP-9, Check on input address length Dia et al. (2021) , Address shortening Munir and Taha (2023)
8B	Outdated compiler version Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Vidal et al. (2024a)	SWC-102, Pragmas version Agarwal et al. (2021) , Unspecified Compiler version Chen et al. (2022b) , Compiler version Dia et al. (2021)
8C	Floating or no pragma Rameder et al. (2022) , Wei et al. (2023) , Kushwaha et al. (2022b)	SWC-103, Floating compiler version Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Undetermined program version prevalence Vidal et al. (2024a)
8D	Token API violation Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	Token standard incompatibility Rameder et al. (2022) , Unmatched ERC-20 Standard Chen et al. (2022b) , Extraneous exception handling Vidal et al. (2024a)
8D1	Improper or missing event handling Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	Missing Reminder Chen et al. (2022b) , Fake Deposit Munir and Taha (2023) , Missing token verification of exchange Vidal et al. (2024a)
8D2	ERC-20 transfer Vidal et al. (2024a)	
8E	Ethereum update incompatibility Rameder et al. (2022)	SWC-134, Failing to accept new deposit, Deposit Acceptance Munir and Taha (2023) , Message call with hardcoded gas amount Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023)
8E1	Call with hardcoded gas amount Vidal et al. (2024a)	
8E2	transfer() and send() with hardcoded gas amount Vidal et al. (2024a)	
8F	Configuration error Rameder et al. (2022)	
8G	Deprecated APIs Chen et al. (2022b)	
9 - Eliminated/Deprecated syntax		
9A	Callstack depth limit Rameder et al. (2022)	Stack size limit Rameder et al. (2022) , Limited stack size Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Stack size Vivar et al. (2021)
9B	Uninitialized storage pointer Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Munir and Taha (2023) , Vidal et al. (2024a)	SWC-109, Uninitialized storage Agarwal et al. (2021) , Initialization of storage pointer Dia et al. (2021)
9C	Erroneous constructor name Rameder et al. (2022)	SWC-118, Incorrect constructor name Rameder et al. (2022) , Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023) , Wei et al. (2023) , Constructor name Dia et al. (2021) , Wrong constructor name Vidal et al. (2024a)
9D	Double constructor Zaazaa and Bakkali (2023) , Zaazaa and El Bakkali (2023)	
10 - Library Misuses		
10A	Invalid Wrapper Check in Library Huang et al. (2024a)	
10B	Unhandled Exceptions in Library Huang et al. (2024a)	
10C	Inappropriate Library Extension Huang et al. (2024a)	
10D	Inappropriate Using For Huang et al. (2024a)	
10E	Incomplete Function Replacement Huang et al. (2024a)	
10F	Overestimated Library Capability Huang et al. (2024a)	
10G	Underestimated Library Capability Huang et al. (2024a)	
10H	Unnecessary Library Using Huang et al. (2024a)	
11 - Machine Unauditable Functional Vulnerabilities		
11A	Price oracle manipulation Sun et al. (2024)	OWASP-SC02, Price manipulation Sun et al. (2024) , Wang et al. (2024c)
11A1	Price oracle manipulation with AMM Sun et al. (2024)	
11A2	Price oracle manipulation by buying token Sun et al. (2024)	
11B	Erroneous Accounting Zhang et al. (2023d)	
11C	ID Uniqueness Violation Wang et al. (2024d)	ID-related violation Zhang et al. (2023d)
11D	Inconsistent State Updates Wang et al. (2024d)	Erroneous State Updates Zhang et al. (2023d)
11E	Privilege Escalation Zhang et al. (2023d) , Wang et al. (2024d)	
11F	Atomicity Violation Zhang et al. (2023d) , Wang et al. (2024d)	
11G	Contracts Implementation Specific Bugs Zhang et al. (2023d)	
11H	Business Logic Flaw Wang et al. (2024d)	
11I	Cross-Bridge Wang et al. (2024d)	

Table 10
Continued.

12 - Storage & Memory		
12A	Memory Manipulation Li et al. (2024c)	
12A1	Write to arbitrary storage location Vidal et al. (2024a) , Rameder et al. (2022)	SWC-124, Arbitrary write Choi et al. (2021)
12A2	Read from Arbitrary Storage Location Vidal et al. (2024a)	
12B	Static array out-of-bounds Song et al. (2022)	
12C	Dynamic array out-of-bounds Song et al. (2022)	
12D	Misleading data location Chen et al. (2022b)	Modifying storage array by value Li et al. (2024c)
13 - Cross-chain System Vulnerabilities		
13A	Block expansion attack Kaur and Gandhi (2020) , Duan et al. (2023a)	
13B	Data synchronization timeout attack Suhail et al. (2022) , Duan et al. (2023a)	
13C	Failure diffusion Yu et al. (2019) , Duan et al. (2023a)	
13D	Privacy leakage attack Xu and Vadgama (2022) , Duan et al. (2023a)	
13E	Unauthorized access Rizvi et al. (2020) , Duan et al. (2023a)	
13F	Disguised and untrusted notary Sun et al. (2020) , Duan et al. (2023a)	
13G	Transaction forgery attack Lv et al. (2022) , Duan et al. (2023a)	
13H	Denial of service attack Tian et al. (2021) , Duan et al. (2023a)	
13I	Hot wallet attack Chepurnoy and Saxena (2019) , Duan et al. (2023a)	
13J	Wormhole attack Malavolta et al. (2018) , Duan et al. (2023a)	
13K	Sore loser attack Xue and Herlihy (2021) , Duan et al. (2023a)	
13L	Routing attack Careem and Dutta (2020) , Duan et al. (2023a)	
13M	Replay attack Sonnino et al. (2020) , Duan et al. (2023a)	
13N	Collusion attack Schrepel (2019) , Duan et al. (2023a)	
13O	Unrestricted Deposit Emitting Zhang et al. (2022a) , Duan et al. (2023a)	
13P	Inconsistent Event Parsing Zhang et al. (2022a) , Duan et al. (2023a)	
13Q	Unauthorized Unlocking Zhang et al. (2022a) , Duan et al. (2023a)	

**Fig. 3.** Number of tools providing a specific functionality.

(2022) developed a smart contract vulnerability detection model based on multitask learning. In some cases, Machine Learning is supported by Pattern Matching and Syntactical Analysis. For example, GPSCVulDetector ([Liu et al., 2023a](#)) combines graph neural networks and pattern matching for comprehensive detection, while SVScanner ([Zhang et al., 2023a](#)) extracts semantic features from ASTs of different patterns and uses a text CNN to detect contract bugs. Symbolic execution and Taint analysis are also used; for example, SIGUARD ([Zhang et al., 2023b](#)), RN-VulDet ([Qian et al., 2023b](#)) and Achecker ([Ghaleb et al., 2023](#)) combine both methods, SmartFast ([Zhaoxuan et al., 2022](#)) and SESCon ([Ali et al., 2021](#)) focus only on taint analysis while ReDetect ([Yu et al., 2021a](#)), and Ethersolve ([Contro et al., 2021](#); [Pasqua et al., 2023](#)) focus on symbolic execution.

Dynamic analysis tools mostly use mutation testing and fuzzing methods. On the one hand, tools such as SuMo ([Barboni et al., 2022](#)), AMAT ([Sujeetha and Akila, 2023](#)), and MulESC ([Sun et al., 2022b](#)) perform mutation testing to assess the quality of test suites. On the other hand, EVMFuzzer ([Fu et al., 2019](#)), IR-Fuzz ([Liu et al., 2023b](#)), Ity-Fuzz ([Shou et al., 2023](#)), and SynTest-Solidity ([Olsthoorn et al., 2022](#)) perform fuzzing. Effuzz ([Ji et al., 2023](#)) and EtherFuzz ([Wang et al., 2022](#)) combine both techniques, selectively mutating the input for the fuzzing phase.

Some tools perform static and dynamic analyses by combining different methods. Smartscan ([Samreen and Alalfi, 2021](#)) identifies po-

tentially vulnerable patterns and then uses dynamic analysis to confirm their precise exploitability. EtherProv ([Linoy et al., 2021](#)) leverages Solidity source code static and dynamic analysis data through contract bytecode instrumentation and taint analysis. Finally, VulHunter ([Li et al., 2023](#)) uses hybrid attention and multi-instance learning mechanisms to detect vulnerabilities, then applies symbolic execution to validate their feasibility.

Table 11 presents the mapping between analysis methods, the intermediate representations (IRs) they rely on, the input types they process, and the observed frequencies of their occurrences across the analyzed studies. The Control Flow Graph (CFG) is the most widely adopted IR, appearing in nearly all analysis methods. Its prevalence reflects its central role in capturing execution structure and control dependencies—fundamental aspects for both static and dynamic analyses. The use of Data Flow, Transaction Analysis, and Execution Traces spans multiple techniques (e.g., Fuzzing, Taint Analysis, Mutation Testing, Runtime Verification), indicating that dynamic information is crucial for vulnerability detection. This trend highlights a shift toward behavior-aware rather than purely syntactic reasoning. Most methods support both EVM bytecode and source code, though with different emphases: Symbolic Execution, Taint Analysis, and Runtime Verification primarily target bytecode, while Pattern Matching, Abstract Interpretation, and Machine Learning mainly rely on source code. This duality reflects the heterogeneous availability of smart contracts, as not all on-chain contracts have

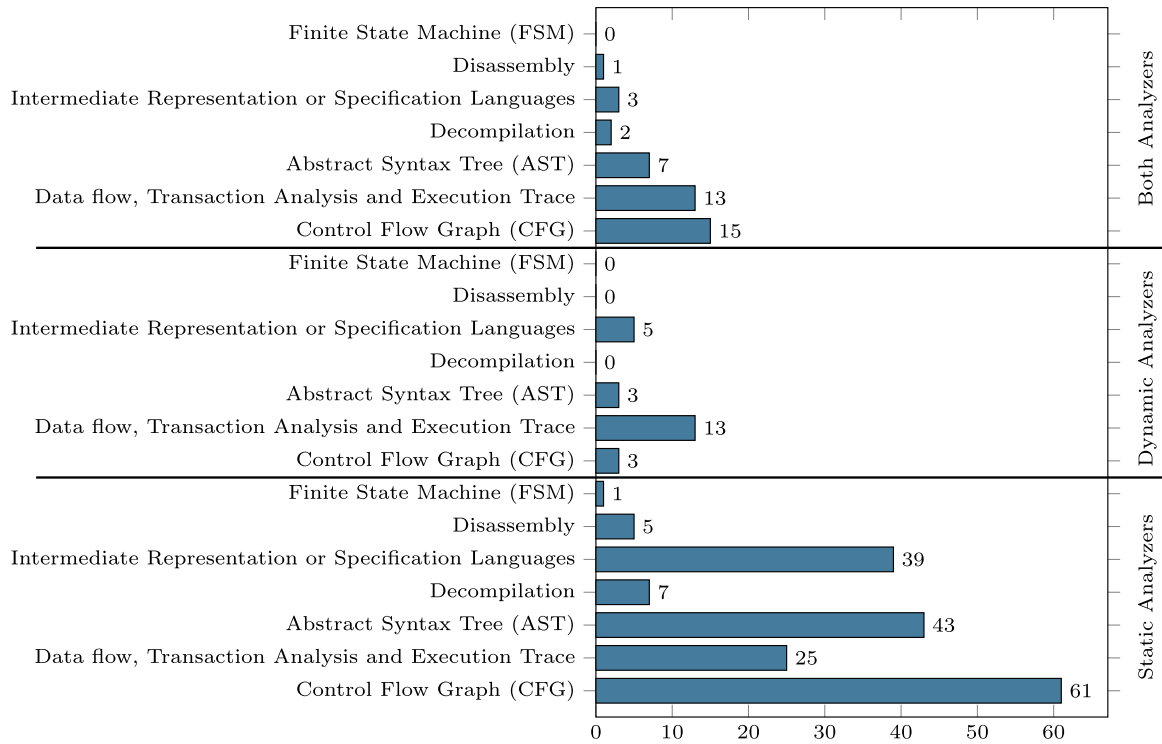


Fig. 4. Number of tools employing a specific code transformation technique.

verified source code. Researchers often combine multiple IRs depending on the abstraction level required by their technique and the type of input available. This diversity illustrates both the maturity and methodological fragmentation of smart contract security analysis. Despite this variety, most methods still depend on CFGs and Data Flow/Transaction Traces, suggesting that researchers tend to reuse conventional program analysis abstractions while overlooking richer or hybrid IRs (e.g., SSA form, semantic graphs, control-data dependency networks) that could capture more nuanced behaviors. Furthermore, Formal Verification and Model Checking appear infrequently, indicating that rigorous methods remain underrepresented compared to heuristic approaches, such as Fuzzing and Machine Learning. Finally, Machine Learning tools show a strong preference for source code (50 occurrences) over bytecode (17), which limits their real-world applicability since most deployed contracts on-chain lack verified source code. Overall, the mapping reveals a research landscape dominated by control-flow-based and source-level analyses, showing clear gaps remain in IR diversity, integration across analysis paradigms, and applicability to real-world bytecode.

Open source tools. The tools identified in our study offer valuable resources and knowledge that can benefit both researchers and practitioners. They can serve as benchmarks and can be extended, enhanced, or integrated into development pipelines. However, only 101 of these tools are open source and publicly available. Of those, few are ready for immediate use, while many remain in the form of demos or experimental implementations designed to evaluate detection accuracy and require further development to become fully functional.

RQ₂ Summary. We collected 219 distinct tools. The tools focus significantly on source code analysis (142 tools) and static analysis methods (145 tools). Although vulnerability detection is the main functionality of these tools (82.5%), there is a significant gap in fully automated solutions that encompass patch and exploit generation. Additionally, only 101 tools are open source, with many still requiring further development to achieve full usability. The study emphasizes the need for improved automation and integration of these tools into an

Integrated Development Environment to improve security development practices.

RQ₂ Key Point. It is unclear which approach performs best for detecting a particular vulnerability. Moreover, few tools are ready for immediate use or easy integration into a development environment. In addition, many solutions are research-oriented with a limited focus on the practical needs of security auditors.

4.3. RQ₃. Which vulnerabilities do the tools address?

Several tools were not included in this mapping for specific reasons. For example, tools that were not explicitly designed to detect vulnerabilities, such as compilers (Mazurek, 2021) and decompilers (Suiche, 2017), were omitted. We also excluded tools that do not focus on specific vulnerabilities, but instead apply machine learning to classify smart contracts as vulnerable (De Salve et al., 2024; Otoni et al., 2023) or use mutation testing without focusing on vulnerability-specific mutation operators (Olsthoom et al., 2022; Sujeetha and Akila, 2023). These tools are too generic to be accurately mapped.

Next, we individually analyze each tool to identify the vulnerabilities it detects. During this mapping process, several challenges arose. Many tools claimed to detect broad categories of vulnerabilities, such as *Gas-related vulnerabilities*, *Access Control Vulnerabilities*, and *Arithmetic vulnerabilities* (Ghaleb et al., 2023; Xu et al., 2021; Shou et al., 2023; Narayana and Sathiyamurthy, 2023; Zheng et al., 2022). However, in our taxonomy, these labels represent categories that include multiple distinct vulnerabilities. Mapping these tools to all vulnerabilities within these categories would lead to broad generalizations. In other cases, tools declared to detect a vulnerability that, in our taxonomy, has different specializations at lower levels. For example, the tool Siguard (Zhang et al., 2023b) declare to detect signature-based vulnerabilities without specifying what they are in detail. In our taxonomy, vulnerability *5C Signature-based vulnerabilities* has five specializations. In this case, we mapped the tools to the vulnerabilities they claim to detect; however, we considered the mapping partial and should be interpreted with caution, as tools may not extend detection to all specialized sub-vulnerabilities

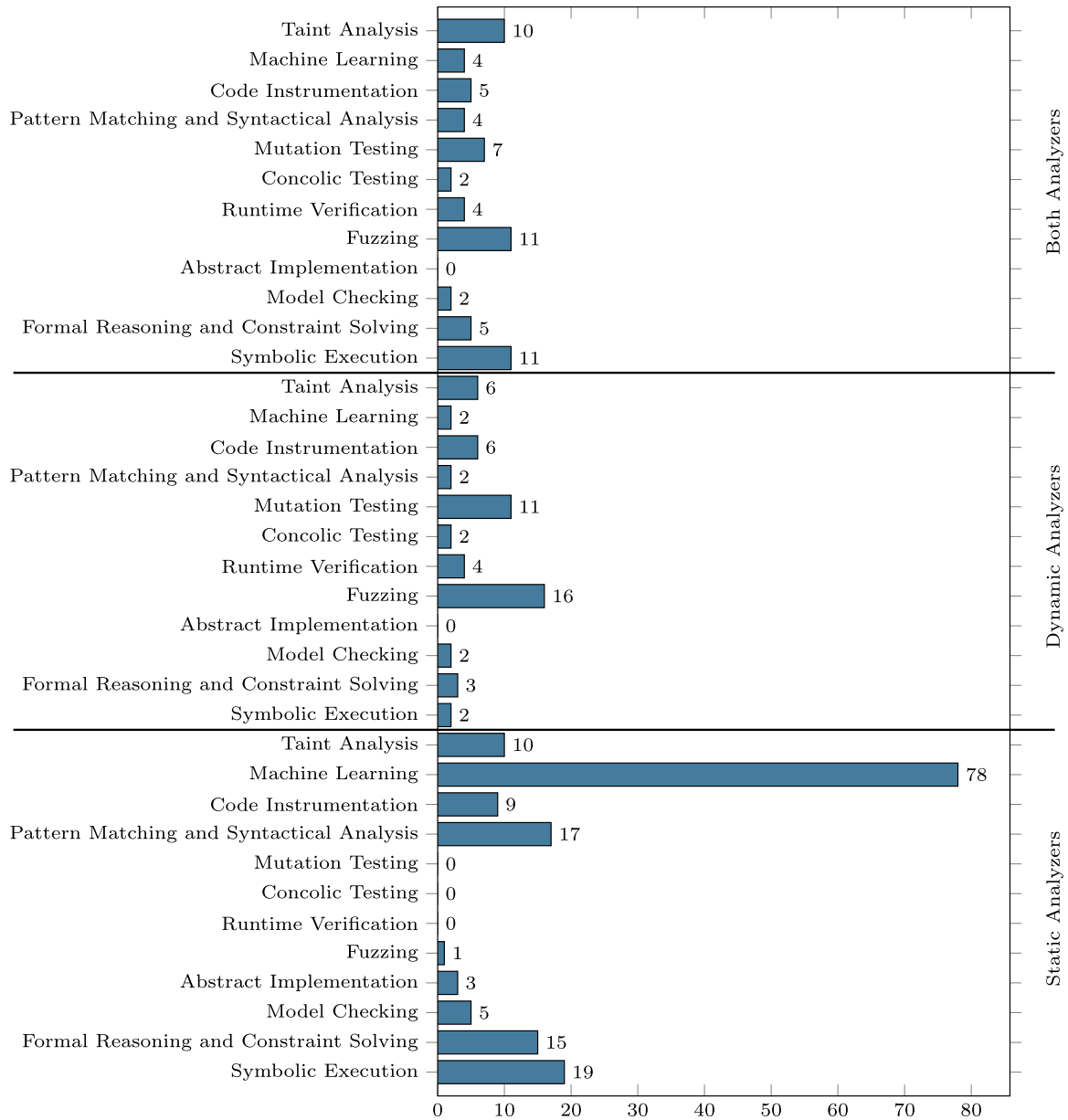


Fig. 5. Number of tools employing a specific method.

at lower levels. Each specialization may be different and may require a different detection mechanism. Performing a truly exhaustive mapping would require an in-depth analysis of each tool's detection logic to determine which second-level vulnerabilities it can identify. Therefore, we have a complete mapping of tools that specify vulnerabilities, closely aligned with our taxonomy, without creating ambiguities in categories and hierarchy.

Our focus was also on tools that facilitate patch generation to address vulnerabilities. For example, tools such as Elysium (Ferreira Torres et al., 2022) that rely on other tools for the detection phase were mapped only to the vulnerabilities for which they can generate patches. Finally, tools that aggregated several others (Vivar et al., 2021; Di Angelo et al., 2023) or that lacked sufficient documentation (Stegeman, 2018; Wu et al., 2020, 2021) were also not fully mapped. For these, we mapped only the well-defined vulnerabilities that matched our taxonomy. The mapping includes 162 tools in total. Of these, 113 were fully mapped, and 49 were partially mapped because the tools identified vulnerabilities that were too generic for precise mapping.

Table 13 shows that the most frequently analyzed vulnerability is Reentrancy, with 130 tools, followed by Timestamp Dependency, with 78 tools, and Integer Overflow/Underflow, with 62 tools.

According to the Systematization of Knowledge by Rezaei et al. (2025), a qualitative analysis of 50 high-impact smart contract exploits between 2022 and 2025 revealed total losses amounting to 1,093,979,779\$. Among these, the Reentrancy vulnerability alone accounted for losses exceeding 118\$ billion. External Dependency Vulnerabilities caused losses for more than 222\$ billion, and Access Control Vulnerabilities accounted for losses exceeding 417\$ billion. A price manipulation vulnerability, not listed among the top 10 detected vulnerabilities, caused a total loss of 279,750,000\$ billion. The multi-tier root-cause framework for smart contract exploits proposed in the SoK (Rezaei et al., 2025) highlighted that incidents are rarely single-shot failures. Only 19 of the 50 exploits relied solely on a code defect; the rest involved chains of two or more weaknesses. The most significant financial losses often stem from failure areas that code-centric audits may overlook, e.g. *Flawed Economic Design & Protocol Logic* and *Proto-*

Table 11

Mapping of methods to intermediate code representations and input types, along with their frequencies.

Method	Intermediate Representation (freq)	Input Type (freq)
Symbolic Execution	CFG (14), Data Flow/ Transaction Analysis/Traces (8), Intermediate Representation/ Specification Language (Opcode) (4), AST (2), Disassembly (2)	EVM Bytecode (15), Source Code (12)
Formal Verification & Constraint Solving	CFG (6), Data Flow/ Transaction Analysis/Traces (4), AST (3), Intermediate Representation/ Specification Language (Opcode) (2), Decompilation/Compiler (1)	EVM Bytecode (4), Source Code (10)
Model Checking	CFG (1)	EVM Bytecode (1), Source Code (1)
Abstract Interpretation	CFG (2), Intermediate Representation/ Specification Language (Opcode) (2), AST (1)	Source Code (2)
Fuzzing	Data Flow/ Transaction Analysis/Traces (11), CFG (10), AST (4), Intermediate Representation/ Specification Language (Opcode) (2), Decompilation/Compiler (1), Disassembly (1)	EVM Bytecode (9), Source Code (15)
Runtime Verification	Data Flow/ Transaction Analysis/Traces (5), CFG (2), AST (2)	EVM Bytecode (2), Source Code (4)
Concolic Testing	Data Flow/ Transaction Analysis/Traces (2), CFG (1), AST (1), Intermediate Representation/ Specification Language (Opcode) (1)	EVM Bytecode (2), Source Code (3)
Mutation Testing	Data Flow/ Transaction Analysis/Traces (7), Intermediate Representation/ Specification Language (Opcode) (2), AST (2), CFG (2), Decompilation/Compiler (1), Disassembly (1)	EVM Bytecode (4), Source Code (9)
Pattern Matching / Rule-Based	CFG (6), Intermediate Representation/ Specification Language (Opcode) (6), AST (6), Data Flow/ Transaction Analysis/Traces (4), Disassembly (2), Decompilation/Compiler (1)	EVM Bytecode (4), Source Code (14)
Code Instrumentation	Data Flow/ Transaction Analysis/Traces (6), AST (5), CFG (5), Intermediate Representation/ Specification Language (Opcode) (2)	EVM Bytecode (5), Source Code (11)
Machine Learning / Deep Learning / AI	CFG (24), Intermediate Representation/ Specification Language (Opcode) (20), AST (17), Data Flow/ Transaction Analysis/Traces (9), Disassembly (2), Decompilation/Compiler (2)	EVM Bytecode (17), Source Code (50)
Taint Analysis	CFG (12), Data Flow/ Transaction Analysis/Traces (11), Intermediate Representation/ Specification Language (Opcode) (5), AST (2), Decompilation/Compiler (2), Disassembly (1)	EVM Bytecode (13), Source Code (10)

col Lifecycle & Governance Failures. When automated tools are insufficient to ensure a satisfactory audit support, several defensive practices that have proven effective at preventing past incidents can be adopted. These include the use of invariants to verify state changes or upgrades, as well as staged roll-outs to gradually deploy and test contract updates.

This knowledge is informative and crucial for understanding the current state of smart contract vulnerabilities. It is worth noting that the tools identified in our study cannot detect all the vulnerabilities in our taxonomy. Of the 192 vulnerabilities, 113 are not addressed by tools. Furthermore, of the 79 vulnerabilities addressed, only 16 are detected by at least 10 tools. In other words, several vulnerabilities can be detected using only a few available tools.

Table 14 provides an overview of the association between detection methods and the vulnerabilities they address, with the frequencies. The results show that no single technique comprehensively covers all vulnerabilities; rather, some methods demonstrate greater versatility and are more widely adopted across the literature. *AI-based*, *Fuzzing*, and *Symbolic Execution* are the most widely used approaches, each addressing multiple high-frequency vulnerability types. AI-based tools are used as generic detection and pattern-recognition strategies, reflecting the growing reliance on data-driven vulnerability analysis. Their widespread use suggests a shift from handcrafted rules to adaptive and learning-based automated systems. Symbolic Execution remains one of the core analysis techniques. It is especially suited to path-dependent vulnerabilities such as reentrancy or arithmetic errors. However, its high computational cost and limited scalability explain its selective application compared to newer approaches. Fuzzing is the third most frequent method. Its focus on runtime behavior makes it particularly effective at detecting vulnerabilities in input validation, exception handling, and transaction state changes. This confirms that fuzzing remains a practical solution for uncovering unexpected execution paths and behavioral flaws. Rule-based and Pattern-Matching techniques focus on recurring vulnerability patterns, particularly those related to unsafe input

handling or data flow (1A1, 2A1, 6A). They provide robust coverage of well-known weaknesses and are often embedded in static analysis frameworks. Formal Verification and Model Checking exhibit lower frequencies but high precision, primarily covering categories such as 1A1, 6A, 2A1, and 3A. They are often used in high-quality academic studies to verify contract correctness, rather than for large-scale vulnerability detection. Their use highlights a trade-off between accuracy and scalability. The results suggest a polarization between traditional and modern approaches. Traditional methods, such as symbolic execution, fuzzing, and formal verification, offer rigorous correctness guarantees but are limited in terms of scalability. Conversely, modern data-driven and stochastic approaches, particularly AI-based analysis and mutation-based approaches, offer higher automation and broader coverage at the cost of precision.

To close the analysis of RQ₃, Table 15 highlights gaps in the literature illustrating the adoption of each detection technique across the 13 vulnerability categories. Detection tools do not cover some categories. In particular, categories such as *4 Resource Consumption & Gas Related Issues*, *7 Bad Coding Quality & Programming Language Specifics*, *8 Environment Configuration Issues*, *9 Eliminated/Deprecated syntax*, *10 Library Misuses*, *12 Storage & Memory*, *13 Cross-chain System Vulnerabilities* are not sufficiently covered by detection tools. The lack of coverage in these categories has practical implications: contracts may remain exposed to vulnerabilities that current tools fail to detect, potentially resulting in economic losses. Some of these categories are inherently more challenging to detect automatically, as they often depend on external context, dynamic interactions, or complex multi-contract architectures. There is a clear need for more comprehensive or hybrid detection approaches that combine static, dynamic, and AI-driven analyses. Concepts such as security smells, anti-patterns, and indicators of technical debt can serve as useful heuristics for identifying vulnerabilities in underexplored categories. Furthermore, creating real-world, cross-chain benchmarks encompassing these neglected categories would support the evaluation and improvement of new detection techniques.

Table 12
A comprehensive list of tools.

Tool	Article	Link	Year	Language	Bytecode	Source Code	Static	Dynamic
SCStudio	Ren et al. (2021a)	url	2021	Python		✓	✓	✓
SmartScan	Samreen and Alalfi (2021)	url	2021			✓	✓	✓
SGUARD	Nguyen et al. (2021)	url	2021	Node.js	✓	✓	✓	✓
WANA	Jiang et al. (2021)	url	2020	Python	✓		✓	✓
Artemis	Wang et al. (2020a)				✓		✓	
SCScan	Hao et al. (2020)					✓	✓	
GasFuzzer	Ashraf et al. (2020)				✓		✓	✓
DEPOSafe	Ji et al. (2020)				✓	✓	✓	✓
ReDefender	Pan et al. (2021)				✓			✓
Sooyeon et al. Tool	Lee and Cho (2021)					✓	✓	✓
OYENTE (Enanced version)	Tian (2019a)				✓		✓	
YuXing et al. tool	Tang et al. (2021)					✓		✓
Targy (built on sFuzz)	Ji et al. (2021b)				✓			✓
OYENTE (Enanced version)	Chen et al. (2019a)				✓		✓	
Pouyan et al.	Momeni et al. (2019a)					✓	✓	
EtherSolve	Contro et al. (2021)	url	2021	Java	✓		✓	
CORREAS et al. (Extends GASOL)	Correas et al. (2021)				✓	✓	✓	
DeeSCVHunter	Yu et al. (2021b)	url	2021	C		✓	✓	
Fuchen et al. (reinforced FISCO-BCOS-evm)		url	2018	C + +	✓			✓
Eth2Vec	Ashizawa et al. (2021)	url	2021	Javascript	✓		✓	
VSCL	Mi et al. (2021)				✓		✓	
ESAF	Vivar et al. (2021)	url	2020	Pseudocode	✓	✓	✓	✓
Yuhang et al.	Sun and Gu (2021)					✓	✓	
Peculiar	Wu et al. (2021)	url	2021	C		✓	✓	
Ben et al.	Wang et al. (2021b)					✓	✓	
Wanqing et al.	Jie et al. (2021)				✓	✓	✓	
EtherProv	Linoy et al. (2021)	url	2021	Python		✓	✓	✓
SmartGift	Zhou et al. (2021b)	url	2021	Python		✓	✓	
AMEVulDetector	Liu et al. (2021b)	url	2021	Python		✓	✓	
SC-VDM					✓		✓	
Xu et al.	Xu et al. (2021)					✓	✓	
GPSCVulDetector	Liu et al. (2023a)	url		Python		✓	✓	
Jianjun et al.	Huang et al. (2021b)				✓		✓	
Ziyuan et al.	Li et al. (2021)					✓	✓	
Shasha et al.	Du and Huang (2020)					✓	✓	
ReJection	Ma et al. (2020)					✓	✓	
MENGLIN et al. (Extends Mythril)	Fu et al. (2019a)				✓		✓	✓
Yuqi et al.	Fan et al. (2021)				✓		✓	
Yang et al.	Yang et al. (2020)					✓	✓	
EVMFuzzer	Fu et al. (2019)	url	2019	Python	✓			✓
CodeNet	Hwang et al. (2022)				✓		✓	
EVMPATCH	Rodler et al. (2020)	url	2021	Python	✓		✓	✓
SmartConDetect	Jeon et al. (2023)	url	2023			✓	✓	
XFuzz	Xue et al. (2024)	url	2022	C + +		✓	✓	✓
RLF	Su et al. (2023)	url	2022	Python		✓		✓
IR-Fuzz	Liu et al. (2023b)	url	2023	C + +		✓		✓
Ity-Fuzz	Shou et al. (2023)	url	2023	Rust		✓		✓
SafeSmartContracts	Tann et al. (2018)	url	2019	Python	✓		✓	
ReChecker (VulDeeSmartContract)	Qian et al. (2020)	url	2020	Python		✓	✓	
DL-MDF	Deng et al. (2023)					✓	✓	
HCC	Giesen et al. (2022)					✓	✓	
DeFinary (built on SCRepair)	Tolmach et al. (2023)	url	2022	JavaScript		✓	✓	✓
NarayanaAndSathiyamurthy2021	Narayana and Sathiyamurthy (2023)					✓	✓	
SmartFast	Zhaoxuan et al. (2022)	url	2022	Python		✓	✓	
MSmart	Fei et al. (2023)					✓	✓	
SuMo	Barboni et al. (2022)	url	2022	JavaScript		✓		✓
ReSuMo	Barboni et al. (2023)					✓	✓	✓
Huang et al.	Huang et al. (2021a)				✓		✓	
M-A-R	Wang et al. (2021d)	url	2021	Python		✓		✓
SMARTTEST	So et al. (2021)					✓	✓	
Porosity	Suiche (2017)	url	2017	C + +	✓		✓	✓
MANDO-GURU	Nguyen et al. (2022)	url	2022	Python		✓	✓	
MANDO-HGT	Nguyen et al. (2023)	url	2023	Python	✓	✓	✓	
SCGRU	Liang and Zhai (2023)					✓	✓	
VulHunter	Li et al. (2023)		2023			✓	✓	✓
Vulpedia	Ye et al. (2022)	url	2022	Python		✓	✓	
ReDetect	Yu et al. (2021a)				✓	✓	✓	
MEVD	Guo et al. (2024b)					✓	✓	
Dynamit	Eshghie et al. (2021)					✓		✓
Aroc	Jin et al. (2022)					✓	✓	
ConFuzzius	Torres et al. (2021)	url	2021	Python		✓	✓	✓
Cai et al.	Cai et al. (2023)					✓	✓	
SmartGraph	Zhukov and Korkhov (2023)					✓	✓	
S-BiGRU	Song et al. (2024)					✓	✓	
GraphSA	He et al. (2023c)					✓	✓	

Table 12
continued.

Tool	Article	Link	Year	Language	Bytecode	Source Code	Static	Dynamic
Duan et al.	Duan et al. (2023b)				✓	✓	✓	
EDIT	De Salve et al. (2024)					✓	✓	✓
ASSBert	Sun et al. (2023)					✓	✓	
GRATDet	Peng et al. (2023)					✓	✓	
Cao et al.	Cao et al. (2023c)					✓	✓	
AutoMESC	Soud et al. (2023)	url	2023	Python	✓	✓	✓	✓
AMAT	Sujeetha and Akila (2023)					✓		✓
RNVulDet	Qian et al. (2023b)	url	2023	Python	✓		✓	✓
SCVDIE	Zhang et al. (2022e)	url	2022	Python		✓	✓	
eTaintier	Ghaleb et al. (2022)	url	2022	Python	✓		✓	
Park	Zheng et al. (2022)				✓		✓	✓
Achecker	Ghaleb et al. (2023)	url	2023	Python	✓		✓	
SIGUARD	Zhang et al. (2023b)	url	2023	Python	✓		✓	
SmartState	Liao et al. (2023)	url	2023	Python	✓		✓	
Qian et al.	Qian et al. (2023c)	url	2023	Python	✓		✓	
SynTest-Solidity	Olsthoorn et al. (2022)	url	2022	TypeScript		✓		✓
Elysium	Ferreira Torres et al. (2022)	url	2022	JavaScript	✓		✓	
Sollicitous (2.0)	Otoni et al. (2023)					✓	✓	
MagicMirror	Feng et al. (2023)					✓	✓	✓
SMARTIAN	Choi et al. (2021)	url	2021	F#	✓		✓	✓
CrossFuzz (ConFuzzius extended)	Yang et al. (2024a)					✓		✓
Block-Gram	Xie et al. (2023)				✓		✓	
EtherGIS	Zeng et al. (2022)				✓		✓	
ESCORT	Lutz et al. (2021)				✓		✓	
SVScanner	Zhang et al. (2023a)					✓	✓	
Effuzz	Ji et al. (2023)					✓		✓
SmartDagger	Liao et al. (2022)				✓		✓	
ReVulDL	Zhang et al. (2023c)	url		Python		✓	✓	
ASGVulDetector	Zhang and Liu (2022)					✓	✓	
BASGVulDetector	Zhang and Liu (2022)				✓		✓	
TxMirror	Yu et al. (2023b)				✓			✓
TxSpector	Zhang et al. (2020a)	url	2020	Go	✓			✓
SPCBIG-EC (SPCNN)	Zhang et al. (2022c)	url	2022	Python		✓	✓	
CBGRU	Zhang et al. (2022b)	url	2022			✓	✓	✓
SmartBugs2.0	Di Angelo et al. (2023)	url	2021	Python	✓	✓	✓	
GraBit	Zhu et al. (2023)					✓	✓	
PSCVFinder	Yu et al. (2023a)					✓	✓	
HGAT	Ma et al. (2023)					✓	✓	
EtherFuzz	Wang et al. (2022)					✓		✓
SoliTester	Hu et al. (2024)					✓		✓
sFuzz2.0	Wang et al. (2023)	url	2023	C + +		✓		✓
HuangEtAl2021	Huang et al. (2021a)				✓		✓	
TIPS	Chen et al. (2023b)	url	2023	Python		✓	✓	
SCsVulLyzer	HajiHosseiniKhani et al. (2024b)	url	2024	Python		✓	✓	
EtherSolve2.0 (Extend EtherSolve)	Pasqua et al. (2023)	url	2023	Java	✓		✓	
MODNN	Zhang et al. (2022d)				✓	✓	✓	
SuperDetector	Dai et al. (2022)					✓	✓	
SolidDetector	Hu et al. (2024)					✓	✓	
SCcheck	Cao et al. (2023b)					✓	✓	
SESCon	Ali et al. (2021)					✓	✓	
Sailfish (built on Slither)	Bose et al. (2022)					✓	✓	
MuIESC	Sun et al. (2022b)	url	2022	Python		✓		✓
FSL-Detect	He et al. (2024a)				✓		✓	
EthVer	Mazurek (2021)	url	2021	Haskell		✓	✓	
Gas Gauge	Nassirzadeh et al. (2023)	url	2021	Python		✓		✓
Pakala	Losi (2024)	url	2018	Python	✓		✓	
SolGuard (Extend Solhint)	Praitheeshan et al. (2021)		2021			✓	✓	
Ethlint		url	2018	JavaScript		✓	✓	
Solitor	Stegeman (2018)	url	2018	Java		✓		✓
SCSVM	Yang et al. (2023)					✓	✓	
SCLMF	Yang et al. (2023)					✓	✓	
Kaya	Wu et al. (2020)	url	2020	JavaScript		✓		✓
ConFuzzius-BI	Sun et al. (2022a)	url	2022	Python		✓	✓	
Horus	Ferreira Torres et al. (2021)	url	2021	Python		✓		✓
Ethchecker	Han et al. (2024)				✓		✓	
Developer	Agarwal et al. (2021)					✓	✓	
DLASC	Liu et al. (2024a)	url	2024	Python	✓		✓	
HARDEN	Lakadawala et al. (2024)				✓		✓	
Jie et al. Framework	Jie et al. (2023)					✓	✓	
Clear	Chen et al. (2024d)	url	2023	Python		✓	✓	
ScType	Zhang (2024)	url	2023	Python		✓	✓	
GPTScan	Sun et al. (2024)	url				✓	✓	
SCVHunter	Luo et al. (2024a)	url	2024	Python			✓	
OVer	Deng et al. (2024)					✓	✓	

Table 12
continued.

Tool	Article	Link	Year	Language	Bytecode	Source Code	Static	Dynamic
DCV	Chen et al. (2024a)					✓	✓	
BlockWatchdog	Yang et al. (2024b)	url	2024		✓		✓	✓
Skyeye	Wang et al. (2024c)	url	2024		✓		✓	
OpenTracer	Chen et al. (2024f)	url	2024					✓
Cobra	Li et al. (2024d)				✓		✓	
AdvScanner	Wu et al. (2024c)	url	2024			✓	✓	
ContractTinker	Wang et al. (2024a)	url	2024	JavaScript		✓	✓	
HighGuard	Eshghie et al. (2024)	url	2024	Python		✓		✓
XploGen	Eshghie and Artho (2024)	url	2024	Python		✓	✓	✓
Trace2Inv	Chen et al. (2024e)					✓		✓
FunRedis	Liu and Song (2024a,b)	url	2024			✓	✓	
SOChecker	Chen et al. (2024c)	url	2024	Python		✓	✓	
Midas	Ye et al. (2024a)	url	2024	Rust				✓
VeriSol-MCE	Godbole and Krishna (2024)		2024			✓		
PonziGuard	Liang et al. (2024)					✓	✓	✓
RLRep	Guo et al. (2024a)	url	2022	Python				
FunFuzz	Ye et al. (2024b)	url	2021	C + +	✓			✓
ContractSentry	Wang and Zhao (2025)					✓	✓	
Oyente (Enhanced version)	JJ and Singh (2024)				✓		✓	
EFCF	Rodler et al. (2023)	url	2023		✓			✓
InvCon +	Liu et al. (2024b)	url	2023			✓		✓
InvCon	Liu and Li (2022)	url	2022	C				✓
ESBMC	Song et al. (2022)					✓	✓	
SmartAxe	Liao et al. (2024)	url	2024		✓		✓	
PrettySmart	Zhong et al. (2024)	url	2024	Python	✓		✓	
SliSE	Wang et al. (2024g)	url	2024	Python		✓	✓	
echidna-parade	Groce and Grieco (2021)	url	2021	Python		✓		✓
Targy	Ji et al. (2021a)				✓			✓
SPCon	Liu et al. (2022)	url	2022	Python		✓		
SigRec	Chen et al. (2021)		2021		✓		✓	
Li et al.	Li (2021)		2021					✓
SmartEmbed	Gao et al. (2020)	url	2021			✓	✓	
ContractCheck	Wang et al. (2024e)	url	2024			✓	✓	
SCAnoGenerator	Zhang et al. (2024)	url	2024	Python		✓	✓	
LENT-SSE	Zheng et al. (2024a)	url	2024	Python		✓	✓	
CSAFuzzer	Yang et al. (2025)				✓		✓	✓
FSL-Detect	He et al. (2023a)				✓		✓	
sGuard +	Gao et al. (2024)					✓	✓	
DFier	Wang et al. (2024i)					✓	✓	✓
BCCC-SCSVullyzer	HajiHosseiniKhani et al. (2024a)				✓	✓	✓	
EVM-Shield	Zhang et al. (2024c)	url	2023			✓	✓	✓
LLMSmartSec	Mothukuri et al. (2024)	url	2024	Python		✓		
ContractArmor	Özdemir Sönmez and Knottenbelt (2024)	url	2024					
SafeCheck	Chen et al. (2024b)				✓		✓	
SolGPT	Zeng et al. (2023)	url	2024			✓		
ReenSAT	He et al. (2024c)					✓	✓	
Erinys	Dong et al. (2024)					✓		✓
ParSE	He et al. (2024b)					✓		✓
FFGDetector	Liu and Jiao (2024)					✓	✓	
MuFuzz	Qian et al. (2024)	url	2024	C + +		✓		
MindTheDApp	Ibba et al. (2024)	url	2024	Python		✓	✓	
CSCV	Ding et al. (2024)					✓		✓
SmartInv	Wang et al. (2024d)	url	2024			✓	✓	
TechyTech	Khan and Namin (2024)					✓		✓
SCVD-SA	Wang et al. (2024b)					✓	✓	
SERNET-T	Guo et al. (2024c)				✓			
FEMD	Li et al. (2024b)					✓	✓	
SESCD	Dai et al. (2024)					✓	✓	
FELLMVP	Luo et al. (2024b)					✓	✓	
DeFiRanger	Wu et al. (2023)						✓	
SoloSphere	Khanzadeh and Alalfi (2024)						✓	
DeepFusion	Chu et al. (2024)	url				✓	✓	
ContractGNN	Wang et al. (2024f)	url				✓	✓	
SCheck	Cao et al. (2023a)					✓	✓	
EvoFuzzer	Li et al. (2024a)					✓	✓	✓
Jain et al	Jain and Tripathi (2024)					✓	✓	
DefectChecker	Chen et al. (2022a)	url	2021	Java	✓		✓	
SoMo	Fang et al. (2023)	url	2023	Python		✓	✓	
MuSe	Iuliano et al. (2025a)	url	2025	JavaScript		✓	✓	

Table 13

Top 10 detected vulnerabilities, impact, and losses in 2022/25 range. The * indicates the vulnerability contributes to the shown amount.

Id	Vulnerability	Detection Tools	Impact	Losses, rezaei2025sokrootcause1
1A1	Reentrancy	130	High	118,501,279\$
2A1	Timestamp dependency	78	Low	*222,750,000\$
6A	Integer overflow/underflow	62	Medium	11,800,000\$
3A	Unchecked low-level call/send return values	51	Medium	11 600 000\$
1D	Vulnerable delegatecall	35	High	4,800,000\$
5A1	Authorization via transaction origin	32	Medium	*417,950,000\$
5B1	Unprotected selfdestruction	31	High	*417,950,000\$
2B	Transaction-ordering dependency	30	Medium	*222,750,000\$
1J	Greedy contract	26	Medium	41,850,000\$
2A	Block State Dependency	15	Low	*222,750,000\$

Table 14

Mapping of methods used to detect vulnerability and their frequencies.

Method	Vulnerability Id (frequency)	Tools
Machine Learning / Deep Learning / AI	1A1 (57), 2A1 (44), 6A (22), 3A (17), 5A1 (15), 2B (13), 5B1 (10), 1D (10), 9A (8), 1J (6)	84
Symbolic Execution	1A1 (16), 3A (9), 6A (8), 1D (7), 1J (7), 2A (7), 2B (6), 5B1 (5), 5A1 (4), 2A1 (4)	32
Fuzzing	1A1 (19), 1D (15), 1J (14), 6A (13), 2A1 (11), 3A (11), 2A2 (10), 3B1 (7), 5B1 (7), 2A (5)	28
Taint Analysis	1A1 (14), 6A (10), 3A (8), 1D (7), 2A1 (7), 1J (6), 5B1 (6), 2A (5), 3C (5), 5A1 (4)	26
Pattern Matching / Rule-Based	1A1 (12), 2A1 (9), 6A (8), 5B1 (6), 5A1 (5), 3A (5), 2B (5), 8B (4), 1J (4), 2C (4)	23
Formal Verification & Constraint Solving	1A1 (9), 6A (8), 2A1 (7), 3A (5), 1D (4), 1J (4), 2B (3), 5A1 (3), 5B1 (3), 4A (2)	23
Code Instrumentation	1A1 (11), 3A (6), 2A1 (4), 6A (4), 5A1 (4), 5B1 (3), 2B (2), 1B (2), 1J (1), 2B1 (1)	20
Mutation Based	1A1 (11), 1D (9), 1J (9), 3A (9), 6A (9), 2A1 (6), 2A2 (5), 5B1 (5), 2A (4), 2B (4)	18
Model Checking	3A (2), 6A (1), 1A1 (1), 1C (1), 11A (1)	9
Runtime Verification	1A1 (5), 5A1 (2), 6A (2), 5B5 (1), 1D (1), 1J (1), 2B (1), 5B1 (1), 5B2 (1), 7I6 (1)	8
Concolic Testing	1A1 (4), 6A (2), 3A (2), 5A1 (1), 1D (1), 2A (1), 2B (1), 5B1 (1), 1C (1), 11A (1)	4
Abstract Interpretation	1A1 (2), 3A (1), 5B1 (1), 5B5.1 (1), 5B5.2 (1), 6A (1), 7G (1), 8B (1)	3

Table 15

Mapping between detection methods and vulnerability categories. A checkmark indicates that at least one vulnerability in the corresponding category is addressed by the method.

Detection Method	1	2	3	4	5	6	7	8	9	10	11	12	13
Symbolic Execution	✓	✓	✓		✓	✓							
Formal Verification & Constraint Solving	✓	✓	✓	✓	✓	✓							
Model Checking	✓		✓			✓					✓		
Abstract Interpretation	✓		✓		✓	✓	✓	✓					
Fuzzing	✓	✓	✓		✓	✓							
Runtime Verification	✓	✓			✓	✓	✓				✓		
Concolic Testing	✓	✓	✓		✓	✓					✓		
Mutation Based	✓	✓	✓		✓	✓							
Pattern Matching / Rule-Based	✓	✓	✓		✓	✓		✓					
Code Instrumentation (ALGO)	✓	✓	✓		✓	✓							
Machine Learning / Deep Learning / AI	✓	✓	✓		✓	✓			✓				
Taint Analysis	✓	✓	✓		✓	✓							

RQ₃ Summary. We mapped 162 tools to our taxonomy. In particular, 113 tools were fully mapped while 49 were only partially mapped due to generic descriptions of the vulnerabilities they identified. The remaining 57 tools were not mapped due to their functionalities or poor documentation. The results indicate that the Reentrancy vulnerability is the most commonly analyzed, detected by 130 tools, followed by Timestamp Dependency (78 tools) and Integer Overflow/Underflow (62 tools).

RQ₃ Key Point. Our study revealed significant coverage gaps: 113 vulnerabilities in our taxonomy remain unaddressed. Many tools lack clear information about which Solidity versions they support, making it difficult to assess their applicability. Tools rarely specify their compatibility with the Solidity pragma. As the language evolves, changes introduced in new versions can significantly affect the performance of the tools. Even if a tool is designed to detect a specific vulnerability, its effectiveness may vary across different Solidity versions. This often leads to inconsistent results and an increased likelihood of false positives. Moreover, entire categories are not covered by detection tools.

4.4. RQ4. How are the tools evaluated?

Table 18 shows collected benchmarks. The “Discussed in” column contains a paper that cites the benchmark, while the “Publication” column contains a paper that releases the benchmark. We collected 133 benchmarks, of which 100 were used to evaluate and compare the performance of multiple tools; we could not find information regarding the usage of the remaining 33 benchmarks. Most benchmarks, i.e., 76, contain only smart contracts written in Solidity, while 18 include only bytecode, and 10 include both. We could not find information on the remaining 29 benchmarks. Furthermore, some tools did not release the benchmarks used for their evaluation, and some benchmarks are closed to users and require access authorization (Vulpedia’s benchmark,⁵ SCS-Vul-Lyzer benchmark.⁶) We found that 77 of the 133 benchmarks are public

⁵ https://drive.google.com/file/d/1kizsz0_8B8nP4UNVr0gYjaj25VVZMO8C/edit

⁶ <https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/>

Table 16
Distribution of smart contracts across pragma versions.

Pragma Version	0.3.x	0.4.x	0.5.x	0.6.x	0.7.x	0.8.x	Total
Number of SC	8	83,631	10,338	5580	325	12	99,894
Percentage (%)	0.01	83.72	10.35	5.58	0.33	0.01	100

and accessible, providing a significant resource for the community. In addition, 27 benchmarks include analyses of tool evaluations or experiment results.

We manually reviewed the 77 publicly accessible benchmarks and categorized them as either real-world (RW) or synthetic (Syn). As a result, 47 are real-world, five are synthetic, and eight are both. The remaining 17 benchmarks lack sufficient documentation, so we were unable to classify them. Most synthetic benchmarks are created using SolidiFi.

We conducted a comprehensive analysis of benchmark coverage and composition, manually examining all publicly accessible datasets to determine which Solidity versions, identified via pragma directives, were used for tool evaluations. In total, we extracted 130,974 smart contracts. After removing duplicates by stripping spaces, tabs, and comments and applying a hash function, we obtained 111,223 unique contracts. Excluding contracts without pragma directives left 99,894 contracts, categorized by version in Table 16. Over 83.7% of these contracts are written in Solidity 0.4.x, reflecting the heavy reliance on legacy contracts from the early Ethereum ecosystem. Only 10.3% and 5.5% use versions 0.5.x and 0.6.x, respectively, showing that more recent releases remain underrepresented. This imbalance may limit the generalizability of tool evaluations, as modern compiler features, syntax changes, and security improvements (e.g., 0.7.x and 0.8.x) are rarely tested. Future benchmarks should prioritize the inclusion of newer versions to ensure realistic and comprehensive evaluation.

Next, we assessed curated smart contracts with labeled or manually injected vulnerabilities. From 21,821 collected contracts, 6319 duplicates were removed, and an additional 3807 lacked pragma directives, resulting in 11,695 contracts mapped to our proposed vulnerability taxonomy for uniform labeling. Table 17 presents vulnerability frequencies across Solidity versions, again highlighting the dominance of legacy 0.4.x contracts. The distribution is highly skewed: only a few vulnerabilities, such as 2A1 *Timestamp Dependency*, 1A1 *Reentrancy*, 3A *Unchecked low-level call*, and 6A *Integer over-underflow*, appear more than 1000 times, while a small subset occurs over 100 times. Most vulnerabilities are represented in fewer than 100 contracts, and some are even fewer than 10 (excluded from the table for clarity), resulting in a long-tail distribution. This suggests that benchmarks focus on well-known, easily reproducible vulnerabilities, leaving many types underrepresented and potentially inflating the performance metrics of tools. The ongoing evolution of Solidity continues to impact static analyzers. Each new version could introduce syntax changes, security mechanisms, and compiler checks, requiring continuous adaptation of analysis tools. In a complementary study, we investigate how Solidity versions affect the performance of vulnerability detection tools (Iuliano et al., 2025b).

Considering only vulnerabilities that occur more than 10 times in a benchmark, the number of vulnerabilities sufficiently represented is 25, about 13% of the taxonomy. Without considering emerging vulnerabilities such as Cross-chain vulnerabilities, Library Misuse vulnerabilities, and Machine-Unauditable vulnerabilities that may not yet be present in a benchmark, the vulnerability representation remains low at 16%, with 25 out of 156. The remaining 131 vulnerabilities are either insufficiently represented in benchmarks or unbenchmarked. The 62% of unbenchmarked vulnerabilities (82 out of 131) are not covered by any tool. We can assume that, on the one hand, the lack of benchmarks prevents the development of a vulnerability-detection tool, which explains the low coverage of existing tools. On the other hand, the remaining 49 vulner-

abilities (38%) are covered by tools; however, their representativeness is too low, or benchmarks are unavailable.

RQ₄ Summary. We collected 133 benchmarks used for evaluating tool performance. The majority (76) contain only Solidity contracts, while some include bytecode or both. In particular, 77 benchmarks are publicly accessible and 27 benchmarks provide tool evaluation results that contribute valuable data to the community.

RQ₄ Point. The most commonly used dataset in the literature is SmartBugs-wild. The most frequently used Solidity version in tool evaluations is 0.4.x, whereas real-world DApps predominantly use 0.8.x. Existing benchmarks are generally small and have not been updated to the latest version of Solidity, reducing their relevance to current smart contract development practices.

5. Discussion and implications

This section discusses the differences between our taxonomy and previous ones, the evolution of the tools during the 2016–2021 period, the updated SLR (Rameder et al., 2022), and the research opportunity highlighted by our results.

5.1. Differences and inconsistencies between taxonomies

In the following, we provide a synthesis concerning the differences and inconsistencies found across different taxonomies.

Differences. As highlighted in the literature review that we updated (Rameder et al., 2022), we confirm that some vulnerabilities describe specific, limited cases of a broader problem. Therefore, we organized our taxonomy into more levels to facilitate understanding of the high-level problems. This structure also allows us to highlight specific vulnerabilities that could arise from high-level issues.

A significant contribution concerns the vulnerability categories identified and utilized in our taxonomy. We have brought clarity to the taxonomy by using the various categories identified in the main taxonomies found in the literature. In particular, we focused on taxonomies that classify vulnerabilities by their causes or effects, using specific categories. We have observed that the intersection of categories from different taxonomies, such as Li et al. (2024c), Vidal et al. (2024a), Rameder et al. (2022), does not yield a complete set to categorize all vulnerabilities present in the literature. In our taxonomy, we first included categories common to the main taxonomies, then added those partially covered by a few of them. For example, the taxonomy of Li et al. (2024c) includes the category Storage and Memory, not present in previous taxonomies like OpenSCV (Vidal et al., 2024a) or the one of Rameder et al. (2022). We decided to include the category in our taxonomy to better categorize vulnerabilities related to the incorrect use of memory or storage. Another example concerns vulnerabilities in libraries. OpenSCV (Vidal et al., 2024a), for instance, does not delve into the misuse of libraries and the vulnerabilities they introduce, as discussed in a previous study by Huang et al. (2024a). We have dedicated a category to library vulnerabilities also because according to Huang et al. (2024a), vulnerable libraries are still spreading after fixed library versions are released, which means 10A, 10B, and 10C are long-term risks to contract security. Another new category concerns the vulnerabilities considered by Zhang et al. (2023d) as Machine Unauditable Bugs (MUBs), which are not detectable with static analysis and require a deeper semantic understanding of the code. The category also includes the severe Price oracle manipulation vulnerability (11A). Finally, we have introduced a category for vulnerabilities that may occur in cross-chain systems. This last category highlights the main issues that can arise when using two or more blockchains together in a system.

Inconsistencies. The work to develop our taxonomy highlighted significant inconsistencies across existing taxonomies in the literature. These inconsistencies arise from differences in the categories used, depending

Table 17
Number of labeled smart contracts for each vulnerability across pragma versions.

Pragma	1A1	1B	1B1	1B2	1D	2A1	2A2	2B	2B1	2C	3A	3B1	3C	1J	4I	4D	5A1	5B5	5B	5B1	5B2	5C3	6A	7I	8A
0.4.x	1390	288	0	0	74	1586	367	10	4	272	1162	8	20	75	6	0	22	0	19	101	46	0	5139	13	1
0.5.x	83	0	30	33	5	37	0	74	32	1	112	2	0	48	82	33	74	34	32	25	0	26	64	9	35
0.6.x	17	1	13	15	3	0	0	13	0	0	35	0	0	10	0	13	13	12	0	27	0	2	15	0	3
0.7.x	0	0	0	0	3	0	0	0	0	0	0	0	0	2	0	0	0	0	3	0	0	0	0	0	0
0.8.x	1	0	0	0	2	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Total	1491	289	43	48	87	1623	367	97	36	273	1310	10	20	135	88	46	109	46	51	156	46	28	5218	22	39

on what each taxonomy considers relevant, as well as in how vulnerabilities are assigned to categories and how alternative names or synonyms for the same vulnerability are grouped.

For instance, in at least four cases (Unsafe Type Inference, Unexpected Ether Balance, External Library Calling, Insufficient Gas Griefing), OpenSCV (Vidal et al., 2024a) assigns the same synonym to multiple distinct vulnerabilities. A clear example is the term *External Library Calling*, which appears as an alternative name for 1.7.2 *Unsafe External Library Call* and 5.14 *Use of Malicious Libraries*, even though they are treated as separate entries in two different categories of the taxonomy. Similarly, 6.1.3 *Improper Locking* and 1.4 *Improper Locking During External Calls* are placed under different categories despite sharing the alternative name *Unexpected Ether Balance*. These examples illustrate a core issue that arises when consulting a taxonomy, particularly when starting from alternative names. Let us say that a vulnerability detector finds the Unexpected Ether Balance vulnerability. For an auditor, it becomes unclear which vulnerability is actually being referenced. This can result in multiple entries, sometimes in different categories, with other root causes and impacts. Our taxonomy addresses these challenges by aiming to unify and clarify the categorization and naming of vulnerabilities. A key focus of our work has been aggregating alternative names and synonyms used in the literature to refer to the same vulnerability. This approach reduces ambiguity and confusion, thereby improving the overall usability of the taxonomy. Furthermore, each category in our taxonomy represents the root cause of the vulnerability, enabling auditors to identify where the vulnerability may lie quickly.

Another example involves 3.1 *Improper Gas Requirements Checking*, which treats as synonyms two vulnerabilities that Rameder et al. (2022) classify under entirely different categories. Specifically, *Under-priced opcodes* is categorized under *Resource Consumption & Gas-Related Issues*, while *Gasless send* is placed under *Exception & Error Handling Disorders*. Rameder et al. (2022) note *Gasless send* as a potential subcategory of *Unexpected Throw or Revert*. Our taxonomy explicitly includes it as a subcategory, providing a more coherent, structured classification. Inconsistencies among taxonomies significantly impact the mapping between tools and the vulnerabilities they are designed to detect. This issue becomes particularly evident in primary studies that propose new tools. As discussed in Section 4.3, some tools claim to detect vulnerabilities using names that, in existing taxonomies, refer to entire categories rather than specific issues. An exception to this is the work by Yang et al. (2025), which adopts a clear and structured approach to describing each vulnerability their tool can detect. For example, the vulnerability Block State Dependency is defined using a precise logical formula: $(\text{TimestampOp} \vee \text{BlockNumberOp}) \wedge \text{TransferEther}$. The formula effectively outlines the specific conditions under which the vulnerability occurs. To our knowledge, this is the only study that provides such detailed specifications for general vulnerability categories, making it easier to understand what the tool is analyzing. Thanks to this level of detail, it becomes clear that the vulnerability Blockhash Dependency falls outside the scope of the tool. As a result, performance analysis becomes more straightforward and avoids introducing false negatives. In contrast, tools that do not specify how a vulnerability is detected or rely on vague or broad definitions are more likely to produce a high number of false positives.

Comparison with standard taxonomies. We mapped our taxonomy to the SWC Registry, the OWASP Top 10, and the DASP Top 10. The SWC Registry is fully integrated into our taxonomy. We identified a suitable location for each vulnerability in the SWC Registry, facilitating navigation and enhancing the usability of our taxonomy. We also mapped our taxonomy to the OWASP Top 10 and the DASP Top 10, identifying two issues. OWASP-SC10 *Denial of Service* and DASP-5 *Denial of Service* were not mapped because they are too broad. In fact, the examples of DoS provided by OWASP include *DoS with block gas limit*, *DoS with failed call*, *DoS Costly Pattern and Loops*, while the examples of DoS provided by DASP include *DoS with block gas limit*, and *DoS with unexpected throw*. In our taxonomy, each DoS example appears as a distinct entry (4I, 4I1, 3E, 3F). Mapping each DoS entry to OWASP-SC10 and DASP-5 would create ambiguity, as different types of DoS caused by different causes are mapped to the same identifier. Therefore, we did not map the DoS category. Another issue is the OWASP-SC07 Flash Loan Attack. The flash loan is a type of attack used to manipulate the price of an asset. In fact, the example provided by OWASP utilizes the Flash Loan attack to exploit the vulnerability OWASP-SC02 Price Oracle Manipulation. As the name suggests, OWASP-SC07 is a type of attack; therefore, we did not map it to our taxonomy. Finally, DASP-10 is not a vulnerability; it is a catch-all category intended to remind people that there are still classes of vulnerabilities that have not yet been identified.

Cross-chain system vulnerabilities and attacks. Although our review of the literature focuses mainly on smart contract vulnerabilities in Ethereum, it has also revealed emerging insights into other types of vulnerabilities. In particular, several studies (Duan et al., 2023a; Lee et al., 2023; Zhang et al., 2022a) have identified a range of problems associated with cross-chain technology. The taxonomies proposed in the literature do not capture this type of vulnerability. Cross-chain technologies have been developed to overcome the interoperability challenges between heterogeneous blockchain systems. Cross-system architectures may integrate Ethereum with other blockchains. In 2016, BTC-Relay (Btc relay, 2016) introduced a one-way cross-chain mechanism between Bitcoin and Ethereum based on a relay chain. That same year, Vitalik Buterin published *Chain Interoperability, buterin2016chain*, providing a comprehensive analysis of the interoperability problem in blockchain ecosystems. Cross-chain smart contracts have shown great potential, particularly in DeFi. However, technical incompatibilities, such as those between Ethereum and Fabric, limit the transfer of information between different blockchains (Buterin, 2016). As a result, various token types proliferate across platforms, complicating token exchange and transaction data sharing (He et al., 2020). Duan et al. (2023a) compiled a catalog of 14 vulnerabilities affecting cross-chain systems, along with the corresponding attacks, their consequences, and potential defense strategies. In a separate study, Zhang et al. (2022a) conducted an in-depth analysis of the security of cross-chain bridges, identifying three new categories of security bugs. They also introduced XSCOPE, ZHANG2022XSCOPE, an automated tool designed to detect security violations in cross-chain bridges and uncover real-world attacks.

Table 18
A list of the benchmarks used in the literature.

In	Papers	Tool/Project	Link	#Contracts	Results	Source	Byte	Address	RW/Syn
Piantadosi et al. (2022)	Ashouri (2020)			12			✓		
Piantadosi et al. (2022)	Azzopardi et al. (2018)								
Piantadosi et al. (2022)	Bhargavan et al. (2016)			112,802			✓		
Piantadosi et al. (2022)	Chang et al. (2019)	sCompile		36,099		✓			
Piantadosi et al. (2022)	Chapman et al. (2019)	Deviant		3		✓			
Piantadosi et al. (2022)	Chatterjee et al. (2019)			36,764		✓			
Piantadosi et al. (2022)	Chen et al. (2017)			16,000		✓			
Piantadosi et al. (2022)	Dong et al. (2019)			2		✓			
Piantadosi et al. (2022)	El-Dostuky and Eladl (2019)	DOORchain				✓			
Piantadosi et al. (2022)	Feist et al. (2019b)	Slither		36,000		✓			
Piantadosi et al. (2022)	Fu et al. (2019b)			10,000		✓			
Piantadosi et al. (2022)	Gao et al. (2019)	Easyflow					✓		
Piantadosi et al. (2022)	Grech et al. (2018)	MadMax				✓			
Piantadosi et al. (2022)	Grossman et al. (2017)			3,444,354		✓			
Piantadosi et al. (2022)	He et al. (2019)			18,498		✓			
Piantadosi et al. (2022)	Hirai (2017)					✓			
Piantadosi et al. (2022)	Honig et al. (2019)					✓			
Piantadosi et al. (2022)	Jiao et al. (2020)			482		✓			
Piantadosi et al. (2022)	Labbib et al. (2020)					✓			
Piantadosi et al. (2022)	Lai and Luo (2020)			7000		✓			
Piantadosi et al. (2022)	Li et al. (2019)	MuSC				✓			
Piantadosi et al. (2022)	Ma et al. (2019)	EVM*		10		✓			
Piantadosi et al. (2022)	Medeiros et al. (2019)	SolUnit				✓			
Piantadosi et al. (2022)	Momeni et al. (2019b)			13,745		✓			
Piantadosi et al. (2022)	Mossberg et al. (2019)			100		✓			
Piantadosi et al. (2022)	Nguyen et al. (2019)	Manticore				✓			
Piantadosi et al. (2022)	Nikolic et al. (2018)			970,898		✓			
Piantadosi et al. (2022)	Park et al. (2018)					✓			
Piantadosi et al. (2022)	Peng et al. (2019)			1838		✓			
Piantadosi et al. (2022)	Sergey et al. (2019)	SIF, SolAnalyser, Oyente, Securify, Maian, SmartCheck, Mythril				✓			
Piantadosi et al. (2022)	Shishkin (2019)					✓			
Piantadosi et al. (2022)	Tian (2019b)			20		✓			
Piantadosi et al. (2022)	Torres et al. (2018)	Osiris	url	50,535	✓		✓	✓	RW
Piantadosi et al. (2022)	Tsankov et al. (2018)	Solidify		24,594	✓		✓		
Piantadosi et al. (2022)	Tsankov et al. (2018)	Solidify		100	✓		✓		Syn
Piantadosi et al. (2022)	Wang et al. (2020b)					✓			
Piantadosi et al. (2022)	Ye et al. (2020)			31,097		✓			
Piantadosi et al. (2022)	Zhang et al. (2020b)			49,522		✓			
Piantadosi et al. (2022)	Zhang et al. (2020c)			2,214,409		✓			
Piantadosi et al. (2022)	Song et al. (2019)	Oyente, ContractWard		49,502		✓			
Zaazaa and El Bakkali (2023), Piantadosi et al. (2022)	Gao et al. (2021)	Deckard, SmartCheck, SMARTEMBED	url	22,725		✓		✓	RW

Table 18
continued.

In	Papers	Tool/Project	Link	# Contracts	Results	Source	Byte	Address	RW/Syn
Zaazaa and El Bakkali (2023), Piantadosi et al. (2022)	Tikhomirov et al. (2018)	Smartcheck	url	4600	✓	✓	✓		RW
Zaazaa and El Bakkali (2023), Piantadosi et al. (2022)	Wang et al. (2019)	NPChecker, Securify, Oyente, Mythril	url	30,000	✓	✓	✓		RW
Zaazaa and El Bakkali (2023)		Smartcheck, Oyente, Mythril, Securify, Slither, Osiris, DeesCVHunter	url		✓	✓	✓		RW
Zaazaa and El Bakkali (2023)		Smartcheck, Oyente, Mythril, Securify, Slither, Osiris, DeesCVHunter	url		✓	✓	✓		RW
Zaazaa and El Bakkali (2023)		Slither	url		✓	✓	✓	✓	RW
Zaazaa and El Bakkali (2023)		Yingjie et al., SmartBugs, SolidIFI	url		✓	✓			Syn
Zaazaa and El Bakkali (2023)	Qian et al. (2020)	VulDeeSmartContract (BLSTM-ATT), Securify, SmartCheck, Mythril, Oyente	url		✓	✓			RW
Zaazaa and El Bakkali (2023)	Liao et al. (2019)	SolidAudit, Oyente, Remix	url	21,044	✓	✓	✓	✓	RW
Zaazaa and El Bakkali (2023)	Wu et al. (2021)	Peculiar	url	40,742	✓	✓	✓	✓	RW
Zaazaa and El Bakkali (2023)	Ashraf et al. (2020)	GasFuzzer	url	3170	✓	✓	✓	✓	RW
Zaazaa and El Bakkali (2023)	Samreen and Alalfi (2021)	SmartScan, SmartCheck	url	500					
Zaazaa and El Bakkali (2023)	Nguyen et al. (2021)	SGUARD		5000					
Zaazaa and El Bakkali (2023)	Pan et al. (2021)	ReDefender		204					
Zaazaa and El Bakkali (2023)	Pan et al. (2021)	ReDefender		395					
Zaazaa and El Bakkali (2023)	Contro et al. (2021)	EtherSolve		1000					
Zaazaa and El Bakkali (2023)	Ashizawa et al. (2021)	Eth2Vec		95,152					
Zaazaa and El Bakkali (2023)	Zhou et al. (2021b)	SMARTGIFT							
Zaazaa and El Bakkali (2023)	Liu et al. (2021b)	AME (CGE)		40,932	✓	✓			
Zaazaa and El Bakkali (2023)	Huang et al. (2021b)	Jianjun et al tool		2,000,000			✓		
Zaazaa and El Bakkali (2023)	Ferreira Torres et al. (2020)	AEGLS, ECFChecker, Sereum		675,444	✓	✓			
Zaazaa and El Bakkali (2023)	Sungjae Hwang (2020)	SC Analyzer	url	55,046	✓	✓			
Zaazaa and El Bakkali (2023)	Nguyen et al. (2020)	stfuzz, ContractFuzzer, Oyente		4112	✓	✓			
Zaazaa and El Bakkali (2023)	Wang et al. (2020a)	Artemis, Maian		12,899	✓	✓			
Zaazaa and El Bakkali (2023)	Hao et al. (2020)	SCScan		1000	✓	✓			
Zaazaa and El Bakkali (2023)	Xue et al. (2021)	Clairvoyance, Slither, Oyente, Securify	url	11,714	✓	✓		✓	RW
Zaazaa and El Bakkali (2023)	Ferreira et al. (2021)	Smartbugs	url	47,398	✓	✓		✓	RW
Zaazaa and El Bakkali (2023)	Ferreira et al. (2021)	SmartBugs	url	143	✓	✓		✓	RW
Zaazaa and El Bakkali (2023)	Albert et al. (2019)	SAFEVM, VeryMax, SeaHorn, CPAchecker, Oyente, EthIR	url	24,294	✓	✓		✓	RW
Zaazaa and El Bakkali (2023)	Albert et al. (2019)	SAFEVM	url	29	✓	✓		✓	RW
Zaazaa and El Bakkali (2023)	Chen et al. (2019b)	OYENTE, MAIAN, Mythril, Evmdis, Miasm, Porosity		4,979,625	✓				
Zaazaa and El Bakkali (2023)	Prechtel et al. (2019)	Mythril-enhanced version		167,698					
Zaazaa and El Bakkali (2023)	Zhou et al. (2018)	SASC, oyente	url	2952			✓	✓	
Zaazaa and El Bakkali (2023)	Schneidewind et al. (2020)	eThor	url	720			✓	✓	
Zaazaa and El Bakkali (2023)	Zhou et al. (2020)	EverEvolvingG	url	344	✓	✓	✓	✓	RW+Syn
Di Angelo and Salzer (2024)	Di Angelo and Salzer (2024)	SmartConDetect	url	4859	✓	✓	✓	✓	RW
Di Angelo and Salzer (2024)	Jeon et al. (2023)		url	10,000	✓	✓			

Table 18
continued.

In	Papers	Tool/Project	Link	#Contracts	Results	Source	Byte	Address	RW/Syn
Wu et al. (2024b)	Chaliasos et al. (2023)	SmartFast	url	149	✓	✓	✓	✓	RW
	Zhaoxuan et al. (2022)	SmartFast	url	13,509	✓	✓		✓	RW
	Zhaoxuan et al. (2022)	MANDO-HGT, MANDO-GURU	url		✓	✓			RW + Syn
	Nguyen et al. (2023)	MANDO-HGT, MANDO-GURU	url		✓	✓		✓	
	Ye et al. (2022)	Vulpedia	url		✓	✓			
	Guo et al. (2024b)	MEVD	url		✓	✓			
	Duan et al. (2023b)	Duan et al. Framework	url					✓	RW
	Qian et al. (2023b)	RNVulDet	url			✓		✓	RW
	Soud et al. (2023)	SmartState	url				✓	✓	RW
	Choi et al. (2021)	Smartian	url			✓			RW
	HajiHosseiniKhani et al. (2024b)	SCsVulLyzr	url	18,796	✓	✓			
	Zhang et al. (2022d)	MODNN	url			✓			
	Dai et al. (2022)	SuperDetector	url			✓		✓	Syn
	Dai et al. (2022)	SuperDetector	url			✓		✓	Syn
	Hu et al. (2024)	SolidDetector	url			✓		✓	RW + Syn
	Nassirzadeh et al. (2023)	Gas Gauge	url		✓	✓		✓	
	Sigma Prime (2023)		url			✓			RW
	Qian et al. (2023c)		url			✓	✓		RW
	Qian et al. (2023c)		url			✓			RW
	Sun et al. (2022a)	ConFuzzius-BI	url	43		✓			RW
	Sun et al. (2022a)	ConFuzzius-BI	url	19		✓		✓	Syn
	Wei et al. (2023)		url	110	✓	✓			RW + Syn
	Di Angelo et al. (2023)		url	248,328	✓	✓	✓	✓	RW
	Sun et al. (2024)	Top200	url						RW
	Sun et al. (2024)	Web3Bugs	url						RW
	Sun et al. (2024)	DefiHacks	url	101	✓	✓		✓	RW
	Wang et al. (2024c)	Rekt	url						RW
	Wang et al. (2024c)	Certik	url						RW
	Wang et al. (2024c)	Slowmist	url						RW
	Wang et al. (2024c)	BlockSec	url						RW
	Luo et al. (2024a)	SCVHunter	url	1200	✓	✓		✓	
	Li et al. (2024d)	Cobra	url			✓			
	Wu et al. (2024c)	AdvScanner	url		✓	✓			RW + Syn
	Wang et al. (2024a)	ContractTinker	url						
	Eshghie and Artho (2024)	XploGen	url		✓	✓	✓		RW
	Liu and Song (2024a,b)	FunRedis	url						RW
	Chen et al. (2024c)	SOCher	url			✓			RW
	Liang et al. (2024)	PonziGuard	url		✓	✓			RW + Syn
	Guo et al. (2024a)	RLRep	url		✓	✓			
	Liu et al. (2024b)	InvCon +	url		✓	✓			
	Ren et al. (2021b)		url	46,186		✓			RW + Syn
	Torres et al. (2021)	ConFuzzius	url	21,000	✓	✓			RW
	Torres et al. (2021)	ConFuzzius	url	128	✓	✓		✓	RW
	Zheng et al. (2023)	SLiSE	url	34	✓	✓			RW
	Zhang et al. (2023d)	Code4rena	url						RW
	Liao et al. (2024)	SmartAxe	url			✓	✓		RW
	HajiHosseiniKhani et al. (2024a)	BCCC-SCsVul-2024	url			✓			RW
	Qian et al. (2024)	MuFuzz	url			✓			
	Wang et al. (2024f)	ContractGNN	url						RW
	Chu et al. (2024)	DeepFusion	url						
	Li et al. (2024c)		url	788		✓			RW + Syn
	Zheng et al. (2024b)	DAppSCAN-source	url	21,457	✓	✓			RW
	Zheng et al. (2024b)	DAppSCAN-bytercode	url	21,457	✓	✓	✓		RW

Table 19
Trend of tools released from Jan 2021 to Feb 2025.

Year	2021	2022	2023	2024	2025
Number of Tools	44	36	48	66	2

5.2. Tool evolution compared to the 2016–2021 period

The number of tools proposed in the literature is increasing strongly. The demand for secure development has also grown with the increasing interest in DApps and security. Table 19 shows the trend of tools released from January 2021 to February 2025. We compared the tools released between 2016 and 2020 with those released between 2021 and early 2025. In particular, we compared how tools have evolved in terms of functionality, methods, and code transformation techniques.

Functionalities. In terms of the functionalities of the developed tools, we observed a significant increase in the use of vulnerability detection tools, rising from 79.5% to 92.8%. Resource analysis and gas usage are also up sharply, from 12.1% to 22.3%, signaling an increasing focus on issues such as sustainability and security. In contrast, there is a decrease in the search for more comprehensive and articulated solutions such as Full Automation and Bulk Analysis, which drop from 41.3% to 5.7%. The focus on exploit generation has also decreased, from 7.1% to 2.9%. These data suggest a growing emphasis on security and resource efficiency.

Code transformation techniques. The use of code transformation techniques has remained more or less stable. A slight increase was seen in the use of CFG, DFG, Execution Traces, and Transactions. Techniques such as disassembly and FSM are less widely used.

Methods. Methods such as mutation testing, fuzzing, taint analysis, and machine learning are increasingly used. Furthermore, methods such as formal reasoning and constraint solving, model checking, pattern matching, and abstract interpretation are omitted. Remain invariable with the use of code instrumentation. Currently, several experiments combine multiple methods. However, the future of research is shifting towards using LLMs capable of performing refactoring and code repair operations, thereby supporting the removal of vulnerabilities from smart contracts.

LLM Era. The methods implemented by the tools reflect the advanced technology of recent years, namely Artificial Intelligence and Large Language Models (LLMs). Several tools adopt LLMs for different types of tasks, all of which were released in 2024. GPTScan (Sun et al., 2024) is one of the first tools that combines GPT with static analysis to detect vulnerabilities in smart contract logic. AdvScan (Wu et al., 2024c) implements a novel method that uses LLM and static analysis to automatically generate adversarial smart contracts designed to exploit reentrancy vulnerabilities in victim contracts. ContractThink (Wang et al., 2024a) is an LLM-empowered tool for real-world vulnerability repair. SOChecker (Chen et al., 2024c) leverages a fine-tuned Llama2 model to complete the code, followed by the application of symbolic execution methods for vulnerability detection. LLMsSmartSec (Mothukuri et al., 2024) is a new approach that accurately identifies and fixes smart contract vulnerabilities. SolGPT (Zeng et al., 2023) is designed to address the central issue of detecting and mitigating the vulnerabilities inherent in smart contracts. FELL MVP (Luo et al., 2024b) integrates ensemble learning with LLMs to classify vulnerabilities in smart contracts. It comprises a fine-tuned LLM for each vulnerability. SolOSphere (Khanzadeh and Alalfi, 2024) offers functionality that consists of parsing and deparsing Solidity code, fetching smart contracts from GitHub, and creating a committed environment for gas analysis. It employs an LLM to generate gas-expensive smart contracts. SmartInv (Wang et al., 2024d) is an automated framework based on foundation models that infers smart contract invariants

and detects bugs at scale. It employs an LLM to generate invariants. Finally, Chen et al. (2023a) evaluated the effectiveness of ChatGPT in detecting vulnerabilities in smart contracts, focusing on its detection capabilities and limitations. The results showed that while ChatGPT could identify a wide range of vulnerabilities and worked efficiently, it struggled with precision and had a higher rate of detection failures. Its overall performance was inferior to the other 14 state-of-the-art tools in most vulnerability categories. The authors also analyzed the causes of false positives, categorizing them into four types: *Protected Mechanism Bias*, *Lack of understanding of actual meaning*, *Disturbed by comments*, and *Interference with unexecuted code*.

Machine un-auditable bugs. Zhang et al. (2023d) systematically investigated real-world smart contract vulnerabilities and studied how many can be exploited by malicious users and cannot be detected by existing analysis tools. They further categorized vulnerabilities into three groups. The vulnerabilities in the first group are hard to exploit, doubtful, or not directly related to the functionalities of a given project. The second group of vulnerabilities involves the use of simple and general oracles, which do not require an in-depth understanding of the code semantics. Examples include Reentrancy and Arithmetic Overflow. Such vulnerabilities can be detected by data flow tracing, static symbolic execution, and other static analysis tools. The third group of vulnerabilities requires high-level semantic oracles for detection and is closely related to business logic. Most of these vulnerabilities are not detectable by existing static analysis tools. This group comprises six main types of vulnerability: price manipulation, ID-related violations, erroneous state updates, atomicity violation, privilege escalation, and erroneous accounting. According to their findings, a large portion of exploitable bugs in the wild (i.e., 80%) are not machine-auditable. According to DeFiHackLabs (Web3 Security Community, 2025), price manipulation caused losses in DApps of more than 90 million. Wu et al. (2023) describe two types of widespread price manipulation attacks that attackers can use to manipulate the price inside a Decentralized Exchange. Attacks apply traditional financial manipulation methods, i.e. front-running (Frontrunning, 2023) and cross-market manipulation (Cross-market manipulation, 2023), to DeFi ecosystems. The work of Wang et al. (2024d) lists several vulnerabilities that require a deeper understanding of the contract, achieved by analyzing natural language from a semantic perspective, i.e., comments, signatures of significant functions, meaningful variable names, and documentation describing transaction scenarios. Price-oracle manipulation is among these vulnerabilities.

Tool compatibility and out-of-date benchmarks. The SmartBugs wild dataset (Durieux et al., 2020) contains contracts written in Solidity 0.4.x and a few in 0.5.x. It is the most widely used benchmark for evaluating tools. Tools evaluated on older versions of Solidity without explicitly stating their compatibility with Solidity versions may lead developers and auditors to misuse them. To our knowledge, tools evaluated on older Solidity versions do not necessarily fail on newer versions, but their effectiveness remains uncertain. Tool compatibility with the latest Solidity versions is a critical issue. However, most tools do not specify which Solidity versions they support, leading users to assume broad compatibility. For example, tools such as EVM-Shield (Zhang et al., 2024c), Dfier (Wang et al., 2024i), sGuard+ (Gao et al., 2024), LENT-SSE (Zheng et al., 2024a), SiSE (Wang et al., 2024g), ContractCheck (Wang et al., 2024e), and PrettySmart (Zhong et al., 2024), published in 2024, were evaluated using the SmartBugs dataset. Li (2021), Jain and Tripathi (2024), and the tool SPCon (Liu et al., 2022) also use SmartBugs. This dataset consists mainly of Solidity 0.4.x contracts, with only a few from Solidity 0.5.x. Meanwhile, Solidity's latest major release, 0.8.x, was introduced in December 2020. None of these tools explicitly states their compatibility with specific Solidity versions. According to Wu et al. (2024a), a comprehensive protection system must be established and continuously improved based on the actual version to obtain secure smart contracts. In the list of benchmarks we collected, we

Listing 1: Solidity $\geq 0.8.0$

```

1 pragma solidity 0.8.0;
2 contract ExampleDC {
3     function f() public pure {
4         uint x = 0;
5         x--; // Revert on underflow
6     }
7 }

```

Listing 2: Solidity $< 0.8.0$

```

1 pragma solidity >=0.4.0 <0.8.0;
2 import "@openzeppelin/SafeMath.sol";
3 contract ExampleSM {
4     using SafeMath for uint;
5     function f() public pure {
6         uint x = 0;
7         x = x.sub(1); //Revert on underflow
8     }
9 }

```

Fig. 6. Underflow check in different Solidity versions.

found two datasets of smart contracts written in Solidity version 0.8.x (Liao et al., 2024; Zheng et al., 2024b). Zheng et al. (2024b) analyzed 1199 open-source audit reports from 29 security teams. They identified 9154 weaknesses and developed two distinct datasets, i.e., DAPPSCAN-source and DAPPSCAN-bytecode. Their work highlighted that, even in the real-world dApps, the most widely used version of Solidity is 0.8.x, whereas in the literature, smart contracts 0.8.x are not widely used to evaluate tool performance or conduct other experiments. Furthermore, Li et al. (2024c) observed that benchmarks used in existing studies suffer from small sizes or limited vulnerability types. Industry auditing experts involved in their work confirm that the absence of these vulnerability types in the benchmarks is due to their rare occurrence in practical projects. This observation underscores the benchmark's alignment with the contemporary landscape of smart contract vulnerabilities. Finally, our mapping between tools and vulnerabilities revealed that 62 tools detect the integer overflow-underflow vulnerability. All these tools could produce false positives when executed on smart contracts written in Solidity 0.8.x, because the latest version of Solidity performs automatic overflow/underflow checks at runtime by default, so there is no need to use libraries like SafeMath or good programming practices to avoid overflow. Tools that flag the omission of SafeMath (or similar libraries) for arithmetic operations when detecting overflow vulnerabilities could produce false positives in contracts compiled with Solidity 0.8.x or later unless the tool is version-aware. Fig. 6 compares two implementations to handle underflow in different Solidity versions.

In addition to benchmarks, the literature often leverages platforms that report real-world incidents or host security audits to collect vulnerable smart contracts. For example, Wang et al. (2024c) evaluated 174 real-world adversarial contracts linked to 159 incidents that collectively resulted in approximately 1.36 billion in financial losses. These incidents were gathered from reputable sources such as Rekt (2025), DeFiHackLabs (Web3 Security Community, 2025), (SlowMist, 2025), (CertiK, 2025), and (BlockSec Team, 2025). Similarly, Sun et al. (2024) assessed GPTSCAN using different datasets of real-world smart contracts like the Top200 dataset,⁷ which includes contracts associated with the 200 largest market capitalizations, and the Web3Bugs⁸ dataset which comprises large-scale contract projects that underwent audits on the Code4rena platform (Code4rena, 2025).

A key aspect of benchmarks is how they are created. Most are sourced from Etherscan (Samreen and Alalfi, 2021; Nguyen et al., 2021; Contro et al., 2021), which allows for the consultation of smart contracts deployed on the blockchain. However, others are purpose-built and contain SCs manually injected with vulnerabilities (Sun et al., 2022a; Dai et al., 2022; Ren et al., 2021b). For instance, Sun et al. (2022a) manually injected 19 SCs with eight different vulnerabilities. This study emphasizes the need for consistent benchmarks free of false positives, nu-

merous in instances, and broad in vulnerability coverage, which can serve as standards for tool evaluation. Ren et al. (2021b) provide three datasets, one composed of manually injected vulnerabilities, one composed of manually validated contracts, and the last one unlabeled. Zheng et al. (2023) proposed a manually validated dataset. The work of HajiHosseiniKhani et al. (2024a) describes the limitations of datasets and proposes a unified and labeled benchmark that merges four different benchmarks together. The dataset also contains vulnerability-free smart contracts. Liu and Jiao (2024) highlighted that there are no official datasets or universally recognized reliable third-party datasets available at present. Their contribution would improve tool assessment and facilitate a fair, unbiased comparison of their performance.

5.3. Research opportunities

Faruk et al. (2024) presents a comprehensive overview of the current state of Web3, examining the challenges and opportunities associated with the development of dApps in this environment. It highlights the difficulties faced by blockchain developers in adopting effective software engineering practices and establishing suitable guidelines for blockchain-based software development. In addition, the paper offers open suggestions for each category of challenges, which can be leveraged to address the identified issues. These results contribute meaningful knowledge to the Web3 community, aiding in the development of better strategies for future challenges.

Vulnerability layers. Our taxonomy focuses mainly on the vulnerabilities affecting smart contracts, with a small contribution to the analysis of the vulnerabilities in the system. A blockchain comprises several layers in addition to the application and presentation layers to which the SCs belong. The Consensus Layer, Network Layer, Data Layer, and Infrastructure Layer each present additional vulnerabilities that need more attention. The shift to Proof of Stake (PoS) has solved many of Ethereum's structural vulnerabilities related to mining and centralization (Selfish mining, 51% Attack, Replay Attack), making the network more secure, sustainable, and resistant to attacks. However, PoS introduces new challenges in governance, staking, and validator protection that need to be addressed with suitable security measures and incentive structures.

Performance comparison. The tools proposed in the current state of the art offer significant benefits, offering researchers and practitioners several approaches and methods to adopt. However, understanding the optimal solution based on specific needs remains unclear due to the challenge of obtaining a comprehensive understanding of the tools' performance. Many tools rely on different benchmarks for performance evaluation, and the execution environments and conditions differ, making it challenging to conduct fair and consistent comparisons. Additional analysis is needed to examine the specific implementations, methods, and techniques to address each vulnerability.

⁷ <https://github.com/MetaTrustLabs/GPTScan-Top200>

⁸ <https://github.com/MetaTrustLabs/GPTScan-Web3Bugs>

Secure-by-design and sustainability. There is a notable gap in tools that support developers during the early stages of implementation. Few tools are designed to be integrated into integrated development environments (IDEs) or development pipelines. For example, SCStudio (Ren et al., 2021a) is a Visual Studio Code plug-in for the development of secure smart contracts. Given the immutable nature of blockchain, it is necessary to focus on developing secure smart contracts by design, reducing the need for time-consuming patch generation and code repair systems. This approach improves security and offers significant advantages from a sustainability perspective. The blockchain ecosystem is notorious for its high energy consumption, and replacing faulty smart contracts with vulnerability-free versions aggravates this issue. Deploying and discarding faulty contracts is an unsustainable practice that contributes to unnecessary energy waste. By prioritizing secure development practices, we can reduce environmental impact, improve the overall efficiency of blockchain operations, and enhance their security.

Tool coverage gaps. Our mapping of tools to vulnerabilities offers a clear view of how the literature is evolving. Unsurprisingly, most tools focus on well-known vulnerabilities such as Reentrancy, Timestamp Dependence, and Integer Over/Underflow. However, there is a noticeable lack of interest in addressing many other vulnerabilities. According to our findings, the top 10 identified vulnerabilities are addressed using an average of 49 tools each. In contrast, the next 69 vulnerabilities attract significantly less attention, with an average of only three tools per vulnerability. Furthermore, no tool addresses the remaining 58% of vulnerabilities. The top 10 vulnerabilities are not guaranteed to have the most significant impact and may deserve further scrutiny. This mapping reveals a substantial gap in interest for a wide range of vulnerabilities.

Unexplored solution. Further analysis is needed to examine the specific implementations, methods, and techniques to address each vulnerability. This would also help identify unexplored solution combinations that could be further developed. Our mapping provides a foundation for this more in-depth analysis. Future research in this field could explore new solutions and combinations that have not yet been implemented or tested. Hejazi and Lashkari (2025) conducted a systematic evaluation of several detection tools using real-world datasets. The results show that while many tools perform well, they do not accurately cover all vulnerability types, highlighting the need for improved integration of detection methodologies. The findings aim to bridge the gaps in existing methods and guide future improvements to enhance the security of dApps.

Software engineering practices. The detection of vulnerabilities in Solidity smart contracts is a rapidly growing trend; yet, despite numerous proposed solutions, no widely accepted standard has been established. Research to date has been rather narrow in scope, and it remains unclear whether existing techniques are sufficient to ensure comprehensive coverage. Being an emerging area, there is a notable lack of well-established concepts coming from software engineering and security. In this context, the notion of code smells, particularly security smells, deserves further attention, even within decentralized solutions. Initially introduced by Di Sorbo et al. (2022), Gas-wasting code smells opened new perspectives for analyzing contract code and its gas consumption. Gas-wasting code smells were related to the Design patterns for gas optimization proposed by Marchesi et al. (2020). A recent study by Jiang et al. (2025) leveraged this concept to connect conventional coding practices with the less intuitive gas computation mechanism, resulting in a new detector that uses code smells as indicators of potential issues. A code smell is a surface indication in the source code that suggests something may be wrong with its design, structure, or implementation. In the context of smart contracts and security, code smells can serve as precursors to potential vulnerabilities. While a code smell alone may not directly lead to exploitation, it often violates best practices and increases the likelihood of security issues. Beyond code smells, the identification and

analysis of harmful yet recurring patterns (or anti-patterns) could provide researchers with new conceptual tools for developing innovative solutions. To date, the most widely recognized pattern in smart contract development is the Check-Effect-Interaction pattern, which is used to implement secure cryptocurrency transfers.

Consolidated ground truth. We verified the ground truth of the benchmarks whenever possible. Given that some benchmark sizes were expressed using the number of chain blocks or files, we further investigated and reported their size in terms of the number of SCs they contain.

New methods and approaches for creating reliable benchmarks have been proposed in the literature. Some of these involve injecting vulnerabilities into smart contracts in a controlled manner using pattern-based or mutation-based techniques. SolidiFi (Ghaleb and Pattabiraman, 2020), MuSe (Iuliano et al., 2025a), and SCAnoGenerator (Zhang et al., 2024) are three tools specifically designed to evaluate the effectiveness of static analysis tools for Solidity smart contracts by systematically injecting known vulnerabilities into the contract code. SolidiFi introduces seven predefined vulnerability patterns in source code, while MuSe uses pattern-based mutation operators to inject six types of vulnerability into real-world smart contracts. SCAnoGenerator facilitates automatic anomaly injection by analyzing the control and data flow of Ethereum smart contracts. Given the limited diversity and size of existing benchmarks, automated solutions for benchmark generation are essential to support a comprehensive and scalable tool evaluation.

The authors of mutation-based tools for vulnerability injection, such as SolidiFi and MuSe, adopt a pattern-matching approach in which vulnerability patterns are derived from real-world cases. Although these patterns capture realistic conditions, they do not guarantee comprehensive coverage of all possible corner cases that may lead to the same vulnerability. Moreover, we found no empirical evaluations comparing the detection performance of tools on synthetic versus real-world vulnerabilities. Researchers have widely adopted synthetic benchmarks, yet it remains unclear whether performance on these benchmarks accurately reflects real-world detection capabilities. In our previous work (Iuliano et al., 2025a), we observed that Slither failed to detect vulnerabilities injected using the pattern-based mutations of MuSe. This result highlights both the need for improvements in static analyzers and the potential of MuSe-generated benchmarks to enhance the evaluation and development of detection tools. Given these findings, it is reasonable to assume that benchmarks generated with MuSe could help reveal the limitations of existing tools and thereby guide the refinement of vulnerability detectors. Nonetheless, an empirical comparison between synthetic and real-world benchmarks is required to validate this hypothesis and to better understand the representativeness of synthetic test cases in practical contexts. Finally, the use cases of smart contracts should also be considered. The most common real-world use cases include token transfers, cross-chain bridges, bidding, voting, lotteries, healthcare applications, investment platforms, price oracles, and others (Gupta, 2025). Synthetic benchmarks should reflect these predominant use cases to more accurately represent practical scenarios and ensure that detection tools are evaluated against realistic contract behaviors.

6. Threats to validity

This section discusses threats to validity, i.e., construct, internal, external, and conclusion validity (Zhou et al., 2016).

6.1. Construct validity

Search method. Missing relevant primary studies could significantly impact the quality of SLR. Therefore, we adopted different strategies to collect as many studies as possible. We utilized the five most widely used digital libraries. We applied backward and forward snowballing to the results from the Digital Libraries and forward snowballing to Rameder et al. (2022). Additionally, we analyzed the five main conferences in the

field under examination to conduct the research as comprehensively as possible.

Inappropriate or incomplete terms in search strings. The research strings and terms used may affect the construct validity. We used the same research strings from the updated SLR, which were well-discussed and well-motivated, and customized them for each digital library. To mitigate the risk that newly introduced terms in the search string would affect results, we conducted a sensitivity analysis in Step 2 of our research method. The results suggest that the difference is marginal ($\approx 5\%$), indicating that our original search query was already reasonably comprehensive.

Intrinsic data quality metrics. A potential construct validity threat arises from the way we calculate the IDQ of publication venues. Specifically, when multiple rankings (SCImago, CORE, Scopus CiteScore) are available, we select the ranking with the highest score as the IDQ. This approach may inflate the perceived quality of journals or conferences, potentially biasing the assessment. We adopted this method to ensure a consistent and fair upgrade of the original SLR conducted in 2021, allowing for a meaningful comparison over time, and to adhere to best practices for updating an SLR (Nepomuceno and Soares, 2019).

6.2. Internal validity

Publication bias. Positive results are more likely to be published than negative results. In addition, some studies tend to report particular success stories. We checked the suitability of each study using inclusion/exclusion criteria to avoid considering only papers reporting positive results.

Subjective quality assessment. The paper assessment was based on our judgment, which could have been biased and potentially expressed a threat. Inclusion and exclusion criteria helped us reduce the number of studies to be analyzed. We independently performed the quality assessment using a form to evaluate all studies, utilizing a form featuring multiple well-defined and detailed responses that left little room for subjectivity. We used two question forms: one for SLRs and Surveys and one for PS.

Vulnerability categorization ambiguity. Some vulnerabilities could reasonably be assigned to more than one category in our taxonomy. To ensure consistency, we classified each vulnerability based on its primary technical cause rather than its potential runtime consequences or attack vectors.

6.3. External validity

Restricted resources. The limited number of researchers who worked on the study could affect the external validity. In the planning phase, we established externally validated roles and tasks to be performed.

Inaccessible papers and databases. A relevant aspect concerns the inaccessibility of papers and databases, which was mainly addressed by utilizing numerous digital libraries, constructing customized research strings for each library, and applying exclusion criterion number four, which excludes papers not accessible online via our university's library network.

6.4. Conclusion validity

Study misclassification. Quality appraisal was based on the classification of the papers (i.e., SLRs, surveys, and primary studies) performed at the beginning of the research. Thus, some studies could have been mistakenly classified and later evaluated. However, some quality assessment questions, particularly those related to primary studies, do not apply to surveys or SLRs, contributing to the correct classification of the analyzed studies.

Bias in study selection. Misinterpretations of the title or abstract could affect the selection of the studies. During the data extraction process, we checked the citation whenever it contained relevant information for our study. When an original study cited in the papers was not included among the studies selected for our research, but was listed in the extracted papers, it was recovered due to Question 1 of the CDQ form for primary studies. This practice allowed us to recover studies that had initially been discarded.

Benchmarks labeling and validation process. The labeling process could have been performed manually or automatically. Some benchmarks are manually labeled (Guo et al., 2024b; Pan et al., 2021; Soud et al., 2023; Dai et al., 2022; Jeon et al., 2023; Wei et al., 2023), whereas automated labeling is quite challenging. Not all vulnerability patterns are used; some are not well-researched, and it is hard to ensure sufficient patterns, leading to benchmarks featuring false positives that directly impact the performance of the tools. Finally, the validation processes of the benchmarks are usually not thoroughly described and documented.

7. Related work

In the recent past, some literature reviews targeted SC vulnerabilities and security. Zaazaa and El Bakkali (2023) conducted a comprehensive examination of scholarly literature from 2016 to 2021, focusing on papers addressing the detection of vulnerabilities in smart contracts. Their objective is to systematically extract and compile all vulnerabilities documented in the selected articles and to analyze the various techniques researchers use to identify these vulnerabilities, with particular emphasis on machine learning (ML) models. In addition, they aim to collect and consolidate a repository of ML datasets to build robust models capable of addressing the problem. The findings reveal insufficient coverage of the identified vulnerabilities and highlight the limited number (four) of blockchain platforms studied for smart contract security. The authors urge developers to exercise greater diligence in their development practices to improve blockchain network security and reduce user risks. Compared to the study, we delved into tool design, analyzing its functionality, methods, and code transformation techniques that it implements.

Oualid Zaazaa and Hanan El Bakkali performed a detailed examination and classification of vulnerabilities (Zaazaa and Bakkali, 2023). Compared to their work, we organized the vulnerabilities hierarchically and categorized them into 14 categories.

Kushwaha et al. conducted two different studies. On the one hand, they systematically reviewed the security vulnerabilities of the Ethereum smart contracts, including well-known attacks and their preventive methods (Kushwaha et al., 2022b). They discussed the vulnerabilities based on their root and subcauses and systematically presented well-known attacks. On the other hand, they systematically reviewed and classified Ethereum smart contract analysis tools (Kushwaha et al., 2022a). They analyzed 86 security analysis tools developed for Ethereum blockchain smart contracts, regardless of tool type or analysis approach, highlighted several challenges, and recommended future research. In our work, we overcome their limitations. We have considered tools that do not have names or have been proposed in pre-print articles and provided a collection of benchmarks for tool evaluation.

Vidal et al. (2024b) review recent smart contract vulnerability detection research, focusing on detection techniques, targeted vulnerabilities, and evaluation datasets. It maps vulnerabilities to the DASP and SWC classification schemes, identifying research trends and gaps. The study highlights that existing schemes, especially SWC, may not capture newer vulnerabilities, suggesting that updated classification frameworks are needed. The heterogeneity in the capabilities and evaluation settings of the detection tools hinders the comparison, pointing to the importance of standardized benchmarks to evaluate the effectiveness of the tools. Compared to this study, we have implemented a structured taxonomy

and analyzed more aspects of vulnerability detection tools, such as code transformation techniques and implemented functionalities.

Vidal et al. (2024a) proposed OpenSCV, a new open and hierarchical taxonomy for smart contract vulnerabilities. OpenSCV is designed to reflect current industry practices, accommodate future changes, and support contributions from the broader community. The taxonomy was developed through an extensive analysis of existing research on vulnerability classification, community-maintained classification schemes, and smart contract vulnerability detection tools. The authors demonstrate that OpenSCV captures the detection capabilities advertised by current tools and emphasizes its relevance to smart contract security research. An expert-based evaluation was performed to validate the taxonomy using a questionnaire involving researchers active in the field. Our taxonomy differs from the previous in several aspects. We focused on vulnerabilities in Solidity smart contracts on the Ethereum blockchain. OpenSCV also includes vulnerabilities related to other blockchains, such as Hyperledger Fabric and EOSIO, and other languages, such as Golang. Our taxonomy has a category for fixed vulnerabilities, like vulnerabilities related to the EVM and not to the Solidity language, while OpenSCV does not divide them. The EVM has undergone different upgrades to resolve some vulnerabilities. We proposed a new category related to library vulnerabilities. We included eight new library vulnerabilities presented by Huang et al. (2024a) and new CVEs (CVE-2023-36979 and CVE-2023-36980) presented by Li et al. (2024d). Our study systematically reviews a wide period with respect to Vidal et al. (2024a), including papers up to February 2025, and then new vulnerabilities.

Kaixuan (Li et al., 2024c) focused on the application of static tools for security testing. In particular, they developed a tool-oriented taxonomy aimed at unifying the taxonomies used by detection tools. They developed an up-to-date and fine-grained vulnerability taxonomy for smart contracts, including 45 unique types. Their taxonomy is based on taxonomies, such as the DASP Top 10 (2018) and the SWC Registry (2020), and includes the vulnerability identifiers used by security testing tools. Our taxonomy takes into account more preexisting taxonomies and is not tool-oriented. We include vulnerability identifiers that come from primary studies that introduced new tools, existing taxonomies proposed in the academic literature, and open-source taxonomies, like DASP Top 10. In our taxonomy, we also included the updated version of the OWASP Smart Contract Top 10⁹ (January 2025).

Hejazi and Lashkari (2025) collected 256 tools developed between 2018 and 2024 to analyze vulnerabilities in smart contracts, categorize them by methodology, such as fuzzing, symbolic execution, and others, and further classify them based on their detection capabilities. Their evaluation followed a two-layered approach. First, a theoretical analysis grouped tools by origin (academic or industry) and analysis approach (static, dynamic, or hybrid) using customized evaluation criteria. Second, an experimental assessment tested selected tools on real-world datasets, requiring significant time and computational resources. Their findings revealed that academic tools tend to prioritize documentation, flexibility, and adaptability, while industry tools emphasize speed and seamless integration into production environments. In contrast, the tools collected in our study focus exclusively on detecting vulnerabilities in Solidity smart contracts on the Ethereum blockchain. Our analysis goes beyond categorization by also collecting detailed information on the code transformation techniques, methodologies, and functionalities implemented by each tool. Using our taxonomy, we provide a comprehensive mapping between tools and the vulnerabilities that they are designed to detect. This mapping highlights existing gaps and reveals trends in how specific techniques and methods are applied to different types of vulnerability. When combined with the performance analysis performed by Hejazi and Lashkari (2025), our results offer a more complete understanding of the current landscape and could inspire future improvements in the smart contract security tooling.

Finally, Rameder et al. (2022) conducted the SLR we updated. The study assesses the state of the art in automated vulnerability analysis of Ethereum smart contracts, focusing on vulnerability classifications, detection methods, security analysis tools, and benchmarks for tool evaluation. Their initial search across major online libraries identified more than 1300 publications. After applying a structured protocol to remove duplicates and implement inclusion/exclusion criteria, they retained 303 publications. The authors assess the quality of these studies based on the reputation of the publication venues and predefined contextual criteria. For 165 publications with at least a medium quality score, they extract data on vulnerabilities, methods, and tools. They then synthesize these data, providing a classification of 54 smart contract vulnerabilities, an overview of 140 tools, and a list of benchmarks used for tool evaluation. Finally, they present a detailed discussion of their findings. Our study overcomes previous limitations, such as the absence of a structured snowballing process and a comprehensive mapping between tools and vulnerabilities. The mapping was not completely carried out. The reason was that each tool employs different methods and may classify contracts differently, even when they seemingly address the same vulnerability. Tools also refer to taxonomies that are incompatible with each other. However, our taxonomy serves as a point of aggregation of many state-of-the-art classifications and allows us to map tools and vulnerabilities.

Research gaps and our contributions. Our paper bridges previous research gaps and overcomes their limitations by expanding existing studies. We propose a hierarchical taxonomy of vulnerabilities distributed on three different levels of abstraction based on their detailed description. We categorized them into a comprehensive group of 14 categories that cover all aspects of vulnerabilities. Our taxonomy is the first to collect all the different categories present in the state of the art in a single point of aggregation. We resolved issues related to the ambiguity of synonyms when consulting existing taxonomies. We analyze tool design, including code transformation techniques and functionality. Additionally, we include unnamed and pre-print tools, establish a list of benchmarks, execute a structured snowballing process, and provide a comprehensive mapping between tools and vulnerabilities.

8. Conclusion

This paper presents a systematic review of the literature on smart contract vulnerabilities, detection tools, and benchmarks used for tool evaluation. The review covers various research aspects and aims to assess the current challenges related to smart contract security.

Our taxonomy merges the vulnerabilities identified in the literature through a rigorous SLR process. It serves as a comprehensive aggregation of the most state-of-the-art classifications. It addresses the existing confusion in the literature by organizing vulnerabilities into a clear hierarchical structure, reducing inconsistencies in nomenclature and making it easier to navigate. This structured categorization enables users to quickly capture the nature of vulnerabilities, facilitating a more immediate understanding of the potential issues that are being addressed. The curated list of tools allowed us to identify which solutions have been explored for vulnerability detection and which areas remain under-investigated. The mapping between these tools and our taxonomy highlights that current tools tend to focus on a narrow subset of vulnerabilities. Moreover, our analysis of existing benchmarks revealed significant issues, such as their limited size and scope, as well as their lack of updates in line with recent versions of Solidity.

Our SLR revealed many aspects to investigate, and as described in Section 5, there are many opportunities. Our research will focus on constructing new benchmarks by mutating smart contracts, we will explore the detection of vulnerabilities, in particular by conducting software component analysis, and we will set up an infrastructure that allows the tools to run within the same environment, under the same conditions, and using the same benchmarks, in order to compare their perfor-

⁹ <https://owasp.org/www-project-smart-contract-top-10/>

mance fairly and unbiased. Our work aims to provide the basis for future research, offering a valuable resource for researchers interested in advancing this field, which could greatly benefit from additional empirical studies and investigations.

CRedit authorship contribution statement

Gerardo Iuliano: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Conceptualization; **Dario Di Nucci:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Conceptualization.

Data availability

I have shared the link to the Replication Package in the step “Information about Available Data/Artifacts”.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Gerardo Iuliano reports financial support was provided by European Union, Italian Government (MUR), and AstraKode S.r.l. Dario Di Nucci reports financial support was provided by European Union and Italian Government (MUR). Dario Di Nucci reports a relationship with AstraKode S.r.l. that includes: funding grants. Dario Di Nucci is an associate editor for Elsevier Journal of Systems and Software. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

Funded by the European Union - Next Generation EU, Mission 4 Component 1, CUP D53D23008400006, and by the Italian Government (MUR, PRIN 2022) code 202233YFCJ. Gerardo Iuliano is funded by the European Union - Next Generation EU, Mission 4 Component 2, CUP D42B24002220004, by the Italian Government (MUR), and by AstraKode S.r.l. The authors thank Fabiano Izzo and the AstraKode team for the fruitful discussions that motivated our literature review and allowed us to elaborate on its implications.

References

Abubakar, M., Onyeshie, B.I., Wadhaj, I., Leimich, P., Ali, H., Buchanan, W.J., 2024. A systematic review of the blockchain technology security challenges and threats classification. In: 2024 6th International Conference on Blockchain Computing and Applications (BCCA). IEEE, pp. 684–697.

Agarwal, R., Thapliyal, T., Shukla, S.K., 2021. Vulnerability and transaction behavior based detection of malicious smart contracts. In: Cyberspace Safety and Security: 13th International Symposium, CSS 2021, Virtual Event, November 9–11, 2021, Proceedings. Springer-Verlag, Berlin, Heidelberg, p. 79–96. https://doi.org/10.1007/978-3-030-94029-4_6

Albert, E., Correias, J., Gordillo, P., Román-Díez, G., Rubio, A., 2019. SAFEVM: a safety verifier for ethereum smart contracts. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, New York, NY, USA, p. 386–389. <https://doi.org/10.1145/3293882.3338999>

Alharby, M., Aldweesh, A., Van Moorsel, A., 2018. Blockchain-based smart contracts: a systematic mapping study of academic research (2018). In: 2018 International Conference on Cloud Computing, Big Data and Blockchain (ICCB). IEEE, pp. 1–6.

Ali, A., Abideen, Z.U., Ullah, K., et al., 2021. Sescon: secure ethereum smart contracts by vulnerable patterns' detection. Secur. Commun. Netw. 2021, 1–14.

Antonopoulos, A.M., Wood, G., 2018. Mastering Ethereum: Building Smart Contracts and Dapps. O'reilly Media.

Ashizawa, N., Yanai, N., Cruz, J.P., Okamura, S., 2021. Eth2Vec: learning contract-wide code representations for vulnerability detection on ethereum smart contracts. In: Proceedings of the 3rd ACM International Symposium on Blockchain and Secure Critical Infrastructure. Association for Computing Machinery, New York, NY, USA, p. 47–59. <https://doi.org/10.1145/3457337.3457841>

Ashouri, M., 2020. Etherolic: a practical security analyzer for smart contracts. In: Proceedings of the 35th Annual ACM Symposium on Applied Computing. ACM, pp. 353–356.

Ashraf, I., Ma, X., Jiang, B., Chan, W.K., 2020. GasFuzzer: fuzzing ethereum smart contract binaries to expose gas-oriented exception security vulnerabilities. IEEE Access. <https://doi.org/10.1109/ACCESS.2020.2995183>

Azzopardi, S., Ellul, J., Pace, G.J., 2018. Monitoring smart contracts: contractlarva and open challenges beyond. In: Proceedings of the International Conference on Runtime Verification, pp. 113–137.

Barboni, M., Morichetta, A., Polini, A., 2022. SuMo: a mutation testing approach and tool for the ethereum blockchain. J. Syst. Software 193, 111445. <https://doi.org/10.1016/j.jss.2022.111445>

Barboni, M., Morichetta, A., Polini, A., Casoni, F., 2023. ReSuMo: a regression strategy and tool for mutation testing of solidity smart contracts. Software Qual. J. 32 (1), 225–253. <https://doi.org/10.1007/s11219-023-09637-1>

Bhargavan, K., Delignat-Lavaud, A., Fournet, C., et al., 2016. Formal verification of smart contracts. In: Proceedings of the 2016 ACM Workshop on Programming Languages and Analysis for Security, pp. 91–96.

BlockSec Team, 2025. Blocksec. <https://blocksecteam.medium.com/>. Accessed: 2025-04-12.

Bose, P., Das, D., Chen, Y., Feng, Y., Kruegel, C., Vigna, G., 2022. Sailfish: vetting smart contract state-inconsistency bugs in seconds. In: 2022 IEEE Symposium on Security and Privacy (SP). IEEE, pp. 161–178.

Brereton, P., Kitchenham, B.A., Budgen, D., Turner, M., Khalil, M., 2007. Lessons from applying the systematic literature review process within the software engineering domain. J. Syst. Software 80 (4), 571–583. Software Performance. <https://doi.org/10.1016/j.jss.2006.07.009>

Btc relay, 2016. Btc relay. <http://btcrelay.org/>

Buterin, V., 2016. Chain interoperability. R3 Res. Pap. 9, 1–25.

Buterin, V., et al., 2013. Ethereum white paper. GitHub Repository 1, 22–23.

Buterin, V., et al., 2014. A next-generation smart contract and decentralized application platform. white Pap. 3 (37), 2–1.

Cai, J., Li, B., Zhang, J., Sun, X., Chen, B., 2023. Combine sliced joint graph with graph neural networks for smart contract vulnerability detection. J. Syst. Software 195, 111550.

Cao, Y., Jiang, F., Xiao, J., Chen, S., Shao, X., Wu, C., 2023a. SCCheck: a novel graph-driven and attention-enabled smart contract vulnerability detection framework for web 3.0 ecosystem. IEEE Trans. Netw. Sci. Eng. 11 (5), 4007–4019.

Cao, Y., Jiang, F., Xiao, J., Chen, S., Shao, X., Wu, C., 2023b. SCCheck: a novel graph-driven and attention-enabled smart contract vulnerability detection framework for web 3.0 ecosystem. IEEE Trans. Netw. Sci. Eng. , 1–12. <https://doi.org/10.1109/TNSE.2023.3324942>

Cao, Y., Jiang, F., Xiao, J., Chen, S., Yang, W., Yi, Y., 2023c. Data flow-driven and attention mechanism-enabled smart contract vulnerability detection for secure and green blockchain-based service networks. In: ICC 2023 - IEEE International Conference on Communications, pp. 5135–5140. <https://doi.org/10.1109/ICC45041.2023.10279381>

Careem, M. A.A., Dutta, A., 2020. Reputation based routing in MANET using blockchain. In: 2020 International Conference on COMmunication Systems & NETworkS (COM-SNETS). IEEE, pp. 1–6.

CertiK, 2025. Certik. <https://www.certik.com/>. Accessed: 2025-04-12.

Chaliasos, S., Charalambous, M.A., Zhou, L., 2023. Smart contract and defi security tools: do they meet the needs of practitioners? <https://synthical.com/article/ed404388-fiad-11ed-9b54-72eb57fa10b3>. arXiv: 2304.02981.

Chaliasos, S., Charalambous, M.A., Zhou, L., Galanopoulou, R., Gervais, A., Mitropoulos, D., Livshits, B., 2024. Smart contract and defi security tools: do they meet the needs of practitioners? In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, pp. 1–13.

Chang, J., Gao, B., Xiao, H., Sun, J., Cai, Y., Yang, Z., 2019. sCompile: critical path identification and analysis for smart contracts. In: Proceedings of the International Conference on Formal Engineering Methods, pp. 286–304.

Chapman, P., Xu, D., Deng, L., Xiong, Y., 2019. Deviant: a mutation testing tool for solidity smart contracts. In: Proceedings of the 2019 IEEE International Conference on Blockchain (Blockchain), pp. 319–324.

Chatterjee, K., Goharshady, A.K., Goharshady, E.K., 2019. The treewidth of smart contracts. In: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, pp. 400–408.

Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., Zheng, Z., 2023a. When chatgpt meets smart contract vulnerability detection: how far are we? ACM Trans. Software Eng. Method. 34 (4), 1–30.

Chen, H., Lu, L., Massey, B., Wang, Y., Loo, B.T., 2024a. Verifying declarative smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–12.

Chen, H., Zhao, X., Wang, Y., Zhen, Z., 2024b. Safecheck: detecting smart contract vulnerabilities based on static program analysis methods. Secur. Privacy 7 (5), e393.

Chen, J., Chen, C., Hu, J., Grundy, J., Wang, Y., Chen, T., Zheng, Z., 2024c. Identifying smart contract security issues in code snippets from stack overflow. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1198–1210.

Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T., 2022a. DefectChecker: automated smart contract defect detection by analyzing EVM bytecode. IEEE Trans. Software Eng. 48 (7), 2189–2207. <https://doi.org/10.1109/TSE.2021.3054928>

Chen, J., Xia, X., Lo, D., Grundy, J., Luo, X., Chen, T., 2022b. Defining smart contract defects on ethereum. IEEE Trans. Software Eng. 48 (1), 327–345.

Chen, Q., Zhou, T., Liu, K., Li, L., Ge, C., Liu, Z., Klein, J., Bissyandé, T.F., 2023b. Tips: towards automating patch suggestion for vulnerable smart contracts. Autom. Software Eng. 30 (2), 31.

Chen, T., Li, X., Wang, Y., et al., 2017. An adaptive gas cost mechanism for ethereum to defend against under-priced dos attacks. In: Proceedings of the International Conference

- on Information Security Practice and Experience, pp. 3–24.
- Chen, T., Li, Z., Luo, X., Wang, X., Wang, T., He, Z., Fang, K., Zhang, Y., Zhu, H., Li, H., et al., 2021. SigRec: automatic recovery of function signatures in smart contracts. *IEEE Trans. Software Eng.* 48 (8), 3066–3086.
- Chen, T., Li, Z., Zhang, Y., Luo, X., Wang, T., Hu, T., Xiao, X., Wang, D., Huang, J., Zhang, X., 2019a. A large-scale empirical study on control flow identification of smart contracts. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 1–11. <https://doi.org/10.1109/ESEM.2019.8870156>
- Chen, T., Li, Z., Zhang, Y., Luo, X., Wang, T., Hu, T., Xiao, X., Wang, D., Huang, J., Zhang, X., 2019b. A large-scale empirical study on control flow identification of smart contracts. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 1–11. <https://doi.org/10.1109/ESEM.2019.8870156>
- Chen, Y., Sun, Z., Gong, Z., Hao, D., 2024d. Improving smart contract security with contrastive learning-based vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–11.
- Chen, Z., Liu, Y., Beillahi, S.M., Li, Y., Long, F., 2024e. Demystifying invariant effectiveness for securing smart contracts. *Proc. ACM Software Eng.* 1 (FSE), 1772–1795.
- Chen, Z., Liu, Y., Beillahi, S.M., Li, Y., Long, F., 2024f. OpenTracer: a dynamic transaction trace analyzer for smart contract invariant generation and beyond. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 2399–2402.
- Chepurnoy, A., Saxena, A., 2019. Multi-stage contracts in the UTXO model. In: Data Privacy Management, Cryptocurrencies and Blockchain Technology: ESORICS 2019 International Workshops, DPM 2019 and CBT 2019, Luxembourg, September 26–27, 2019, Proceedings 14. Springer, pp. 244–254.
- Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K., 2021. SMARTIAN: enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 227–239. <https://doi.org/10.1109/ASE51524.2021.9678888>
- Chu, H., Zhang, P., Dong, H., Xiao, Y., Ji, S., 2024. DeepFusion: smart contract vulnerability detection via deep learning and data fusion. *IEEE Trans. Reliab.* 74 (3), 3544–3558.
- Chu, H., Zhang, P., Dong, H., Xiao, Y., Ji, S., Li, W., 2023. A survey on smart contract vulnerabilities: data sources, detection and repair. *Inf. Softw. Technol.*, 107221.
- Code4rena, 2025. Code4rena. <https://code4rena.com/>. Accessed: 2025-04-12.
- Contro, F., Crosara, M., Ceccato, M., Preda, M.D., 2021. Ethersolve: computing an accurate control-flow graph from ethereum bytecode. In: 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC), pp. 127–137. <https://doi.org/10.1109/ICPC52881.2021.00021>
- Correas, J., Gordillo, P., Román Díez, G., 2021. Static profiling and optimization of ethereum smart contracts using resource analysis. *IEEE Access*. <https://doi.org/10.1109/ACCESS.2021.3057565>
- Cross-market manipulation, 2023. Cross-market manipulation. <https://en.wikipedia.org/wiki/Market-manipulation#Cross-market-manipulation>.
- Dai, C., Ding, H., Ma, W., Wu, Y., 2024. Smart contract vulnerability detection with self-ensemble pre-trained language models. In: 2024 International Conference on Computer, Information and Telecommunication Systems (CITS). IEEE, pp. 1–8.
- Dai, M., Yang, Z., Guo, J., 2022. SuperDetector: a framework for performance detection on vulnerabilities of smart contracts. *J. Phys. Conf. Ser.* 2289 (1), 12010.
- Dannen, C., 2017. Introducing Ethereum and Solidity. Vol. 1. Springer.
- De Salve, A., Brighente, A., Conti, M., 2024. Edit: a data inspection tool for smart contracts temporal behavior modeling and prediction. *Future Gener. Comput. Syst.* 154, 413–425.
- Deng, W., Wei, H., Huang, T., Cao, C., Peng, Y., Hu, X., 2023. Smart contract vulnerability detection based on deep learning and multimodal decision fusion. *Sensors* 23, 7246. <https://doi.org/10.3390/s23167246>
- Deng, X., Beillahi, S.M., Minwalla, C., Du, H., Veneris, A., Long, F., 2024. Safeguarding DeFi smart contracts against oracle deviations. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–12.
- Di Angelo, M., Durieux, T., Ferreira, J.F., Salzer, G., 2023. Smartbugs 2.0: an execution framework for weakness detection in ethereum smart contracts. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, pp. 2102–2105.
- Di Angelo, M., Salzer, G., 2024. Consolidation of ground truth sets for weakness detection in smart contracts. In: Essex, A., Matsuo, S., Kulyk, O., Gudgeon, L., Klages-Mundt, A., Perez, D., Werner, S., Bracciali, A., Goodell, G. (Eds.), *Financial Cryptography and Data Security. FC 2023 International Workshops*. Springer Nature Switzerland, Cham, pp. 439–455.
- Di Angelo, M., Durieux, T., Ferreira, J., Salzer, G., 2023. Evolution of automated weakness detection in ethereum bytecode: a comprehensive study. *Empir. Software Eng.* 29. <https://doi.org/10.1007/s10664-023-10414-8>.
- Di Sorbo, A., Laudanna, S., Vacca, A., Visaggio, C.A., Canfora, G., 2022. Profiling gas consumption in solidity smart contracts. *J. Syst. Software* 186, 111193. <https://doi.org/10.1016/j.jss.2021.111193>
- Dia, B., Ivaki, N., Laranjeiro, N., 2021. An empirical evaluation of the effectiveness of smart contract verification tools. In: 2021 IEEE 26th Pacific Rim International Symposium on Dependable Computing (PRDC), pp. 17–26. <https://doi.org/10.1109/PRDC53464.2021.00013>
- Ding, Y., Gervais, A., Wattenhofer, R., Sato, H., 2024. Hunting DeFi vulnerabilities via context-sensitive concolic verification. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp. 324–325.
- Dong, C., Huang, H., Shang, Y., 2024. Erinsy: efficient fuzzing by function invoke sequence generation for smart contracts. In: Proceedings of the 2024 8th International Conference on Big Data and Internet of Things, pp. 236–241.
- Dong, C., Li, Y., Tan, L., 2019. A new approach to prevent reentrant attack in solidity smart contracts. In: Proceedings of the CCF China Blockchain Conference, pp. 83–103.
- Du, S., Huang, H., 2020. A general framework of smart contract vulnerability mining based on control flow graph matching. In: Sun, X., Wang, J., Bertino, E. (Eds.), *Artificial Intelligence and Security*. Springer Singapore, Singapore, pp. 166–175.
- Duan, L., Sun, Y., Ni, W., Ding, W., Liu, J., Wang, W., 2023a. Attacks against cross-chain systems and defense approaches: a contemporary survey. *IEEE/CAA J. Autom. Sin.* 10 (8), 1647–1667.
- Duan, L., Yang, L., Liu, C., Ni, W., Wang, W., 2023b. A new smart contract anomaly detection method by fusing opcode and source code features for blockchain services. *IEEE Trans. Netw. Serv. Manage.* 20 (4), 4354–4368. <https://doi.org/10.1109/TNSM.2023.3278311>
- Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P., 2020. Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 530–541.
- El-Dosuky, M.A., Eladl, G.H., 2019. DOORchain: Deep ontology-based operation research to detect malicious smart contracts. In: Proceedings of the World Conference on Information Systems and Technologies, pp. 538–545.
- Eshghie, M., Artho, C., 2024. Oracle-guided vulnerability diversity and exploit synthesis of smart contracts using LLMs. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 2240–2248.
- Eshghie, M., Artho, C., Gurov, D., 2021. Dynamic vulnerability detection on smart contracts using machine learning. In: Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 305–312. <https://doi.org/10.1145/3463274.3463348>
- Eshghie, M., Artho, C., Stammer, H., Ahrendt, W., Hildebrandt, T., Schneider, G., 2024. HighGuard: cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 2378–2381.
- Fan, Y., Shang, S., Ding, X., 2021. Smart contract vulnerability detection based on dual attention graph convolutional network. In: Gao, H., Wang, X. (Eds.), *Collaborative Computing: Networking, Applications and Worksharing*. Springer International Publishing, Cham, pp. 335–351.
- Fang, Y., Wu, D., Yi, X., Wang, S., Chen, Y., Chen, M., Liu, Y., Jiang, L., 2023. Beyond “protected” and “private”: an empirical security analysis of custom function modifiers in smart contracts. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 1157–1168.
- Faruk, M. J.H., Raya, P., Siam, M.K., Cheng, J.Q., Shahriar, H., Cuzzocrea, A., Bringas, P.G., 2024. A systematic literature review of decentralized applications in web3: identifying challenges and opportunities for blockchain developers. In: 2024 IEEE International Conference on Big Data (BigData). IEEE, pp. 6240–6249.
- Fei, J., Chen, X., Zhao, X., 2023. MSmart: smart contract vulnerability analysis and improved strategies based on smartcheck. *Applied Sciences* 13 (3), 1733.
- Feist, J., Grieco, G., Groce, A., 2019a. Slither: a static analysis framework for smart contracts. In: 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB). IEEE, pp. 8–15.
- Feist, J., Grieco, G., Groce, A., 2019b. Slither: a static analysis framework for smart contracts. In: Proceedings of the 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), pp. 8–15.
- Feng, H., Ren, X., Wei, Q., Lei, Y., Kacker, R., Kuhn, D.R., Simos, D.E., 2023. MagicMirror: towards high-coverage fuzzing of smart contracts. In: 2023 IEEE Conference on Software Testing, Verification and Validation (ICST), pp. 141–152. <https://doi.org/10.1109/ICST57152.2023.00022>
- Ferreira, J.A.F., Cruz, P., Durieux, T., Abreu, R., 2021. Smartbugs: a framework to analyze solidity smart contracts. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 1349–1352. <https://doi.org/10.1145/3324884.3415298>
- Ferreira Torres, C., Baden, M., Norvill, R., Fiz Pontiveros, B.B., Jonker, H., Mauw, S., 2020. AEGIS: Shielding vulnerable smart contracts against attacks. In: Proceedings of the 15th ACM Asia Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 584–597. <https://doi.org/10.1145/3320269.3384756>
- Ferreira Torres, C., Iannillo, A.K., Gervais, A., State, R., 2021. The eye of horus: spotting and analyzing attacks on ethereum smart contracts. In: Borisov, N., Diaz, C. (Eds.), *Financial Cryptography and Data Security*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 33–52.
- Ferreira Torres, C., Jonker, H., State, R., 2022. Elysium: context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses. Association for Computing Machinery, New York, NY, USA, p. 115–128. <https://doi.org/10.1145/3545948.3545975>
- Frontrunning, 2023. Frontrunning <https://en.wikipedia.org/wiki/Front-running>.
- Fu, M., Wu, L., Hong, Z., Zhu, F., Sun, H., Feng, W., 2019a. A critical-path-coverage-based vulnerability detection method for smart contracts. *IEEE Access* 7, 147327–147344. <https://doi.org/10.1109/ACCESS.2019.2947146>
- Fu, M., Wu, L., Hong, Z., Zhu, F., Sun, H., Feng, W., 2019b. A critical-path-coverage-based vulnerability detection method for smart contracts. *IEEE Access* 7, 147327–147344.
- Fu, Y., Ren, M., Ma, F., Shi, H., Yang, X., Jiang, Y., Li, H., Shi, X., 2019. EVMFuzzer: detect EVM vulnerabilities via fuzz testing. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 1110–1114. <https://doi.org/10.1145/3338906.3341175>
- Gao, C., Yang, W., Ye, J., Xue, Y., Sun, J., 2024. sGuard + : machine learning guided rule-based automated vulnerability repair on smart contracts. *ACM Trans. Software Eng.*

- Method. 33 (5), 1–55.
- Gao, J., Liu, H., Liu, C., Li, Q., Guan, Z., Chen, Z., 2019. Easyflow: keep ethereum away from overflow. In: *Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pp. 23–26.
- Gao, Z., Jiang, L., Xia, X., Lo, D., Grundy, J., 2020. Checking smart contracts with structural code embedding. *IEEE Trans. Software Eng.* 47 (12), 2874–2891.
- Gao, Z., Jiang, L., Xia, X., Lo, D., Grundy, J., 2021. Checking smart contracts with structural code embedding. *IEEE Trans. Software Eng.* 47 (12), 2874–2891. <https://doi.org/10.1109/TSE.2020.2971482>
- Garner, P., Hopewell, S., Chandler, J., MacLehose, H., Schünemann, H., Akl, E., Beyene, J., Chang, S., Churchill, R., Dearness, K., Guyatt, G., Lefebvre, C., Liles, B., Marshall, R., Garcia, L., Mavergames, C., Nasser, M., Qaseem, A., Sampson, M., Wilson, E., 2016. When and how to update systematic reviews: consensus and checklist. *BMJ* 354, i3507. <https://doi.org/10.1136/bmj.i3507>
- Ghaleb, A., Pattabiraman, K., 2020. How effective are smart contract analysis tools? Evaluating smart contract static analysis tools using bug injection. In: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 415–427.
- Ghaleb, A., Rubin, J., Pattabiraman, K., 2022. eTaint: detecting gas-related vulnerabilities in smart contracts. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, p. 728–739. <https://doi.org/10.1145/3533767.3534378>
- Ghaleb, A., Rubin, J., Pattabiraman, K., 2023. AChecker: statically detecting smart contract access control vulnerabilities. In: *Proceedings of the 45th International Conference on Software Engineering*. IEEE Press, p. 945–956. <https://doi.org/10.1109/ICSE48619.2023.00087>
- Giesen, J.-R., Andreina, S., Rodler, M., Karame, G.O., Davi, L., 2022. Practical mitigation of smart contract bugs. *arXiv: 2203.00364*.
- Godbole, S., Krishna, P.R., 2024. Poster: verisol-MCE: verification-based condition coverage analysis of smart contracts using model checker engines. In: *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, pp. 434–437.
- Grech, N., Kong, M., Jurisevic, A., Brent, L., Scholz, B., Smaragdakis, Y., 2018. Madmax: surviving out-of-gas conditions in ethereum smart contracts. *Proc. ACM Program. Lang.* 2, 1–27.
- Groce, A., Grieco, G., 2021. echidna-parade: a tool for diverse multicore smart contract fuzzing. In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 658–661.
- Grossman, S., Abraham, I., Golan-Gueta, G., et al., 2017. Online detection of effectively callback free objects with applications to smart contracts. *Proc. ACM Program. Lang.* 2, 1–28.
- Guo, H., Chen, Y., Chen, X., Huang, Y., Zheng, Z., 2024a. Smart contract code repair recommendation based on reinforcement learning and multi-metric optimization. *ACM Trans. Software Eng. Method.* 33 (4), 1–31.
- Guo, J., Lu, L., Li, J., 2024b. Smart contract vulnerability detection based on multi-scale encoders. *Electronics* 13 (3), 489.
- Guo, L., Yang, W., Hu, L., Ma, L., Wu, Z., Lu, Z., 2024c. SERNET-T: a smart contract multi-label vulnerability detection model based on se attention and one-dimensional residual network. In: *2024 IEEE International Conference on Cognitive Computing and Complex Data (ICCD)*. IEEE, pp. 260–265.
- Gupta, S., 2025. 10 real world use cases for ethereum. <https://medium.com/blockchain-vidhya/11-real-world-use-cases-for-ethereum-352ad4509fa1>. Accessed: 18-Nov-2025.
- HajiHosseinKhani, S., Lashkari, A.H., Oskui, A.M., 2024a. Unveiling smart contracts vulnerabilities: toward profiling smart contracts vulnerabilities using enhanced genetic algorithm and generating benchmark dataset. *Blockchain: Res. Appl.*, 100253.
- HajiHosseinKhani, S., Lashkari, A.H., Oskui, A.M., 2024b. Unveiling vulnerable smart contracts: toward profiling vulnerable smart contracts using genetic algorithm and generating benchmark dataset. *Blockchain: Res. Appl.* 5 (1), 100171.
- Han, Q., Wang, L., Zhang, H., Shi, L., Wang, D., 2024. Ethchecker: a context-guided fuzzing for smart contracts. *J. Supercomput.* 80 (10), 13949–13975. <https://doi.org/10.1007/s11227-024-05954-9>
- Hao, X., Ren, W., Zheng, W., Zhu, T., 2020. SCSan: a SVM-based scanning system for vulnerabilities in blockchain smart contracts. In: *2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pp. 1598–1605. <https://doi.org/10.1109/TrustCom50675.2020.00221>
- He, D., Ding, K., Chan, S., Guizani, M., 2023a. Unknown threats detection methods of smart contracts. *IEEE Internet Things J.* 11 (3), 4430–4441.
- He, D., Ding, K., Chan, S., Guizani, M., 2024a. Unknown threats detection methods of smart contracts. *IEEE Internet Things J.* 11 (3), 4430–4441.
- He, D., Wu, R., Li, X., Chan, S., Guizani, M., 2023b. Detection of vulnerabilities of blockchain smart contracts. *IEEE Internet Things J.* 10 (14), 12178–12185. <https://doi.org/10.1109/JIOT.2023.3241544>
- He, H., Luo, Z., Wang, Q., Chen, M., He, H., Gao, L., Zhang, H., 2020. Joint operation mechanism of distributed photovoltaic power generation market and carbon market based on cross-chain trading technology. *IEEE Access* 8, 66116–66130.
- He, J., Balunovic, M., Ambroladze, N., Tsankov, P., Vechev, M., 2019. Learning to fuzz from symbolic execution with application to smart contracts. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 531–548.
- He, L., Zhao, X., Wang, Y., 2024b. Parse: efficient detection of smart contract vulnerabilities via parallel and simplified symbolic execution. In: *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, pp. 272–273.
- He, L., Zhao, X., Wang, Y., 2024c. ReenSAT: reentrancy vulnerability detection in smart contracts using semantic-enhanced SAT evaluation. *IEEE Trans. Reliab.* 74 (2), 2708–2722.
- He, L., Zhao, X., Wang, Y., Yang, J., Sun, X., 2023c. GraphSA: Smart Contract Vulnerability Detection Combining Graph Neural Networks and Static Analysis. Vol. 372 of *FAIA*. chapter GraphSA: Smart Contract Vulnerability Detection Combining Graph Neural Networks and Static Analysis. pp. 1020–1027. <https://doi.org/10.3233/FAIA230374>
- Hejazi, N., Lashkari, A.H., 2025. A comprehensive survey of smart contracts vulnerability detection tools: techniques and methodologies. *J. Netw. Comput. Appl.*, 104142.
- Hirai, Y., 2017. Defining the ethereum virtual machine for interactive theorem provers. In: *International Conference on Financial Cryptography and Data Security*, pp. 520–535.
- Honig, J.J., Everts, M.H., Huisman, M., 2019. Practical mutation testing for smart contracts. In: *Cristina, P.-S., Guillermo, N.-A., Alex, B., Joaquin, G.J. (Eds.), Data Privacy Management, Cryptocurrencies and Blockchain Technology*. Springer, pp. 289–303.
- Hu, T., Li, B., Pan, Z., Qian, C., 2024. Detect defects of solidity smart contract based on the knowledge graph. *IEEE Trans. Reliab.* 73 (1), 1–17.
- Hu, T., Li, J., Xu, X., Li, B., 2024. Solitester: detecting exploitable external-risky vulnerability in smart contracts using contract account triggering method. *J. Software: Evol. Process.*, e2633.
- Huang, J., Han, S., You, W., Shi, W., Liang, B., Wu, J., Wu, Y., 2021a. Hunting vulnerable smart contracts via graph embedding based bytecode matching. *IEEE Trans. Inf. Forensics Secur.* 16, 2144–2156.
- Huang, J., Han, S., You, W., Shi, W., Liang, B., Wu, J., Wu, Y., 2021b. Hunting vulnerable smart contracts via graph embedding based bytecode matching. *IEEE Trans. Inf. Forensics Secur.* <https://doi.org/10.1109/TIFS.2021.3050051>
- Huang, J., Zhou, K., Xiong, A., Li, D., 2022. Smart contract vulnerability detection model based on multi-task learning. *Sensors* 22 (5), 1829.
- Huang, M., Chen, J., Jiang, Z., Zheng, Z., 2024a. Revealing hidden threats: an empirical study of library misuse in smart contracts. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3597503.3623335>
- Huang, Y., Wu, X., Wang, Q., Qian, Z., Chen, X., Tang, M., Zheng, Z., 2024b. The sword of damocles: upgradeable smart contract in ethereum. In: *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, pp. 333–345.
- Hwang, S.-J., Choi, S.-H., Shin, J., Choi, Y.-H., 2022. CodeNet: code-targeted convolutional neural network architecture for smart contract vulnerability detection. *IEEE Access* 10, 32595–32607. <https://doi.org/10.1109/ACCESS.2022.3162065>
- Ibba, G., Aufiero, S., Bartolucci, S., Neykova, R., Ortu, M., Tonelli, R., Destefanis, G., 2024. MindtheDApp: a toolchain for complex network-driven structural analysis of ethereum-based decentralized applications. *IEEE Access* 12, 28382–28394.
- Iuliano, G., Allocca, L., Cicalese, M., Di Nucci, D., 2025a. Automated vulnerability injection in solidity smart contracts: a mutation-based approach for benchmark development. *arXiv:2504.15948*.
- Iuliano, G., Corradini, D., Pasqua, M., Ceccato, M., Nucci, D.D., 2025b. How do solidity versions affect vulnerability detection tools? An empirical study. *arXiv: 2504.05515*.
- Jain, V.K., Tripathi, M., 2024. An integrated deep learning model for ethereum smart contract vulnerability detection. *Int. J. Inf. Secur.* 23 (1), 557–575.
- Jeon, S., Lee, G., Kim, H., Woo, S., 2023. Design and evaluation of highly accurate smart contract code vulnerability detection framework. *Data Min. Knowl. Discov.* <https://doi.org/10.1007/s10618-023-00981-1>
- Ji, R., He, N., Wu, L., Wang, H., Bai, G., Guo, Y., 2020. DEPOSafe: demystifying the fake deposit vulnerability in ethereum smart contracts. In: *2020 25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, pp. 125–134. <https://doi.org/10.1109/ICECCS51672.2020.00022>
- Ji, S., Dong, J., Qiu, J., Gu, B., Wang, Y., Wang, T., 2021a. Increasing fuzz testing coverage for smart contracts with dynamic taint analysis. In: *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*. IEEE, pp. 243–247.
- Ji, S., Dong, J., Qiu, J., Gu, B., Wang, Y., Wang, T., 2021b. Increasing fuzz testing coverage for smart contracts with dynamic taint analysis. In: *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*, pp. 243–247. <https://doi.org/10.1109/QRS54544.2021.00035>
- Ji, S., Wu, J., Qiu, J., Dong, J., 2023. Effuzz: efficient fuzzing by directed search for smart contracts. *Inf. Software Technol.* 159, 107213.
- Jiang, B., Chen, Y., Wang, D., Ashraf, I., Chan, W.K., 2021. WANA: symbolic execution of wasm bytecode for extensible smart contract vulnerability detection. In: *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*, pp. 926–937. <https://doi.org/10.1109/QRS54544.2021.00102>
- Jiang, F., Chao, K., Xiao, J., Liu, Q., Gu, K., Wu, J., Cao, Y., 2023. Enhancing smart-contract security through machine learning: a survey of approaches and techniques. *Electronics* 12 (9), 2046.
- Jiang, J., Li, Z., Qin, H., Jiang, M., Luo, X., Wu, X., Wang, H., Tang, Y., Qian, C., Chen, T., 2025. Unearthing gas-wasting code smells in smart contracts with large language models. *IEEE Trans. Software Eng.* 51 (4), 879–903. <https://doi.org/10.1109/TSE.2024.3491578>
- Jiao, J., Lin, S.W., Sun, J., 2020. A generalized formal semantic framework for smart contracts. In: *FASE*, pp. 75–96.
- Jie, W., Chen, Q., Wang, J., Voundi Koe, A.S., Li, J., Huang, P., Wu, Y., Wang, Y., 2023. A novel extended multimodal AI framework towards vulnerability detection in smart contracts. *Inf. Sci.* 636, 118907. <https://doi.org/10.1016/j.ins.2023.03.132>
- Jie, W., Koe, A. S.V., Huang, P., Zhang, S., 2021. Full-stack hierarchical fusion of static features for smart contracts vulnerability detection. In: *2021 IEEE International Conference on Blockchain (Blockchain)*, pp. 95–102. <https://doi.org/10.1109/Blockchain53845.2021.00091>
- Jin, H., Wang, Z., Wen, M., Dai, W., Zhu, Y., Zou, D., 2022. Aroc: an automatic repair framework for on-chain smart contracts. *IEEE Trans. Software Eng.* 48 (11), 4611–4629. <https://doi.org/10.1109/TSE.2021.3123170>
- JJ, L., Singh, K., 2024. Enhancing oyente: four new vulnerability detections for improved smart contract security analysis. *Int. J. Inf. Technol.* 16 (6), 3389–3399.

- Kaleem, M., Mavridou, A., Laszka, A., 2020. Vyper: a security comparison with solidity based on common vulnerabilities. In: 2020 2nd Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS), pp. 107–111. <https://doi.org/10.1109/BRAINS49436.2020.9223278>
- Kaur, G., Gandhi, C., 2020. Scalability in blockchain: challenges and solutions. In: Handbook of Research on Blockchain Technology. Elsevier, pp. 373–406.
- Kaushal, R.K., Kumar, N., Panda, S.N., Kukreja, V., 2021. Immutable smart contracts on blockchain technology: its benefits and barriers. In: 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions)(ICRITO). IEEE, pp. 1–5.
- Khan, Z.A., Namin, A.S., 2024. Involuntary transfer: a vulnerability pattern in smart contracts. IEEE Access 12, 62459–62479.
- Khanzadeh, S., Alalfi, M.H., 2024. SolOSphere: a framework for gas optimization in solidity smart contracts. In: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C). IEEE, pp. 35–45.
- Kiani, R., Sheng, V.S., 2024. Ethereum smart contract vulnerability detection and machine learning-driven solutions: a systematic literature review. Electronics 13 (12), 2295.
- Kitchenham, B., Charters, S., 2007. Guidelines for performing systematic literature reviews in software engineering 2.
- Kushwaha, S.S., Joshi, S., Singh, D., Kaur, M., Lee, H.-N., 2022a. Ethereum smart contract analysis tools: a systematic review. IEEE Access 10, 57037–57062. <https://doi.org/10.1109/ACCESS.2022.3169902>
- Kushwaha, S.S., Joshi, S., Singh, D., Kaur, M., Lee, H.-N., 2022b. Systematic review of security vulnerabilities in ethereum blockchain smart contract. IEEE Access 10, 6605–6621. <https://doi.org/10.1109/ACCESS.2022.3140091>
- Lahbib, A., Wakrime, A.A., Laouiti, A., Toumi, K., Martin, S., 2020. An event-b based approach for formal modelling and verification of smart contracts. In: Proceedings of the International Conference on Advanced Information Networking and Applications, pp. 1303–1318.
- Lai, E., Luo, W., 2020. Static analysis of integer overflow of smart contracts in ethereum. In: Proceedings of the 2020 4th International Conference on Cryptography, Security and Privacy, pp. 110–115.
- Lakadawala, H., Dzibede, K., Chen, Y., 2024. Detecting reentrancy vulnerability in smart contracts using graph convolution networks. In: 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC), pp. 188–193. <https://doi.org/10.1109/CCNC51664.2024.10454763>
- Lee, S., Cho, E.-S., 2021. Lightweight extension of an execution environment for safer function calls in solidity/ethereum virtual machine smart contracts. In: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 689–695. <https://doi.org/10.1109/SANER50967.2021.00087>
- Lee, S.-S., Murashkin, A., Derka, M., Gorzny, J., 2023. SoK: not quite water under the bridge: review of cross-chain bridge hacks. In: 2023 IEEE International Conference on Blockchain and Cryptocurrency (ICBC). IEEE, pp. 1–14.
- Li, B., Pan, Z., Hu, T., 2024a. EvoFuzzer: an evolutionary fuzzer for detecting reentrancy vulnerability in smart contracts. IEEE Trans. Netw. Sci. Eng. 11 (6), 5790 – 5802
- Li, C., 2021. Gas estimation and optimization for smart contracts on ethereum. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, pp. 1082–1086.
- Li, J., Li, R., Lu, Y., Cai, B., Cao, Y., Lin, H., Li, X., Li, C., 2024b. FEMD: feature enhancement-aided multimodal feature fusion approach for smart contract vulnerability detection. In: 2024 IEEE 30th International Conference on Parallel and Distributed Systems (ICPADS). IEEE, pp. 641–648.
- Li, K., Xue, Y., Chen, S., Liu, H., Sun, K., Hu, M., Wang, H., Liu, Y., Chen, Y., 2024c. Static application security testing (SAST) tools for smart contracts: how far are we? Proc. ACM Software Eng. 1 (FSE), 1447–1470.
- Li, W., Li, X., Li, Z., Zhang, Y., 2024d. Cobra: interaction-aware bytecode-level vulnerability detector for smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 1358–1369.
- Li, Z., Guo, W., Xu, Q., Xu, Y., Wang, H., Xian, M., 2021. Research on blockchain smart contracts vulnerability and a code audit tool based on matching rules. In: Proceedings of the 2020 International Conference on Cyberspace Innovation of Advanced Technologies. Association for Computing Machinery, New York, NY, USA, p. 484–489. <https://doi.org/10.1145/3444370.3444617>
- Li, Z., Lu, S., Zhang, R., Zhao, Z., Liang, R., Xue, R., Li, W., Zhang, F., Gao, S., 2023. VulHunter: hunting vulnerable smart contracts at EVM bytecode-level via multiple instance learning. IEEE Trans. Software Eng. 49 (11), 4886–4916. <https://doi.org/10.1109/TSE.2023.3317209>
- Li, Z., Wu, H., Xu, J., Wang, X., Zhang, L., Chen, Z., 2019. MuSC: a tool for mutation testing of ethereum smart contract. In: Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1198–1201.
- Liang, J., Zhai, Y., 2023. SCGRU: a model for ethereum smart contract vulnerability detection combining CNN and bigru-attention. In: 2023 8th International Conference on Signal and Image Processing (ICSIP), pp. 831–837. <https://doi.org/10.1109/ICSIP57908.2023.10270857>
- Liang, R., Chen, J., Wu, C., He, K., Wu, Y., Sun, W., Du, R., Zhao, Q., Liu, Y., 2024. Towards effective detection of ponzi schemes on ethereum with contract runtime behavior graph. ACM Trans. Software Eng. Method. 34 (4), 1–32
- Liao, J.-W., Tsai, T.-T., He, C.-K., Tien, C.-W., 2019. SoliAudit: smart contract vulnerability assessment based on machine learning and fuzz testing. In: 2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS), pp. 458–465. <https://doi.org/10.1109/IOTSMS48152.2019.8939256>
- Liao, Z., Hao, S., Nan, Y., Zheng, Z., 2023. SmartState: detecting state-reverting vulnerabilities in smart contracts via fine-grained state-dependency analysis. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, New York, NY, USA, p. 980–991. <https://doi.org/10.1145/3597926.3598111>
- Liao, Z., Nan, Y., Liang, H., Hao, S., Zhai, J., Wu, J., Zheng, Z., 2024. SmartAxe: detecting cross-chain vulnerabilities in bridge smart contracts via fine-grained static analysis. Proc. ACM Software Eng. 1 (FSE), 249–270.
- Liao, Z., Zheng, Z., Chen, X., Nan, Y., 2022. SmartDagger: a bytecode-based static analysis approach for detecting cross-contract vulnerability. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, New York, NY, USA, p. 752–764. <https://doi.org/10.1145/3533767.3534222>
- Lino, S., Ray, S., Stakhanova, N., 2021. EtherProv: provenance-aware detection, analysis, and mitigation of ethereum smart contract security issues. In: 2021 IEEE International Conference on Blockchain (Blockchain), pp. 1–10. <https://doi.org/10.1109/Blockchain53845.2021.00014>
- Liu, H., Luo, X., Liu, H., Xia, X., 2021a. Merkle tree: a fundamental component of blockchains. In: 2021 International Conference on Electronic Information Engineering and Computer Science (EIECS), pp. 556–561. <https://doi.org/10.1109/EIECS53707.2021.9588047>
- Liu, Y., Li, Y., 2022. InvCon: a dynamic invariant detector for ethereum smart contracts. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, pp. 1–4.
- Liu, Y., Li, Y., Lin, S.-W., Artho, C., 2022. Finding permission bugs in smart contracts with role mining. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 716–727.
- Liu, Y., Song, W., 2024a. FunRedis: a function redispatch tool to reduce invocation gas fees in solidity smart contracts. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 516–527.
- Liu, Y., Song, W., 2024b. FunRedis: reordering function dispatch in smart contract to reduce invocation gas fees. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 516–527.
- Liu, Y., Wang, C., Ma, Y., 2024a. DL4sc: a novel deep learning-based vulnerability detection framework for smart contracts. Autom. Software Eng. 31 (1), 24. <https://doi.org/10.1007/s10515-024-00418-z>
- Liu, Y., Zhang, C., et al., 2024b. Automated invariant generation for solidity smart contracts. arXiv:2401.00650
- Liu, Z., Jiao, J., 2024. FFGDetector: vulnerability detection in cross-contract feature flow graph using GCN. In: 2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA). IEEE, pp. 751–757.
- Liu, Z., Qian, P., Wang, X., Zhu, L., He, Q., Ji, S., 2021b. Smart contract vulnerability detection: from pure neural network to interpretable graph feature and expert pattern fusion. In: Zhou, Z.-H. (Ed.), Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21. International Joint Conferences on Artificial Intelligence Organization, pp. 2751–2759. Main Track. <https://doi.org/10.24963/ijcai.2021/379>
- Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., Wang, X., 2023a. Combining graph neural networks with expert knowledge for smart contract vulnerability detection. IEEE Trans. Knowl. Data Eng. 35 (2), 1296–1310. <https://doi.org/10.1109/TKDE.2021.3095196>
- Liu, Z., Qian, P., Yang, J., Liu, L., Xu, X., He, Q., Zhang, X., 2023b. Rethinking smart contract fuzzing: fuzzing with invocation ordering and important branch revisiting. IEEE Trans. Inf. Forensics Secur. 18, 1237–1251.
- Losi, P.-A., 2024. Pakala: a toolkit for analyzing ethereum smart contracts. Accessed: 2024-10-08. <https://www.palkeo.com/en/projects/ethereum/pakala.html>
- Luo, F., Luo, R., Chen, T., Qiao, A., He, Z., Song, S., Jiang, Y., Li, S., 2024a. SCVHUNTER: smart contract vulnerability detection based on heterogeneous graph attention network. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, pp. 1–13.
- Luo, Y., Xu, W., Andersson, K., Hossain, M.S., Xu, D., 2024b. FELL MVP: an ensemble llm framework for classifying smart contract vulnerabilities. In: 2024 IEEE International Conference on Blockchain (Blockchain). IEEE, pp. 89–96.
- Lutz, O., Chen, H., Fereidooni, H., Sendner, C., Dmitrienko, A., Sadeghi, A.R., Koushanfar, F., 2021. Escort: Ethereum smart contracts vulnerability detection using deep neural network and transfer learning. arXiv: 2103.12607.
- Lv, Z., Wu, D., Yang, W., Duan, L., 2022. Attack and protection schemes on fabric isomorphic crosschain systems. Int. J. Distrib. Sens. Netw. 18 (1), 15501477211059945.
- Ma, C., Liu, S., Xu, G., 2023. Hgat: smart contract vulnerability detection method based on hierarchical graph attention network. J. Cloud Comput. Adv. Syst. Appl. 12 (1), 93–13.
- Ma, F., Fu, Y., Ren, M., et al., 2019. EVM*: from offline detection to online reinforcement for ethereum virtual machine. In: Proceedings of the 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 554–558.
- Ma, R., Jian, Z., Chen, G., Ma, K., Chen, Y., 2020. Rejection: a AST-based reentrancy vulnerability detection method. In: Han, W., Zhu, L., Yan, F. (Eds.), Trusted Computing and Information Security - 13th Chinese Conference, CTCIS 2019, Revised Selected Papers. Springer, Germany, pp. 58–71. Publisher Copyright: © Springer Nature Singapore Pte Ltd 2020; 13th Chinese Conference on Trusted Computing and Information Security, CTCIS 2019; Conference date: 24-10-2019 Through 27-10-2019. https://doi.org/10.1007/978-981-15-3418-8_5
- Malavolta, G., Moreno-Sanchez, P., Schneidewind, C., Kate, A., Maffei, M., 2018. Anonymous multi-hop locks for blockchain scalability and interoperability. Cryptol. ePrint Arch.
- Marchesi, L., Marchesi, M., Destefanis, G., Barabino, G., Tigano, D., 2020. Design patterns for gas optimization in ethereum. In: 2020 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE). IEEE, pp. 9–15.
- Mazurek, L., 2021. EthVer: formal verification of randomized ethereum smart contracts. In: Financial Cryptography and Data Security. FC 2021 International Workshops.

- Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 364–380.
- Medeiros, H., Vilain, P., Mylopoulos, J., Jacobsen, H.A., 2019. SolUnit: a framework for reducing execution time of smart contract unit tests. In: *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*, pp. 264–273.
- Mendes, E., Wohlin, C., Felizardo, K., Kalinowski, M., 2020. When to update systematic literature reviews in software engineering. *Journal of Systems and Software*, 167, 110607.
- Mi, F., Wang, Z., Zhao, C., Guo, J., Ahmed, F., Khan, L., 2021. VSCL: automating vulnerability detection in smart contracts with deep learning. In: *2021 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–9. <https://doi.org/10.1109/ICBC51069.2021.9461050>
- Mishra, D.P., Senapati, S., Dey, T., Lenka, R.K., 2024. Smart contract: tools and challenges. In: *2024 1st International Conference on Cognitive, Green and Ubiquitous Computing (IC-CGU)*. IEEE, pp. 1–6.
- Momeni, P., Wang, Y., Samavi, R., 2019a. Machine learning model for smart contracts security analysis. In: *2019 17th International Conference on Privacy, Security and Trust (PST)*, pp. 1–6. <https://doi.org/10.1109/PST47121.2019.8949045>
- Momeni, P., Wang, Y., Samavi, R., 2019b. Machine learning model for smart contracts security analysis. In: *Proceedings of the 2019 17th International Conference on Privacy, Security and Trust (PST)*, pp. 1–6.
- Mossberg, M., Manzano, F., Hennenfent, E., et al., 2019. Manticore: a user-friendly symbolic execution framework for binaries and smart contracts. In: *Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1186–1189.
- Mothukuri, V., Parizi, R.M., Massa, J.L., 2024. LLMSmartSec: smart contract security auditing with llm and annotated control flow graph. In: *2024 IEEE International Conference on Blockchain (Blockchain)*. IEEE, pp. 434–441.
- Munir, S., Taha, W., 2023. Pre-deployment analysis of smart contracts – a survey. *arXiv: 2301.06079*.
- Nakamoto, S., 2008. Bitcoin: a peer-to-peer electronic cash system. *Bitcoin*, 4 (2), 15.
- Narayana, K.L., Sathiyamurthy, K., 2023. Automation and smart materials in detecting smart contracts vulnerabilities in blockchain using deep learning. *Mater. Today: Proc.* 81, 653–659.
- Nassirzadeh, B., Sun, H., Banescu, S., Ganesh, V., 2023. Gas gauge: a security analysis tool for smart contract out-of-gas vulnerabilities. In: *Pardalos, P., Kotsireas, I., Guo, Y., Knottenbelt, W. (Eds.), Mathematical Research for Blockchain Economy*. Springer International Publishing, Cham, pp. 143–167.
- Nepomuceno, V., Soares, S., 2019. On the need to update systematic literature reviews. *Inf. Software Technol.* 109, 40–42. <https://doi.org/10.1016/j.infsof.2019.01.005>
- Nguyen, H.H., Nguyen, N.-M., Doan, H.-P., Ahmadi, Z., Doan, T.-N., Jiang, L., 2022. Mando-guru: vulnerability detection for smart contract source code by heterogeneous graph embeddings. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Association for Computing Machinery, New York, NY, USA, p. 1736–1740. <https://doi.org/10.1145/3540250.3558927>
- Nguyen, H.H., Nguyen, N.-M., Xie, C., Ahmadi, Z., Kudendo, D., Doan, T.-N., Jiang, L., 2023. MANDO-HGT: heterogeneous graph transformers for smart contract vulnerability detection. In: *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, pp. 334–346.
- Nguyen, Q.B., Nguyen, A.Q., Nguyen, V.H., Nguyen-Le, T., Nguyen-An, K., 2019. Detect abnormal behaviours in ethereum smart contracts using attack vectors. In: *Proceedings of the International Conference on Future Data and Security Engineering*, pp. 485–505.
- Nguyen, T.D., Pham, L.H., Sun, J., 2021. sGuard: towards fixing vulnerable smart contracts automatically. In: *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 1215–1229. <https://doi.org/10.1109/SP40001.2021.00057>
- Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T., 2020. sFuzz: an efficient adaptive fuzzer for solidity smart contracts. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. Association for Computing Machinery, New York, NY, USA, p. 778–788. <https://doi.org/10.1145/3377811.3380334>
- Nikolic, I., Kolluri, A., Sergey, L., Saxena, P., Hobor, A., 2018. Finding the greedy, prodigal, and suicidal contracts at scale. pp. 653–663.
- Obradović, M., Vuletić, P., 2024. Blockchain programming languages vulnerabilities and error mitigation strategies. In: *2024 11th International Conference on Electrical, Electronic and Computing Engineering (IcETRAN)*. IEEE, pp. 1–6.
- Olsthoorn, M., Stallenberg, D., van Deursen, A., Panichella, A., 2022. Syntest-solidity: automated test case generation and fuzzing for smart contracts. In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. Association for Computing Machinery, New York, NY, USA, p. 202–206. <https://doi.org/10.1145/3510454.3516869>
- Otoni, R., Marescotti, M., Alt, L., Eugster, P., Hyvärinen, A., Sharygina, N., 2023. A solicitous approach to smart contract verification. *ACM Trans. Priv. Secur.* 26 (2). <https://doi.org/10.1145/3564699>
- Özdemir Sönmez, F., Knottenbelt, W.J., 2024. ContractArmor: attack surface generator for smart contracts. *Procedia Comput. Sci.* 231, 8–15. *14th International Conference on Emerging Ubiquitous Systems and Pervasive Networks / 13th International Conference on Current and Future Trends in Information and Communication Technologies in Healthcare (EUSPN/ICTH 2023)*. <https://doi.org/10.1016/j.procs.2023.12.151>
- Pan, Z., Hu, T., Qian, C., Li, B., 2021. ReDefender: a tool for detecting reentrancy vulnerabilities in smart contracts effectively. In: *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*, pp. 915–925. <https://doi.org/10.1109/QRS54544.2021.00101>
- Park, D., Zhang, Y., Saxena, M., Daian, P., Rosu, G., 2018. A formal verification tool for ethereum VM bytecode. In: *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 912–915.
- Pasqua, M., Benini, A., Contro, F., Crosara, M., Dalla Preda, M., Ceccato, M., 2023. Enhancing ethereum smart-contracts static analysis by computing a precise control-flow graph of ethereum bytecode. *J. Syst. Software* 200, 111653.
- Peng, C., Akca, S., Rajan, A., 2019. SIF: A framework for solidity contract instrumentation and analysis. In: *Proceedings of the 2019 26th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 466–473.
- Peng, G., Wenzhong, Y., Liejun, W., Fuyuan, W., KeZiErBieKe, H., Yuanyuan, L., 2023. GRATDet: smart contract vulnerability detector based on graph representation and transformer. *Comput. Mater. Continua* 76 (2), 1439–1462. <https://doi.org/10.32604/cmc.2023.038878>
- Piantadosi, V., Rosa, G., Placella, D., Scalabrino, S., Oliveto, R., 2022. Detecting functional and security-related issues in smart contracts: a systematic literature review. *Software: Pract. Exper.* 53. <https://doi.org/10.1002/spe.3156>
- Politou, E., Casino, F., Alepis, E., Patsakis, C., 2019. Blockchain mutability: challenges and proposed solutions. *IEEE Trans. Emerging Top. Comput.* 9 (4), 1972–1986.
- Praitheeshan, P., Pan, L., Zheng, X., Jolfaei, A., Doss, R., 2021. Solguard: preventing external call issues in smart contract-based multi-agent robotic systems. *Inf Sci (Nij)* 579, 150–166.
- Prechtel, D., Groß, T., Müller, T., 2019. Evaluating spread of ‘gasless send’ in ethereum smart contracts. In: *2019 10th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, pp. 1–6. <https://doi.org/10.1109/NTMS.2019.8763848>
- Qian, P., Cao, R., Liu, Z., Li, W., Li, M., Zhang, L., Xu, Y., Chen, J., He, Q., 2023a. Empirical review of smart contract and DeFi security: vulnerability detection and automated repair. *arXiv: 2309.02391*.
- Qian, P., He, J., Lu, L., Wu, S., Lu, Z., Wu, L., Zhou, Y., He, Q., 2023b. Demystifying random number in ethereum smart contract: taxonomy, vulnerability identification, and attack detection. *IEEE Trans. Software Eng.* 49 (7), 3793–3810. <https://doi.org/10.1109/TSE.2023.3271417>
- Qian, P., Liu, Z., He, Q., Zimmermann, R., Wang, X., 2020. Towards automated reentrancy detection for smart contracts based on sequential models. *IEEE Access* 8, 19685–19695. <https://doi.org/10.1109/ACCESS.2020.2969429>
- Qian, P., Liu, Z., Yin, Y., He, Q., 2023c. Cross-modality mutual learning for enhancing smart contract vulnerability detection on bytecode. In: *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, p. 2220–2229. <https://doi.org/10.1145/3543507.3583367>
- Qian, P., Wu, H., Du, Z., Vural, T., Rong, D., Cao, Z., Zhang, L., Wang, Y., Chen, J., He, Q., 2024. MuFuzz: sequence-aware mutation and seed mask guidance for blockchain smart contract fuzzing. In: *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, pp. 1972–1985.
- Ralph, P., 2019. Toward methodological guidelines for process theories and taxonomies in software engineering. *IEEE Trans. Software Eng.* 45 (7), 712–735. <https://doi.org/10.1109/TSE.2018.2796554>
- Rameder, H., 2021. Systematic review of ethereum smart contract security vulnerabilities, analysis methods and tools. PhD thesis.
- Rameder, H., Di Angelo, M., Salzer, G., 2022. Review of automated vulnerability analysis of smart contracts on ethereum. *Front. Blockchain* 5. <https://doi.org/10.3389/fbloc.2022.814977>
- Rekt, 2025. Rekt. <https://rekt.news/>. Accessed: 2025-04-12.
- Ren, M., Ma, F., Yin, Z., Li, H., Fu, Y., Chen, T., Jiang, Y., 2021a. SCStudio: a secure and efficient integrated development environment for smart contracts. In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, p. 666–669. <https://doi.org/10.1145/3460319.3469078>
- Ren, M., Yin, Z., Ma, F., Xu, Z., Jiang, Y., Sun, C., Li, H., Cai, Y., 2021b. Empirical evaluation of smart contract testing: what is the best choice? In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 566–579.
- Rezaei, H., Eshghie, M., Anderesson, K., Palmieri, F., 2025. SoK: root cause of \$1 billion loss in smart contract real-world attacks via a systematic literature review of vulnerabilities. *arXiv: 2507.20175*.
- Rizvi, S., Orr, R.J., Cox, A., Ashokkumar, P., Rizvi, M.R., 2020. Identifying the attack surface for iot network. *Internet Things* 9, 100162.
- Rodler, M., Li, W., Karame, G.O., Davi, L., 2020. EVMPatch: timely and automated patching of ethereum smart contracts. In: *USENIX Security Symposium*. <https://api.semanticscholar.org/CorpusID:222090393>.
- Rodler, M., Paaßen, D., Li, W., Bernhard, L., Holz, T., Karame, G., Davi, L., 2023. EFCF: high performance smart contract fuzzing for exploit generation. In: *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, pp. 449–471.
- Rosaie, S.M., Jule, D., 2022. Smart contracts security threats and solutions. *Int. J. Inf. Technol. Web Eng. (IJITWE)* 17 (1), 1–30.
- Saldana, J., 2021. The coding manual for qualitative researchers. *Qualitative research in organizations and management: an international journal*, 12 (2).
- Samreen, N.F., Alalfi, M.H., 2020. Reentrancy vulnerability identification in ethereum smart contracts. In: *2020 IEEE International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*. IEEE, pp. 22–29.
- Samreen, N.F., Alalfi, M.H., 2021. Smartscan: an approach to detect denial of service vulnerability in ethereum smart contracts. In: *2021 IEEE/ACM 4th International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, pp. 17–26.
- Schneidewind, C., Grishchenko, I., Scherer, M., Maffei, M., 2020. eThor: practical and provably sound static analysis of ethereum smart contracts. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, p. 621–640. <https://doi.org/10.1145/3372297.3417250>
- Schreppel, T., 2019. Collusion by blockchain and smart contracts. *Harv. JL & Tech.* 33, 117.

- Sergey, I., Nagaraj, V., Johannsen, J., Kumar, A., Trunov, A., Hao, K.C.G., 2019. Safer smart contract programming with scilla. *Proc. ACM Program. Lang.* 3, 1–30.
- Shishkin, E., 2019. Debugging smart contract's business logic using symbolic model checking. *Program. Comput. Software* 45(8), 590–599.
- Shou, C., Tan, S., Koushik, S., 2023. ItyFuzz: snapshot-based fuzzer for smart contract. In: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Computing Machinery, New York, NY, USA, p. 322–333. <https://doi.org/10.1145/3597926.3598059>
- Sigma Prime, 2023. Solidity security: comprehensive guide to smart contract security. <https://blog.sigmaprime.io/solidity-security.html>. Accessed: [Date of access].
- SlowMist, 2025. Slowmist. <https://hacked.slowmist.io/>. Accessed: 2025-04-12.
- So, S., Hong, S., Oh, H., 2021. SmarTest: effectively hunting vulnerable transaction sequences in smart contracts through language model-guided symbolic execution. In: *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, pp. 1361–1378. <https://www.usenix.org/conference/usenixsecurity21/presentation/so>.
- Song, J., He, H., Lv, Z., Su, C., Xu, G., Wang, W., 2019. An efficient vulnerability detection model for ethereum smart contracts. In: *International Conference on Network and System Security*, pp. 433–442.
- Song, K., Matulevicius, N., de Lima Filho, E.B., Cordeiro, L.C., 2022. Esmc-solidity: an SMT-based model checker for solidity smart contracts. In: *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pp. 65–69.
- Song, S., Yu, X., Ma, Y., Li, J., Yu, J., 2024. Multi-model smart contract vulnerability detection based on biGRU. In: Luo, B., Cheng, L., Wu, Z.-G., Li, H., Li, C. (Eds.), *Neural Information Processing*. Springer Nature Singapore, Singapore, pp. 3–14.
- Sonnino, A., Bano, S., Al-Bassam, M., Danezis, G., 2020. Replay attacks and defenses against cross-shard consensus in sharded distributed ledgers. In: *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, pp. 294–308.
- Soud, M., Qasse, I., Liebel, G., Hamdaqa, M., 2023. AutoMESOC: automatic framework for mining and classifying ethereum smart contract vulnerabilities and their fixes. In: *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pp. 410–417. <https://doi.org/10.1109/SEAA60479.2023.00068>
- Staderini, M., 2022. Towards the Assessment and the Improvement of Smart Contract Security. Ph.D. thesis University of Florence. Florence, Italy. PhD thesis. https://fiore.unifi.it/retrieve/e398c382-6050-179a-e053-3705fe0a4cff/Towards_Assessment_Improvement_Solidity_Smart_Contracts.pdf.
- Stegeman, L., 2018. Solitor: Runtime Verification of Smart Contracts on the Ethereum Network. Master's thesis. University of Twente.
- Stegeman, L., 2018. Solitor: runtime verification of smart contracts on the ethereum network. <http://essay.utwente.nl/76902/>.
- Strong, D., Lee, Y., Wang, R., 2002. Data quality in context. *Commun. ACM* 40. <https://doi.org/10.1145/253769.253804>
- Su, J., Dai, H.-N., Zhao, L., Zheng, Z., Luo, X., 2023. Effectively generating vulnerable transaction sequences in smart contracts with reinforcement learning-guided fuzzing. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3551349.3560429>
- Suhail, S., Hussain, R., Jurdak, R., Oravec, A., Salah, K., Hong, C.S., Matulevicius, R., 2022. Blockchain-based digital twins: research trends, issues, and future challenges. *ACM Comput. Surv. (CSUR)* 54 (11s), 1–34.
- Suiche, M., 2017. Porosity: a decompiler for blockchain-based smart contracts bytecode. *DEF Con* 25 (11), 1–29.
- Sujeetha, R., Akila, K., 2023. Automated mutation analysis for smart contract using ama tool with enhanced ga and machine learning approach. *J. Theor. Appl. Inf. Technol.* 101 (21), 6931–6942.
- Sun, J., Huang, S., Wang, X., Wang, M., Du, J., 2022a. A detection method for scarcity defect of blockchain digital asset based on invariant analysis. In: *2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS)*, pp. 73–84. <https://doi.org/10.1109/QRS57517.2022.00018>
- Sun, J., Huang, S., Zheng, C., Wang, T., Zong, C., Hui, Z., 2022b. Mutation testing for integer overflow in ethereum smart contracts. *Tsinghua Sci. Technol.* 27 (1), 27–40.
- Sun, W., Wang, L., Wang, P., Zhang, Y., 2020. Collaborative blockchain for space-air-ground integrated networks. *IEEE Wireless Commun.* 27 (6), 82–89.
- Sun, X., Tu, L., Zhang, J., Cai, J., Li, B., Wang, Y., 2023. ASSBERT: active and semi-supervised bert for smart contract vulnerability detection. *J. Inf. Secur. Appl.* 73, 103423.
- Sun, Y., Gu, L., 2021. Attention-based machine learning model for smart contract vulnerability detection. *J. Phys. Conf. Ser.* 1820 (1), 012004. <https://doi.org/10.1088/1742-6596/1820/1/012004>
- Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y., 2024. GPTScan: detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13.
- Sungjae Hwang, S.R., 2020. Gap between theory and practice : an empirical study of security patches in solidity. In: *Proceedings of the 42nd International Conference on Software Engineering*.
- Szabo, N., 1994. Smart contracts <https://l0twinterschool2006/szabo.best.vwh.net/smart.contracts.html>.
- Szabo, N., 1997. Formalizing and securing relationships on public networks. *First Monday*.
- Tang, Y., Li, Z., Bai, Y., 2021. Rethinking of reentrancy on the ethereum. In: *2021 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech)*, pp. 68–75. <https://doi.org/10.1109/DASC-PiCom-CBDCom-CyberSciTech52372.2021.00025>
- Tann, W.J., Han, X.J., Gupta, S.S., Ong, Y., 2018. Towards safer smart contracts: A sequence learning approach to detecting vulnerabilities. *arXiv:1811.06632*
- Tian, H., Xue, K., Luo, X., Li, S., Xu, J., Liu, J., Zhao, J., Wei, D. S.L., 2021. Enabling cross-chain transactions: a decentralized cryptocurrency exchange protocol. *IEEE Trans. Inf. Forensics Secur.* 16, 3928–3941.
- Tian, Z., 2019a. Smart contract defect detection based on parallel symbolic execution. In: *2019 3rd International Conference on Circuits, System and Simulation (ICCSS)*, pp. 127–132. <https://doi.org/10.1109/ICIRSYSSIM.2019.8935603>
- Tian, Z., 2019b. Smart contract defect detection based on parallel symbolic execution. In: *Proceedings of the 2019 3rd International Conference on Circuits, System and Simulation (ICCSS)*, pp. 127–132.
- Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., Alexandrov, Y., 2018. Smartcheck: static analysis of ethereum smart contracts. In: *Proceedings of the 1st International Workshop on Emerging Trends in Software Engineering for Blockchain*. Association for Computing Machinery, New York, NY, USA, p. 9–16. <https://doi.org/10.1145/3194113.3194115>
- Tolmach, P., Li, Y., Lin, S.-W., 2023. Property-based automated repair of DeFi protocols. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3551349.3559560>
- Torres, C.F., Iannillo, A.K., Gervais, A., State, R., 2021. ConFuzzius: a data dependency-aware hybrid fuzzer for smart contracts. In: *2021 IEEE European Symposium on Security and Privacy*, pp. 103–119. <https://doi.org/10.1109/EuroSP51992.2021.00018>
- Torres, C.F., Schütte, J., State, R., 2018. Osiris: hunting for integer bugs in ethereum smart contracts. In: *Proceedings of the 34th Annual Computer Security Applications Conference*, pp. 664–676.
- Tsankov, P., Dan, A., Drachler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M., 2018. Securify: practical security analysis of smart contracts. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 67–82.
- Vacca, A., Di Sorbo, A., Visaggio, C.A., Canfora, G., 2021. A systematic literature review of blockchain and smart contract development: techniques, tools, and open challenges. *J. Syst. Software* 174, 110891.
- Vidal, F.R., Ivaki, N., Laranjeiro, N., 2024a. Openscv: an open hierarchical taxonomy for smart contract vulnerabilities. *Empir. Software Eng.* 29 (4), 101.
- Vidal, F.R., Ivaki, N., Laranjeiro, N., 2024b. Vulnerability detection techniques for smart contracts: a systematic literature review. *J. Syst. Software* 217, 112160. <https://doi.org/10.1016/j.jss.2024.112160>
- Vivar, A.L., Orozco, A. L.S., Villalba, L. J.G., 2021. A security framework for ethereum smart contracts. *Comput. Commun.* 172, 119–129.
- Wang, A., Wang, H., Jiang, B., Chan, W.K., 2020a. Artemis: an improved smart contract verification tool for vulnerability detection. In: *2020 7th International Conference on Dependable Systems and Their Applications (DSA)*, pp. 173–181. <https://doi.org/10.1109/DSA51864.2020.00031>
- Wang, B., Chu, H., Zhang, P., Dong, H., 2021a. Smart contract vulnerability detection using code representation fusion. In: *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*, pp. 564–565. <https://doi.org/10.1109/APSEC53868.2021.00069>
- Wang, B., Chu, H., Zhang, P., Dong, H., 2021b. Smart contract vulnerability detection using code representation fusion. *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*, 564–565 <https://api.semanticscholar.org/CorpusID:246945372>
- Wang, C., Zhang, J., Gao, J., Xia, L., Guan, Z., Chen, Z., 2024a. Contracttinker: LLM-empowered vulnerability repair for real-world smart contracts. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 2350–2353.
- Wang, D., Chen, J., Cai, S., Feng, Q., Chen, Y., Hu, X., 2024b. SCVD-SA: a smart contract vulnerability detection method based on hybrid deep learning model and self-attention mechanism. In: *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering-Companion (SANER-C)*. IEEE, pp. 207–214.
- Wang, H., Hu, Y., Wu, H., Liu, D., Peng, C., Wu, Y., Fan, M., Liu, T., 2024c. Skyeye: detecting imminent attacks via analyzing adversarial smart contracts. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 1570–1582.
- Wang, H., Wang, Z., Liu, S., Sun, J., Zhao, Y., Wan, Y., Nguyen, T.D., 2023. sFuzz2.0: storage-access pattern guided smart contract fuzzing. *J. Software : Evol. Process.* 36 (4), e2557.
- Wang, Q., Li, R., Wang, Q., Chen, S., 2021c. Non-fungible token (NFT): overview, evaluation, opportunities and challenges. *arXiv:2105.07447*.
- Wang, S., Zhang, C., Su, Z., 2019. Detecting nondeterministic payment bugs in ethereum smart contracts. *Proc. ACM Program. Lang.* 3 (OOPSLA). <https://doi.org/10.1145/3360615>
- Wang, S., Zhao, X., 2025. Contractsentry: a static analysis tool for smart contract vulnerability detection. *Autom. Software Eng.* 32 (1), 1.
- Wang, S.J., Pei, K., Yang, J., 2024d. SmartInv: multimodal learning for smart contract invariant inference. In: *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, pp. 2217–2235.
- Wang, X., Sun, J., Hu, C., Yu, P., Zhang, B., Hou, D., et al., 2022. EtherFuzz: mutation fuzzing smart contracts for TOD vulnerability detection. *Wireless Commun. Mobile Comput.* 2022, 1–8.
- Wang, X., Tian, S., Cui, W., 2024e. ContractCheck: checking ethereum smart contracts in fine-grained level. *IEEE Trans. Software Eng.* 50 (7), 1789–1806
- Wang, Y., Zhao, X., He, L., Zhen, Z., Chen, H., 2024f. ContractGNN: ethereum smart contract vulnerability detection based on vulnerability sub-graphs and graph neural networks. *IEEE Trans. Netw. Sci. Eng.* 11 (6), 6382–6395
- Wang, Z., Chen, J., Wang, Y., Zhang, Y., Zhang, W., Zheng, Z., 2024g. Efficiently detecting reentrancy vulnerabilities in complex smart contracts. *Proc. ACM Software Eng.* 1 (FSE), 161–181.
- Wang, Z., Dai, W., Choo, K.K.R., Jin, H., Zou, D., 2020h. FSFC: An input filter-based secure framework for smart contract. *J. Netw. Comput. Appl.* 154, 102530.

- Wang, Z., Dai, W., Li, M., Choo, K.-R.R., Zou, D., 2024i. DFier: a directed vulnerability verifier for ethereum smart contracts. *J. Netw. Comput. Appl.* 231, 103984.
- Wang, Z., Wen, B., Luo, Z., Liu, S., 2021d. M-a-r: a dynamic symbol execution detection method for smart contract reentry vulnerability. In: *Blockchain and Trustworthy Systems*. Springer Singapore, Singapore. Communications in Computer and Information Science, pp. 418–429.
- Web3 Security Community, 2025. DeFi hacks analysis - root cause. <https://wooded-meter-1d8.notion.site/0e85e02c5ed34df3855ea9f3ca40f53b?v=22e5e2c506ef4caeb40b4f78e23517ee>. Accessed: 2025-04-12.
- Wei, Z., Sun, J., Zhang, Z., Zhang, X., Yang, X., Zhu, L., 2023. Survey on quality assurance of smart contracts. *arXiv:2311.00270*.
- Wu, G., Wang, H., Lai, X., Wang, M., He, D., Chan, S., 2024a. A comprehensive survey of smart contract security: state of the art and research directions. *J. Netw. Comput. Appl.*, 103882.
- Wu, H., Qin, Y., Lin, B., Mao, X., Lei, Y., Zhang, Z., Wang, S., Zhang, H., 2021. Peculiar: smart contract vulnerability detection based on crucial data flow graph and pre-training techniques. <https://doi.org/10.1109/ISSRES2982.2021.00047>
- Wu, S., Li, Z., Yan, L., Chen, W., Jiang, M., Wang, C., Luo, X., Zhou, H., 2024b. Are we there yet? Unraveling the state-of-the-art smart contract fuzzers. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13.
- Wu, S., Yu, Z., Wang, D., Zhou, Y., Wu, L., Wang, H., Yuan, X., 2023. DeFiRanger: detecting deFi price manipulation attacks. *IEEE Trans. Dependable Secure Comput.* 21 (4), 4147–4161.
- Wu, Y., Xie, X., Peng, C., Liu, D., Wu, H., Fan, M., Liu, T., Wang, H., 2024c. AdvScanner: generating adversarial smart contracts to exploit reentrancy vulnerabilities using LLM and static analysis. In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, pp. 1019–1031.
- Wu, Z., Zhang, J., Gao, J., Li, Y., Li, Q., Guan, Z., Chen, Z., 2020. Kaya: a testing framework for blockchain-based decentralized applications. In: *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, pp. 826–829.
- Xie, X., Wang, H., Jian, Z., Fang, Y., Wang, Z., Li, T., 2023. Block-gram: mining knowledgeable features for efficiently smart contract vulnerability detection. *Digit. Commun. Netw.* 11 (1), 1–12.
- Xu, J., Vadgama, N., 2022. From banks to DeFi: the evolution of the lending market. Enabling the Internet of Value: How Blockchain Connects Global Businesses, 53–66.
- Xu, R., Tang, Z., Ye, G., Wang, H., Ke, X., Fang, D., Wang, Z., 2022. Detecting code vulnerabilities by learning from large-scale open source repositories. *J. Inf. Secur. Appl.* 69, 103293. <https://doi.org/10.1016/j.jisa.2022.103293>
- Xu, Y., Hu, G., You, L., Cao, C., 2021. A novel machine learning-based analysis model for smart contract vulnerability. *Secur. Commun. Netw.* 2021, 1–12.
- Xue, Y., Herlihy, M., 2021. Hedging against sore loser attacks in cross-chain transactions. In: *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pp. 155–164.
- Xue, Y., Ma, M., Lin, Y., Sui, Y., Ye, J., Peng, T., 2021. Cross-contract static analysis for detecting practical reentrancy vulnerabilities in smart contracts. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, p. 1029–1040. <https://doi.org/10.1145/3324884.3416553>
- Xue, Y., Ye, J., Zhang, W., Sun, J., Ma, L., Wang, H., Zhao, J., 2024. xFuzz: machine learning guided cross-contract fuzzing. *IEEE Trans. Dependable Secure Comput.* 21 (2), 515–529. <https://doi.org/10.1109/TDSC.2022.3182373>
- Yang, H., Gu, X., Chen, X., Zheng, L., Cui, Z., 2024a. CrossFuzz: cross-contract fuzzing for smart contract vulnerability detection. *Sci. Comput. Program.* 234, 103076.
- Yang, J., Zhao, X., Zhang, H., He, L., Wang, S., Gou, N., 2025. CSAFuzzer: fuzzing smart contracts combining with static analysis. *Empir. Software Eng.* 30 (3), 62.
- Yang, S., Chen, J., Huang, M., Zheng, Z., Huang, Y., 2024b. Uncover the premeditated attacks: detecting exploitable reentrancy vulnerabilities by identifying attacker contracts. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–12.
- Yang, Z., Keung, J., Zhang, M., Xiao, Y., Huang, Y., Hui, T., 2020. Smart contracts vulnerability auditing with multi-semantics. In: *2020 IEEE 44th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 892–901. <https://doi.org/10.1109/COMPSAC48688.2020.0-153>
- Yang, Z., Zhu, W., Yu, M., 2023. Improvement and optimization of vulnerability detection methods for ethernet smart contracts. *IEEE Access* 11, 78207–78223. <https://doi.org/10.1109/ACCESS.2023.3298672>
- Ye, J., Ma, M., Lin, Y., Ma, L., Xue, Y., Zhao, J., 2022. Vulpedia: detecting vulnerable ethereum smart contracts via abstracted vulnerability signatures. *J. Syst. Software* 192, 111410. <https://doi.org/10.1016/j.jss.2022.111410>
- Ye, J., Ma, M., Peng, T., Xue, Y., 2020. A software analysis based vulnerability detection system for smart contracts. In: Stan, J., Aneta, A.P.-M., Lech, M. (Eds.), *Integrating Research and Practice in Software Engineering*. Springer, pp. 69–81.
- Ye, M., Lin, X., Nan, Y., Wu, J., Zheng, Z., 2024a. Midas: mining profitable exploits in on-chain smart contracts via feedback-driven fuzzing and differential analysis. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 794–805.
- Ye, M., Nan, Y., Dai, H.-N., Yang, S., Luo, X., Zheng, Z., 2024b. FunFuzz: a function-oriented fuzzer for smart contract vulnerability detection with high effectiveness and efficiency. *ACM Trans. Software Eng. Method* 33 (7), 1–20.
- Yu, G., Nie, T.-Z., Li, X.-H., Zhang, Y.-F., Shen, D.-R., Bao, Y.-B., 2019. The challenge and prospect of distributed data management techniques in blockchain systems. *Chin. J. Comput.* 42, 1–27.
- Yu, L., Lu, J., Liu, X., Yang, L., Zhang, F., Ma, J., 2023a. PSCVFinder: a prompt-tuning based framework for smart contract vulnerability detection. In: *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, pp. 556–567.
- Yu, R., Shu, J., Yan, D., Jia, X., 2021a. ReDetect: reentrancy vulnerability detection in smart contracts with high accuracy. In: *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*, pp. 412–419. <https://doi.org/10.1109/MSN53354.2021.00069>
- Yu, R., Zhang, Y., Wang, Y., Liu, C., 2023b. TxMirror: when the dynamic EVM stack meets transactions for smart contract vulnerability detection. *Symmetry* 15 (7), 1345.
- Yu, X., Zhao, H., Hou, B., Ying, Z., Wu, B., 2021b. DeeSCVHunter: a deep learning-based framework for smart contract vulnerability detection. In: *2021 International Joint Conference on Neural Networks (IJCNN)*, pp. 1–8. <https://doi.org/10.1109/IJCNN52387.2021.9534324>
- Yulianto, S., Hendric Spits Warnars, H.L., Prabowo, H., Hidayanto, A.N., 2023. Security risks and best practices for blockchain and smart contracts: a systematic literature review. In: *2023 International Conference on Information Management and Technology (ICIMTech)*, pp. 1–6. <https://doi.org/10.1109/ICIMTech59029.2023.10278055>
- Zaazaa, O., Bakkali, H.E., 2023. Unveiling the landscape of smart contract vulnerabilities: a detailed examination and codification of vulnerabilities in prominent blockchains. *Int. J. Comput. Netw. Commun.* 15 (6), 55–75. <https://doi.org/10.5121/ijnc.2023.15603>
- Zaazaa, O., El Bakkali, H., 2023. A systematic literature review of undiscovered vulnerabilities and tools in smart contract technology. *J. Intell. Syst.* 32 (1), 20230038.
- Zeng, Q., He, J., Zhao, G., Li, S., Yang, J., Tang, H., Luo, H., 2022. EtherGIS: a vulnerability detection framework for ethereum smart contracts based on graph learning features. In: *2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC)*, pp. 1742–1749. <https://doi.org/10.1109/COMPSAC54236.2022.00277>
- Zeng, S., Zhang, H., Wang, J., Shi, K., 2023. SolGPT: a gpt-based static vulnerability detection model for enhancing smart contract security. In: *International Conference on Algorithms and Architectures for Parallel Processing*. Springer, pp. 42–62.
- Zhang, B., 2024. Towards finding accounting errors in smart contracts. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–13.
- Zhang, H., Zhang, W., Feng, Y., Liu, Y., 2023a. SVScanner: detecting smart contract vulnerabilities via deep semantic extraction. *J. Inf. Secur. Appl.* 75, 103484.
- Zhang, J., Gao, J., Li, Y., Chen, Z., Guan, Z., Chen, Z., 2022a. Xscope: hunting for cross-chain bridge attacks. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pp. 1–4.
- Zhang, J., Li, Y., Gao, J., Guan, Z., Chen, Z., 2023b. Siguard: detecting signature-related vulnerabilities in smart contracts. In: *Proceedings of the 45th International Conference on Software Engineering: Companion Proceedings*. IEEE Press, p. 31–35. <https://doi.org/10.1109/ICSE-Companion58688.2023.00019>
- Zhang, L., Chen, W., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H., 2022b. CBGRU: a detection method of smart contract vulnerability based on a hybrid model. *Sensors* 22 (9), 3577.
- Zhang, L., Li, Y., Jin, T., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H., 2022c. SPCBIG-EC: a robust serial hybrid model for smart contract vulnerability detection. *Sensors* 22 (12), 4621.
- Zhang, L., Wang, J., Wang, W., Jin, Z., Su, Y., Chen, H., 2022d. Smart contract vulnerability detection combined with multi-objective detection. *Comput. Netw.* 217, 109289.
- Zhang, L., Wang, J., Wang, W., Jin, Z., Zhao, C., Cai, Z., Chen, H., 2022e. A novel smart contract vulnerability detection method based on information graph and ensemble learning. *Sensors* 22 (9), 3581.
- Zhang, M., Zhang, X., Zhang, Y., Lin, Z., 2020a. {TXSPECTOR}: uncovering attacks in ethereum from transactions. In: *29th USENIX Security Symposium (USENIX Security 20)*, pp. 2775–2792.
- Zhang, P., Wang, B., Luo, X., Dong, H., 2024. SCAnoGenerator: automatic anomaly injection for ethereum smart contracts. *IEEE Trans. Software Eng.* 50 (11), 2983–3006. <https://doi.org/10.1109/TSE.2024.3464539>
- Zhang, Q., Wang, Y., Li, J., Ma, S., 2020b. EthPloit: from fuzzing to efficient exploit generation against smart contracts. In: *Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 116–126.
- Zhang, X., Sun, W., Xu, Z., Cheng, H., Cai, C., Cui, H., Li, Q., 2024c. EVM-Shield: in-contract state access control for fast vulnerability detection and prevention. *IEEE Trans. Inf. Forensics Secur.* 19, 2517–2532.
- Zhang, Y., Liu, D., 2022. Toward vulnerability detection for ethereum smart contracts using graph-matching network. *Future Internet* 14 (11), 326.
- Zhang, Y., Ma, S., Li, J., Li, K., Nepal, S., Gu, D., 2020c. SMARTSHIELD: automatic smart contract protection made easy. In: *Proceedings of the 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 23–34.
- Zhang, Z., Lei, Y., Yan, M., Yu, Y., Chen, J., Wang, S., Mao, X., 2023c. Reentrancy vulnerability detection and localization: a deep learning based two-phase approach. In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, New York, NY, USA, pp. 1–13. <https://doi.org/10.1145/3551349.3560428>
- Zhang, Z., Zhang, B., Xu, W., Lin, Z., 2023d. Demystifying exploitable bugs in smart contracts. In: *Proceedings of the 45th International Conference on Software Engineering*. IEEE Press, p. 615–627. <https://doi.org/10.1109/ICSE48619.2023.00061>
- Zhaoxuan, L., Lu, S., Zhang, R., Xue, R., Ma, W., Liang, R., Zhao, Z., Gao, S., 2022. SmartFast: an accurate and robust formal analysis tool for ethereum smart contracts. *Empir. Software Eng.* 27. <https://doi.org/10.1007/s10664-022-10218-2>
- Zheng, P., Su, B., Luo, X., Chen, T., Zhang, N., Zheng, Z., 2024a. Lent-sse: leveraging executed and near transactions for speculative symbolic execution of smart contracts. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 566–577.
- Zheng, P., Zheng, Z., Luo, X., 2022. Park: accelerating smart contract vulnerability detection via parallel-fork symbolic execution. In: *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*. Association for Comput-

- ing Machinery, New York, NY, USA, p. 740–751. <https://doi.org/10.1145/3533767.3534395>
- Zheng, Z., Su, J., Chen, J., Lo, D., Zhong, Z., Ye, M., 2024b. DAppSCAN: building large-scale datasets for smart contract weaknesses in dapp projects. *IEEE Trans. Software Eng.* 50 (6), 1360–1373.
- Zheng, Z., Zhang, N., Su, J., Zhong, Z., Ye, M., Chen, J., 2023. Turn the rudder: a beacon of reentrancy detection for smart contracts on ethereum. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 295–306. <https://doi.org/10.1109/ICSE48619.2023.00036>
- Zhong, Z., Zheng, Z., Dai, H.-N., Xue, Q., Chen, J., Nan, Y., 2024. PrettySmart: detecting permission re-delegation vulnerability for token behaviors in smart contracts. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pp. 1–12.
- Zhou, E., Hua, S., Pi, B., Sun, J., Nomura, Y., Yamashita, K., Kurihara, H., 2018. Security assurance for smart contract. In: 2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS), pp. 1–5. <https://doi.org/10.1109/NTMS.2018.8328743>
- Zhou, H., Milani Fard, A., Makanju, A., 2022. The state of ethereum smart contracts security: vulnerabilities, countermeasures, and tool support. *J. Cybersecur. Privacy* 2 (2), 358–378.
- Zhou, K., Cheng, J., Li, H., Yuan, Y., Liu, L., Li, X., 2021a. SC-VDM: a lightweight smart contract vulnerability detection model. In: Tan, Y., Shi, Y., Zomaya, A., Yan, H., Cai, J. (Eds.), *Data Mining and Big Data*. Springer Singapore, Singapore, pp. 138–149.
- Zhou, S., Yang, Z., Xiang, J., Cao, Y., Yang, Z., Zhang, Y., 2020. An ever-evolving game: evaluation of real-world attacks and defenses in ethereum ecosystem. In: 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, pp. 2793–2810. <https://www.usenix.org/conference/usenixsecurity20/presentation/zhou-shunfan>.
- Zhou, T., Liu, K., Li, L., Liu, Z., Klein, J., Bissyandé, T.F., 2021b. SMARTGIFT: learning to generate practical inputs for testing smart contracts. In: 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 23–34. <https://doi.org/10.1109/ICSME52107.2021.00009>
- Zhou, X., Jin, Y., Zhang, H., Li, S., Huang, X., 2016. A map of threats to validity of systematic literature reviews in software engineering. In: 2016 23rd Asia-Pacific Software Engineering Conference (APSEC), pp. 153–160. <https://doi.org/10.1109/APSEC.2016.031>
- Zhu, H., Yang, K., Wang, L., Xu, Z., Sheng, V.S., 2023. GraBit: a sequential model-based framework for smart contract vulnerability detection. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). IEEE, pp. 568–577.
- Zhu, H., Yang, L., Wang, L., Sheng, V.S., 2024. A survey on security analysis methods of smart contracts. *IEEE Trans. Serv. Comput.* 17 (6), 4522–4539.
- Zhukov, A., Korkhov, V., 2023. SmartGraph: static analysis tool for solidity smart contracts. In: Gervasi, O., Murgante, B., Rocha, A. M. A.C., Garau, C., Scorza, F., Karaca, Y., Torre, C.M. (Eds.), *Computational Science and Its Applications – ICCSA 2023 Workshops*. Springer Nature Switzerland, Cham, pp. 584–598.
- Zou, W., Lo, D., Kochhar, P.S., Le, X.-B.D., Xia, X., Feng, Y., Chen, Z., Xu, B., 2019. Smart contract development: challenges and opportunities. *IEEE Trans. Software Eng.* 47 (10), 2084–2106.